

The Sharpee Author and Developer Manual

Authoring Interactive Fiction with Sharpee

David Cornelson

The Sharpee Author and Developer Manual

Authoring Interactive Fiction with Sharpee

Published by David Cornelson

Copyright © 2026 David Cornelson. All rights reserved. No part of this book may be reproduced in any form without written permission from the publisher, except for brief quotations used in reviews.

First edition.

Sharpee is an open-source interactive-fiction authoring system. The Family Zoo example code in this book is real, compiled, and transcript-tested against the Sharpee platform.

About This Book

The first drafts of this book were generated by Claude. Every chapter was then reviewed, corrected, expanded, reorganized, and edited by David Cornelson, who also designed and implemented Sharpee.

In other words, the writing was produced by a large language model, but the opinions, architecture, examples, and technical decisions belong to a much smaller one.

Cover: *Celestial Globe with Clockwork*, Gerhard Emmoser (German, 1579). The Metropolitan Museum of Art, New York. Public domain.

- [Preface](#)
 - [0.1 Two perspectives, one example](#)
- [Introduction](#)
- [How to Read This Book](#)
 - [0.2 What kind of reader are you?](#)
 - [0.3 Two tracks](#)
 - [0.4 The “Under the Hood” box](#)
 - [0.5 How the zoo grows](#)
 - [0.6 Conventions](#)
- [Volume I — Getting Started](#)
- [1 Installing Sharpee: The CLI and Your First Project](#)
 - [1.1 Why Node and TypeScript?](#)
 - [1.2 Working in a terminal](#)
 - [1.3 Installing Node and npm](#)
 - [1.4 Installing the CLI](#)
 - [1.5 Creating a story project](#)
 - [1.6 Building the story](#)
 - [1.7 Playing it](#)
 - [1.8 The CLI at a glance](#)
 - [1.9 A TypeScript primer](#)
 - [1.10 Key takeaway](#)
- [2 Your First Room: Entities, Traits, and the World](#)
 - [2.1 The Story interface](#)
 - [2.2 Entities and traits](#)
 - [2.3 The shape of the file](#)
 - [2.4 Creating the player](#)
 - [2.5 Building the world](#)
 - [2.6 Placing things](#)
 - [2.7 Exposing the story](#)
 - [2.8 Try it](#)
 - [2.9 Key takeaway](#)
- [3 The Play Loop: How a Turn Works](#)
 - [3.1 The loop](#)
 - [3.2 Anatomy of a turn](#)

- [3.3 One turn, start to finish](#)
- [3.4 Key takeaway](#)
- [Volume II — Building a World](#)
- [4 Rooms & Navigation: Exits Wired in Pairs](#)
 - [4.1 Exits live on the room](#)
 - [4.2 Two rules that trip up everyone](#)
 - [4.3 Getting a trait back with `.get\(\)`](#)
 - [4.4 Putting it together](#)
 - [4.5 The going action is free](#)
 - [4.6 The map](#)
 - [4.7 Try it](#)
 - [4.8 Key takeaway](#)
- [5 Scenery & Portable Objects: Everything Is Portable by Default](#)
 - [5.1 Everything is portable by default](#)
 - [5.2 `EntityType.SCENERY` makes a thing fixed](#)
 - [5.3 When you still add `SceneryTrait` by hand](#)
 - [5.4 Aliases make objects findable](#)
 - [5.5 What you get for free](#)
 - [5.6 How taking actually decides](#)
 - [5.7 Putting it together](#)
 - [5.8 Try it](#)
 - [5.9 Key takeaway](#)
- [6 Containers & Supporters: What Holds What](#)
 - [6.1 Two kinds of holders](#)
 - [6.2 `ContainerTrait`](#)
 - [6.3 `SupporterTrait`](#)
 - [6.4 Capacity limits](#)
 - [6.5 Traits are composable](#)
 - [6.6 Try it](#)
 - [6.7 Key takeaway](#)
- [7 Openable Things, Locked Doors & Keys: Gating the Way Through](#)
 - [7.1 `OpenableTrait`](#)
 - [7.2 `LockableTrait`](#)
 - [7.3 `DoorTrait` and the `exit\_via` property](#)

- [7.4 Wiring it all together](#)
- [7.5 Try it](#)
- [7.6 Key takeaway](#)
- [8 Light & Dark: What the Player Can See](#)
 - [8.1 Dark rooms](#)
 - [8.2 LightSourceTrait](#)
 - [8.3 SwitchableTrait](#)
 - [8.4 The flashlight pattern](#)
 - [8.5 Other light-source patterns](#)
 - [8.6 Darkness as a gate](#)
 - [8.7 Wiring it into the zoo](#)
 - [8.8 Try it](#)
 - [8.9 Key takeaway](#)
- [9 The Map & Regions: Grouping Rooms](#)
 - [9.1 The map is the exits](#)
 - [9.2 Regions: grouping rooms](#)
 - [9.3 Region-wide properties](#)
 - [9.4 Crossing the boundary](#)
 - [9.5 Nesting and querying](#)
 - [9.6 Key takeaway](#)
- [Volume III — Making It Interactive](#)
- [10 The Standard Actions: The Four-Phase Model](#)
 - [10.1 The verbs you got for free](#)
 - [10.2 One shape for every action](#)
 - [10.3 Why the discipline matters](#)
 - [10.4 Key takeaway](#)
- [11 Scope & Visibility: What the Player Can Reach](#)
 - [11.1 What scope is](#)
 - [11.2 Degrees of access](#)
 - [11.3 Sight and darkness](#)
 - [11.4 Permissive parser, strict action](#)
 - [11.5 Key takeaway](#)
- [12 Readable Objects & Switchable Devices: Things That Carry State](#)
 - [12.1 ReadableTrait: what an object says](#)

- [12.2 Readable scenery: the info plaque](#)
- [12.3 Readable items: the brochure](#)
- [12.4 SwitchableTrait: a device with on/off state](#)
- [12.5 Switchable vs. openable](#)
- [12.6 Try it](#)
- [12.7 Key takeaway](#)
- [Volume IV — Custom Behavior](#)
- [13 Event Handlers: Reacting to What Happens](#)
 - [13.1 Every action emits an event](#)
 - [13.2 Two kinds of handler](#)
 - [13.3 Reading the event data](#)
 - [13.4 Setting up: the gift shop, the press, and remembering IDs](#)
 - [13.5 Reaction pattern: the goats eat the feed](#)
 - [13.6 Transformation pattern: put A in, get B out](#)
 - [13.7 Try it](#)
 - [13.8 Key takeaway](#)
- [14 Custom Actions: Teaching the Parser New Verbs](#)
 - [14.1 The four-phase action](#)
 - [14.2 A complete custom action: feeding](#)
 - [14.3 A second action: photographing](#)
 - [14.4 Telling the engine: getCustomActions](#)
 - [14.5 Teaching the parser: extendParser](#)
 - [14.6 Supplying the text: extendLanguage](#)
 - [14.7 Try it](#)
 - [14.8 Key takeaway](#)
- [15 Capability Dispatch: One Verb, Many Rules](#)
 - [15.1 When to reach for it](#)
 - [15.2 The three pieces](#)
 - [15.3 The dispatch action](#)
 - [15.4 Making the zoo's animals pettable](#)
 - [15.5 How it fits together](#)
 - [15.6 Try it](#)
 - [15.7 Key takeaway](#)

- [16 Custom Traits & Behaviors: Data and Logic, Kept Apart](#)
 - [16.1 Traits are data; behaviors are rules](#)
 - [16.2 Defining a custom trait](#)
 - [16.3 Defining the behavior](#)
 - [16.4 Putting the pair to work](#)
 - [16.5 Which tool, when?](#)
 - [16.6 Key takeaway](#)
- [Volume V — Words](#)
- [17 Extending the Grammar: Teaching New Sentence Shapes](#)
 - [17.1 How the parser matches](#)
 - [17.2 Where patterns go](#)
 - [17.3 Slots](#)
 - [17.4 Aliases: many patterns, one action](#)
 - [17.5 Prepositions and two slots](#)
 - [17.6 Constraining a slot](#)
 - [17.7 Priority](#)
 - [17.8 Key takeaway](#)
- [18 The Language Layer: Messages & Message IDs](#)
 - [18.1 Intent here, words there](#)
 - [18.2 Registering your text](#)
 - [18.3 Parameters](#)
 - [18.4 Naming message IDs](#)
 - [18.5 Why route everything through IDs](#)
 - [18.6 Key takeaway](#)
- [19 The Formatter Chain: Grammar in the Template, Not the Text](#)
 - [19.1 The problem with hardcoded articles](#)
 - [19.2 Formatter syntax](#)
 - [19.3 Why pass the entity, not its name](#)
 - [19.4 Capitalization and other text formatters](#)
 - [19.5 Verb agreement](#)
 - [19.6 Lists](#)
 - [19.7 Key takeaway](#)
- [Volume VI — Living Worlds](#)
- [20 Non-Player Characters: Actors That Take Turns](#)
 - [20.1 Creating an NPC entity](#)

- [20.2 Built-in behaviors](#)
- [20.3 Writing a custom behavior](#)
- [20.4 Registering the plugin and behaviors](#)
- [20.5 Try it](#)
- [20.6 Key takeaway](#)
- [21 Scenes: Named Windows of Story Time](#)
 - [21.1 What a scene is](#)
 - [21.2 Creating a scene](#)
 - [21.3 Querying a scene](#)
 - [21.4 Reacting to transitions](#)
 - [21.5 Common shapes](#)
 - [21.6 Scenes versus timed events](#)
 - [21.7 Key takeaway](#)
- [22 Turns, Timed Events & Daemons: Giving the World a Clock](#)
 - [22.1 How the scheduler ticks](#)
 - [22.2 Daemons: run every turn](#)
 - [22.3 Conditional daemons: react to state](#)
 - [22.4 Fuses: count down and fire](#)
 - [22.5 Giving the announcements their words](#)
 - [22.6 Try it](#)
 - [22.7 Key takeaway](#)
- [23 Scoring & Endgame: Winning the Game](#)
 - [23.1 The score ledger](#)
 - [23.2 Defining the scoring table](#)
 - [23.3 Awarding points as the player plays](#)
 - [23.4 The victory daemon](#)
 - [23.5 Try it](#)
 - [23.6 Key takeaway](#)
- [Volume VII — Presentation](#)
- [24 Channels: The Universal UI Surface](#)
 - [24.1 One surface for everything](#)
 - [24.2 A turn produces a packet](#)
 - [24.3 Channel modes](#)
 - [24.4 The channels you get for free](#)
 - [24.5 Capability negotiation](#)

- [24.6 Defining your own channel](#)
- [24.7 Key takeaway](#)
- [25 The Web Client: A Framework-Free UI](#)
 - [25.1 What the client is responsible for](#)
 - [25.2 How a turn reaches the screen](#)
 - [25.3 Commands flow back the same way](#)
 - [25.4 Why no framework](#)
 - [25.5 Overriding a renderer](#)
 - [25.6 Save, restore, and theme: for free](#)
 - [25.7 Key takeaway](#)
- [26 Decoration & Theming: Style Without HTML on the Wire](#)
 - [26.1 Decoration: styling without HTML on the wire](#)
 - [26.2 Platform vocabulary and the `sharpee-` namespace](#)
 - [26.3 Theming: one DOM, many skins](#)
 - [26.4 Applying themes](#)
 - [26.5 The status line](#)
 - [26.6 Key takeaway](#)
- [27 Media & Audio: Sight and Sound as Channels](#)
 - [27.1 Media is just more channels](#)
 - [27.2 Capability gating](#)
 - [27.3 The audio model](#)
 - [27.4 Supplying your own assets](#)
 - [27.5 Fades, not cuts](#)
 - [27.6 Room atmospheres in practice](#)
 - [27.7 Key takeaway](#)
- [Volume VIII — Shipping](#)
- [28 The Multi-File Story: Putting It All Together](#)
 - [28.1 When one file stops working](#)
 - [28.2 Organizing by concern, not by code type](#)
 - [28.3 Each file exports a builder and its IDs](#)
 - [28.4 The index wires it together](#)
 - [28.5 A feature that spans the files: after hours](#)
 - [28.6 Key takeaway](#)

- [29 Transcript Testing & Walkthroughs: Proving the Game Still Works](#)
 - [29.1 A test that reads like play](#)
 - [29.2 Two kinds of test](#)
 - [29.3 Asserting more than text](#)
 - [29.4 Handling variable outcomes](#)
 - [29.5 Running them](#)
 - [29.6 Key takeaway](#)
- [30 Saving & Restoring: State Lives in the World](#)
 - [30.1 State lives in the world, so it saves itself](#)
 - [30.2 The one thing you must save yourself](#)
 - [30.3 How the browser persists a save](#)
 - [30.4 Save formats change with a version](#)
 - [30.5 Key takeaway](#)
- [31 Building & Publishing: The Single-Player Browser Client](#)
 - [31.1 The author toolchain](#)
 - [31.2 Adding the browser client](#)
 - [31.3 Hosting it](#)
 - [31.4 Versioning](#)
 - [31.5 Publishing a story as a package](#)
 - [31.6 Key takeaway](#)
- [Appendix A — Architecture Map](#)
 - [31.7 The layers](#)
 - [31.8 The two flows](#)
 - [31.9 The universal surface](#)
 - [31.10 Where does this belong?](#)
- [Appendix B — Action Catalog](#)
- [Appendix C — Trait Catalog](#)
- [Appendix D — Message-ID Reference](#)
- [Appendix E — Grammar Reference](#)
 - [31.11 Pattern syntax](#)
 - [31.12 Core grammar rules](#)

Preface

Sharpee is a parser-based interactive-fiction authoring system. You write a story in TypeScript; Sharpee gives you a world model, a parser, a library of standard actions, and a presentation layer, and turns your code into a playable game you can run in a terminal, ship to the web, or host for many players at once.

This book teaches you to author with Sharpee by building one story, **Family Zoo**, from a single room to a complete, shippable game. It grows the zoo one concept at a time, and every version of the code in these pages is real, compiled, and transcript-tested.

0.1 Two perspectives, one example

Interactive-fiction systems often keep two layers distinct: the language you write a story in, and the language the system itself is built from. Inform 7, for instance, lets authors write in readable natural-language syntax and drops to Inform 6 for lower-level work, a deliberate split that makes the authoring surface welcoming, with a deeper layer waiting when you need it.

Sharpee makes a different tradeoff: the story you write and the library you call are the **same TypeScript**. The cost is that there's no gentle natural-language surface to start from; you're writing code from line one. The benefit is that there's no second language to learn when you want to see underneath. So this book carries both perspectives over a single running example, and you can move between them whenever you're ready:

- The **author's perspective** is the main narrative: building the zoo with Sharpee's traits, actions, and messages as a ready-made vocabulary.

- The **programmer's perspective** is a skippable deeper layer, the public API of the @sharpee/* library: the classes, methods, and types your author code is actually calling.

You can read the narrative straight through and ship a game without ever opening the deeper layer. Or you can follow both and come away understanding the library behind every line. The next chapter explains how to choose.

Introduction

Although this is a technical manual, it's very personal to me. Interactive Fiction quite literally altered my life and DNA. I was in high school playing baseball and getting mostly average grades in the classes I liked and one day I accidentally stumbled into a room where two students were hovered over paper terminals: one playing Crowther and Woods' Adventure (ADVENT), the other playing the MIT mainframe version of Zork (though the program was named DUNGEON).

I never returned to the baseball team or any other sport during the remaining two years of high school. Lightning had struck harder than the spider that transformed Peter Parker into Spider-Man.

Since that day I have off and on been very passionate about playing IF games and eventually even more passionate about building my own authoring system. That day also redirected my life towards a 40+ year career as a software engineer and architect from Milwaukee to Chicago.

Over the years I tried to dig into compilers and virtual machines, but that level of programming seemed to slip through my synapses, and I was never able to build anything useful. Add to that, my experience in the business world deepened my understanding of programming design patterns and architecture philosophy.

Then about three years ago ChatGPT arrived and later Claude. Generative AI allowed me to explore the concepts my previous capabilities had struggled to visualize. It didn't do the work for me. It augmented what I already knew about software and about Interactive Fiction and created an environment where I could direct my ideas into that bucket list item, a new parser-based interactive fiction platform called Sharpee.

Sharpee owes its implementation to everyone that stands out in the Interactive Fiction community. From Crowther and Woods', to the Infocom Implementors, to Graham Nelson, Andrew Plotkin, Emily Short,

Mike Roberts, Kent Tessman, Jon Ingold, Tara McGrew, and so many more. I would not possess the domain knowledge without the entire community from Usenet to ifMud to intfiction.org.

With the IF domain knowledge and my software engineering experience I have been able to leverage Anthropic's Claude to help design and create a working Interactive Fiction system in Typescript. This book is the first complete technical guide to building parser-based interactive fiction using Sharpee.

– David Cornelson

How to Read This Book

0.2 What kind of reader are you?

Sharpee has no natural-language authoring layer; you write TypeScript from Chapter 1. If that’s new to you, the primer there gets you going. Use this table to find your starting point:

If you’re...	Start here
An IF author comfortable with code (or willing to learn)	Read straight through: the Author’s Track below.
An experienced TypeScript / JavaScript developer	The Programmer’s Track : chapter bodies <i>and</i> every “Under the Hood” box.
New to programming	Read Chapter 1, including its TypeScript primer, carefully; then Chapters 2–9 on the Author’s Track, revisiting the primer as needed.
Here to understand or extend the engine	Skim Appendix A — Architecture Map , then follow the Programmer’s Track.

Whichever you are, the two tracks below are how the book serves you.

0.3 Two tracks

This book has one spine and two depths. Pick the **track** that fits why you’re reading.

- **Author’s Track**: read the chapter bodies straight through. This is everything you need to build and ship the Family Zoo. Skip every

“Under the Hood” box; you lose nothing required to finish the game.

- **Programmer’s Track:** read the chapter bodies *and* every “Under the Hood” box. The boxes form a guided tour of the public `@sharpee/*` library API behind the author’s code.

You can switch tracks at any time, and you can change your mind chapter to chapter. The narrative never depends on the boxes.

We use “track” for reading order. “Path” in this book always means an API or import path, never a way of reading.

0.4 The “Under the Hood” box

Whenever the author code uses a piece of the Sharpee library, an **Under the Hood** box shows the public surface of what was used: the class, its constructor, the relevant methods and types. It shows *signatures only*: never the implementation, never internal platform code, never anything you don’t get from the installed npm package. Here is the legend, and a live example:

UNDER THE HOOD: CONTAINERTRAIT • @SHARPEE/WORLD-MODEL

```
class ContainerTrait implements ITrait {
  constructor(data?: Partial<ContainerTrait>);
  capacity?: { maxWeight?: number; maxVolume?: number;
    maxItems?: number };
  isTransparent: boolean;
  enterable: boolean;
}
```

When you write `new ContainerTrait({ capacity: { maxItems: 10 } })`, you’re constructing this class. The standard `take` and `inventory` actions read its `capacity` to decide what fits.

If you're on the Author's Track, that box was safe to skip; the chapter body already told you the player can carry things. If you're on the Programmer's Track, you just saw the exact class behind it.

0.5 How the zoo grows

Every chapter advances the same story. The code is versioned: `v01` is a single room, `v02` adds a second room, and so on through the complete game. Each chapter's code matches a real, compiled version in the book's companion repository, under `tutorials/familyzoo/src/`; browse it on GitHub at <https://github.com/ChicagoDave/sharpee/tree/main/tutorials/familyzoo/src>. You don't need to clone anything to read along; anything you read here, you can run.

0.6 Conventions

- Commands you type into the game are shown in a transcript block, prefixed `>`.
- TypeScript you write appears in fenced code blocks.
- Under the Hood boxes are the only place library *signatures* appear; author code in the body is always code *you* write.

VOLUME I — GETTING STARTED

1 Installing Sharpee: The CLI and Your First Project

Before you can build the Family Zoo, you need a working toolchain: Node.js, an editor, and the `sharpee` command. This chapter gets you from an empty folder to a built story in a handful of commands, and along the way explains the foundation the whole platform stands on.

1.1 Why Node and TypeScript?

Most interactive-fiction systems give you a purpose-built authoring language: Inform's natural-language syntax, TADS's bespoke object language. Sharpee takes a different path: a story is plain **TypeScript** that runs on **Node.js**, the same mainstream toolchain used across the web- and server-software world.

That's a real trade-off. The cost is that you write actual code from the start; there's no gentler English-like surface to ease in on. The payoff is everything that comes *with* a mainstream ecosystem:

- **Real editors.** Visual Studio Code (or any TypeScript-aware editor) gives you autocomplete, inline documentation, and will flag as you type.
- **A type checker.** TypeScript catches whole classes of mistakes, such as a misspelled property or the wrong kind of value, at compile time instead of mid-playthrough.
- **The npm ecosystem.** Installing Sharpee, or any other library, is one command, and the platform itself is just a package your story depends on.
- **Ordinary tooling.** Version control, testing, formatting: the things millions of developers already use work on a Sharpee story unchanged.

In short, the “platform” isn’t a separate application you launch. It’s a library you import. The rest of this chapter installs that ecosystem and the Sharpee command that drives it.

1.2 Working in a terminal

Every Sharpee command in this book is typed into a **terminal**, a text window where you enter commands and read their output. If you’ve never opened one, here’s where to find it:

- **macOS:** open **Terminal** (Applications → Utilities, or press ⌘-Space and type “Terminal”).
- **Windows:** open **Windows Terminal** or **PowerShell** from the Start menu.
- **Linux:** open your terminal emulator (often **Ctrl-Alt-T**).

You’ll see a prompt waiting for input. Type a command, press **Enter** to run it, and its output appears on the lines below. Throughout the book, a shaded block like this is one or more commands to type at that prompt:

```
node --version
```

Two you’ll lean on constantly: `cd <folder>` moves you into a folder (so `cd my-game` steps into the project you just made), and running a command with no arguments, plain `sharpee`, prints its help.

1.3 Installing Node and npm

Node.js is the runtime that executes your story; **npm** is its package manager, and it installs *with* Node, so you don’t fetch it separately. If you’ve done any modern JavaScript work you likely have both already; if not, install them once:

1. Go to nodejs.org and download the **LTS** (“Long-Term Support”) build for your operating system.

2. Run the installer and accept the defaults. (If you expect to juggle several Node versions, a version manager like **nvm** or **fnm** is a fine alternative, but the installer is the simplest start.)
3. Open a terminal and confirm both tools are on your path:

```
node --version    # want v18.0.0 or newer
npm --version
```

If each prints a version number, you're set. `npm` is how you'll install Sharpee next, and any other library a story ever needs.

1.4 Installing the CLI

Everything you do to a story, scaffold it, compile it, bundle it, goes through one command, `sharpee`. It ships in the `@sharpee/devkit` package; install it once, globally:

```
npm install -g @sharpee/devkit
```

Now `sharpee` is on your path. The platform itself (`@sharpee/sharpee`: the world model, parser, standard library, and presentation layer) is *not* installed globally; each story pulls it in as an ordinary dependency, so different stories can pin different platform versions.

1.5 Creating a story project

`sharpee init` scaffolds a new project. On its own it walks you through a short wizard (story title, package ID, author, description), each question defaulting to a sensible value (the directory name, your username, and so on). Pass `-y` to accept every default and scaffold in one shot, which is what we'll do here:

```
sharpee init my-zoo -y
cd my-zoo
npm install
```

(Drop the `-y` if you'd rather answer the prompts yourself; the `my-zoo` argument just supplies the default for the first question.) `init` writes a small, complete starting point; `npm install` pulls down the platform it pins. After that you have:

```
my-zoo/
  src/
    index.ts      # a single starter file that hosts your story
    package.json  # pinned to the platform version devkit shipped with
    tsconfig.json # TypeScript config, set up for Sharpee
```

`src/index.ts` is where the story lives. Right now it's a starter stub; in the next chapter you'll replace it with the first room of the zoo. The platform arrives prebuilt in `node_modules`, so there is no platform to compile; only your story.

1.6 Building the story

```
sharpee build
```

`build` compiles `src/` and produces two things in `dist/`:

- `dist/index.js`: your compiled story.
- `dist/<id>.sharpee`: a single zipped **story bundle**, the unit you hand to a client or share for distribution.

Whenever you change the story, `sharpee build` again. (If a build ever looks stale, delete `dist/` and rebuild from a clean slate; `build` already checks that each step emits output, so a stale `dist/` rarely survives a rebuild on its own.)

1.7 Playing it

A story bundle isn't much fun to read as a `.sharpee` file; you want to *play* it. The simplest way is a self-contained web client. Add one to your project, then build:

```
sharpee init-browser      # adds src/browser-entry.ts
sharpee build             # now also emits a web client →
    dist/web/
```

`dist/web/` is a complete, static web page. Serve it with any static file server and open it in a browser:

```
python3 -m http.server -d dist/web
```

That's the loop you'll use throughout the book: edit `src/`, `sharpee build`, refresh the page, play.

Every snippet, online. As you work through the book, the full set of code snippets is browsable chapter by chapter, each step alongside the complete runnable file it builds up to, at sharpee.net/book-snippets.

1.8 The CLI at a glance

You've met most of these already; here's the full set you'll reach for as an author:

Command	What it does
<code>sharpee init [dir] [-y]</code>	Scaffold a new story project (<code>-y</code> skips the prompts)
<code>sharpee init-browser</code>	Add a web client (<code>src/browser-entry.ts</code>)
<code>sharpee build</code>	

Command	What it does
	Compile <code>src/</code> and emit the <code>.sharpee</code> bundle (and the web client, if present)
<code>sharpee build-browser</code>	Rebuild only the web client → <code>dist/web/</code>
<code>sharpee introspect</code>	Print the project's rooms, objects, and NPCs as JSON
<code>sharpee ifid</code>	Generate or validate an IFID (a story's unique identifier)

Run `sharpee` with no arguments any time to see the current list.

1.9 A TypeScript primer

The chapters ahead are full of TypeScript, but they lean on only a handful of its features. If the snippets below read clearly, you have everything you need. The book introduces the rest in context, and your editor fills the gaps.

TypeScript is JavaScript with types. A *type annotation*, the `: string` part below, tells the compiler what kind of value something holds, so it can flag a mistake before you run anything:

```
const title: string = 'Willowbrook Family Zoo';
let turns: number = 0;
```

`const` declares a value that won't be reassigned; `let`, one that will. You'll rarely write the annotations by hand, since TypeScript infers most of them, but you'll read them constantly.

Objects are bundles of named values, written `{ key: value }`. In a type, a `?` marks a property as optional:


```
const options = { isOpen: false, capacity: 10 };  
// e.g. `capacity?: number` means capacity may be left out
```

Classes are templates you make instances of with `new`. Most of Sharpee's building blocks, traits especially, are classes:

```
const light = new IdentityTrait({ name: 'flashlight' });
```

An interface is a contract. A class that implements an interface promises to provide everything the interface requires. Every Sharpee story is a class that implements the `Story` interface:

```
class MyStory implements Story { /* ... */ }
```

Imports bring in code from other packages; exports hand yours out:

```
import { IdentityTrait } from '@sharpee/world-model';  
export const story = new MyStory();
```

Functions can be written compactly as arrow functions, common in the short callbacks you'll pass to the platform:

```
items.some(item => item.name === 'feed');
```

That's the working vocabulary. Don't try to memorize it. You'll absorb it by building the zoo, one chapter at a time.

1.10 Key takeaway

Sharpee stories are TypeScript on Node, so the toolchain is the mainstream one: install Node (which brings npm), then the CLI with `npm install -g @sharpee/devkit`. Scaffold a project with `sharpee init`, write your story in `src/index.ts`, compile and bundle it with `sharpee build`, and play it by adding a web client (`sharpee init-`

browser) and serving dist/web/ . The Sharpee platform has everything you need to create an interactive fiction story.

2 Your First Room: Entities, Traits, and the World

You're standing at the entrance to the Willowbrook Family Zoo. There's a welcome sign and a ticket booth. You can look around and examine things, but there's nowhere to go yet.

That's it: one room, two things to look at. This is the simplest possible Sharpee story, and it's where the Family Zoo begins. By the end of this chapter you'll have a game you can play.

2.1 The Story interface

Every Sharpee story is a TypeScript class that implements the `Story` interface. The engine calls your class's methods during startup to build the world. Three things every story must provide:

1. `config`: metadata such as the story's title, author, version, and ID. The engine shows this as a banner when the game starts.
2. The `createPlayer(world)` method creates the player character. The engine calls this first. You create an entity, attach traits, and return it.
3. The `initializeWorld(world)` method builds the world, including: rooms, objects, connections. The engine calls this after `createPlayer`.

There are optional methods too (`extendParser`, `extendLanguage`, `onEngineReady`, and others), but a basic story needs none of them.

UNDER THE HOOD: STORY · @SHARPEE/ENGINE

```
interface Story {
    config: StoryConfig;
    initializeWorld(world: WorldModel): void;
    createPlayer(world: WorldModel): IEntity;
    // optional:
    getCustomActions?(): any[];
    getCustomVocabulary?(): CustomVocabulary;
    extendParser?(parser: Parser): void;
    isComplete?(): boolean;
}
```

The three required members are exactly the three things above. Everything else is optional and we'll meet the relevant ones in later chapters.

2.2 Entities and traits

Everything in a Sharpee game is an **entity**: rooms, objects, characters, doors, even the player. An entity by itself is just an empty shell with an ID. You make it useful by attaching **traits**: components that answer “what is this thing?” and “what can it *do*?”

You create entities with `world.createEntity(name, type)`. The type is a hint to the engine: `EntityType.ROOM`, `EntityType.ITEM`, `EntityType.ACTOR`, `EntityType.SCENERY`, and so on.

In this version we use five traits:

- **IdentityTrait**: a name, description, and aliases. Almost every entity has one. The `description` is what `examine` shows; the `aliases` are the alternative words the parser will accept.
- **ActorTrait**: marks an entity as a character. `isPlayer: true` tells the engine this is *the* player.

- **ContainerTrait** : lets an entity hold other entities. The player needs it to carry an inventory.
- **SceneryTrait** : marks an entity as fixed. The player can examine it but not take it.
- **RoomTrait** : marks an entity as a room, with exits and a darkness flag.

2.3 The shape of the file

Before the methods, the top of the file: the **imports**, the **config**, and the **class** that holds everything. Every symbol the story uses comes from one of two packages: `@sharpee/engine` (the `Story` contract and `StoryConfig`) and `@sharpee/world-model` (the world, entity types, and traits).

```

import { Story, StoryConfig } from '@sharpee/engine';
import { WorldModel, IFEntity, EntityType } from '@sharpee/world-
  model';
import {
  IdentityTrait,
  ActorTrait,
  ContainerTrait,
  RoomTrait,
  SceneryTrait,
} from '@sharpee/world-model';

const config: StoryConfig = {
  id: 'familyzoo',
  title: 'Family Zoo',
  author: 'Sharpee Tutorial',
  version: '0.1.0',
  description:
    'A small family zoo: Learn Sharpee one concept at a
      time.',
};

class FamilyZooStory implements Story {
  config = config;

  // createPlayer(world)    - fills in next
  // initializeWorld(world) - and after that
}

```

The two methods below are members of this `FamilyZooStory` class; they go where the comments are. We'll write each one, then assemble the whole file at the end.

The scaffolded `src/index.ts` isn't written exactly like the file we build here, but both are valid. The stub imports `Story` and `StoryConfig` from `@sharpee/sharpee` (a convenience barrel that re-exports the engine, world model, and parser as one package), where the book imports them from `@sharpee/engine` directly; the two names refer to the same types. The stub also defines the story as a plain **object literal** (`export const story: Story = { config, createPlayer, ... }`) rather than a `class`. An object literal and a class instance satisfy the `Story` interface identically. We use the class form throughout the book

because it gives the two methods a natural home and reads well as the story grows. Either style works; pick one and stay consistent.

2.4 Creating the player

The engine calls `createPlayer` first. Inside the class, you build the player like any other entity: create it, add traits, return it.

```
createPlayer(world: WorldModel): IEntity {
    const player = world.createEntity('yourself', EntityType.ACTOR);

    player.add(new IdentityTrait({
        name: 'yourself',
        description: 'Just an ordinary visitor to the zoo.',
        aliases: ['self', 'myself', 'me'],
        properName: true,
        article: '',
    }));

    player.add(new ActorTrait({ isPlayer: true }));

    player.add(new ContainerTrait({
        capacity: { maxItems: 10 },
    }));

    return player;
}
```

The `ContainerTrait` is what makes `take` and `inventory` work. Without it, the player has nowhere to put anything.

UNDER THE HOOD: CONTAINERTRAIT • @SHARPEE/WORLD-MODEL

```
class ContainerTrait implements ITrait {  
  constructor(data?: Partial<ContainerTrait>);  
  capacity?: { maxWeight?: number; maxVolume?: number;  
    maxItems?: number };  
  isTransparent: boolean;  
  enterable: boolean;  
}
```

The constructor takes a `Partial` of the trait's own fields, so you set only what you need (here, just `capacity`). The standard `take` action reads `capacity` to decide whether an item fits.

2.5 Building the world

`initializeWorld` runs after the player exists. Create a room, give it a `RoomTrait` and an `IdentityTrait`, then add some scenery.


```

initializeWorld(world: WorldModel): void {
    const entrance = world.createEntity('Zoo Entrance',
        EntityType.ROOM);

    entrance.add(new RoomTrait({ exits: {}, isDark: false }));
    entrance.add(new IdentityTrait({
        name: 'Zoo Entrance',
        description:
            'You stand before the gates of the Willowbrook Family Zoo.'
            +
            'A cheerful welcome sign arches over the entrance, and a'
            +
            'small ' +
            'ticket booth sits to one side.',
        aliases: ['entrance', 'gates', 'gate'],
        article: 'the',
    }));

    const sign = world.createEntity('welcome sign',
        EntityType.SCENERY);
    sign.add(new IdentityTrait({
        name: 'welcome sign',
        description: 'A brightly painted wooden sign welcomes you to'
            +
            'the zoo.',
        aliases: ['sign', 'wooden sign'],
        article: 'a',
    }));

    const booth = world.createEntity('ticket booth',
        EntityType.SCENERY);
    booth.add(new IdentityTrait({
        name: 'ticket booth',
        description:
            'A small wooden booth with a sliding glass window. A sign'
            +
            'in the ' +
            'window reads "Self-Guided Tours / No Ticket Needed'
            +
            'Today!'",
        aliases: ['booth', 'ticket booth', 'window'],
        article: 'a',
    }));

    world.moveEntity(sign.id, entrance.id);
    world.moveEntity(booth.id, entrance.id);

    const player = world.getPlayer();

```

```
if (player) world.moveEntity(player.id, entrance.id);
}
```

The ticket booth is built exactly like the sign: an entity and an `IdentityTrait` for its name and description. Both are typed `EntityType.SCENERY`, so they stay put, and both are placed in the entrance; now `examine booth` in the “Try it” list has something to find.

2.6 Placing things

Creating an entity doesn’t put it anywhere. You place it with `world.moveEntity(entityId, locationId)`, which puts the entity *inside* the location, whether that’s an object in a room, an item in a container, or the player in a room. Forget this step and the entity exists in the database but is invisible: the player can never reach it.

The player is no exception. `world.moveEntity(player.id, entrance.id)` is what sets the starting location.

2.7 Exposing the story

The two methods live inside the `FamilyZooStory` class from “The shape of the file.” The last piece is the bottom of the file: the engine loads your story from the module’s exports, so provide both a named `story` and a default. It then works however the module is loaded.

```
export const story: Story = new FamilyZooStory();
export default story;
```

The `: Story` annotation matters: it types `story` as the full `Story` interface, including the *optional* hooks like `extendParser` and `extendLanguage` you’ll add in later chapters. The browser client that `sharpee init-browser` generated in Chapter 1 checks for those hooks (`if (story.extendParser) ...`), and under TypeScript’s `strict` mode that check only compiles if the type knows the hooks *might* exist. Without

the annotation, `story` is typed as just `FamilyZooStory`, which doesn't have them yet, and the browser build fails. Annotate the export and every chapter builds.

That's the whole file: imports, `config`, the `class` with `createPlayer` and `initializeWorld`, and these exports.

2.8 Try it

> look	See the room description
> examine sign	Read the welcome sign
> examine booth	Look at the ticket booth
> take sign	Can't, it's scenery ("fixed in place")
> inventory	Check what you're carrying (nothing yet)

2.9 Key takeaway

A Sharpee story is a class with a config, a player creator, and a world initializer. The world is made of entities with traits. Place everything explicitly, or it won't exist.

3 The Play Loop: How a Turn Works

In the last chapter you built a room and played it: you typed a command, pressed Enter, and read a response. This chapter opens the hood on that exchange: what the engine does in the moment between the player’s keystroke and the next prompt. You won’t write any new code here. Understanding the loop is what makes everything after it make sense.

3.1 The loop

A running Sharpee story is a simple cycle, repeated until the player quits:

1. Show the player where they are and a prompt.
2. Wait for a typed command.
3. Run the **turn**: interpret the command and apply its effects.
4. Show the result.
5. Go back to step 2.

This is the **play loop**, and the engine provides it. You never write it; you supply the world it operates on. A “turn” is one trip through step 3, and that step has more going on inside it than it looks.

3.2 Anatomy of a turn

When the player types `examine sign` and presses Enter, the engine takes that line through a short pipeline. Each stage hands its result to the next.

3.2.1 Parse

The **parser** turns the raw text into a structured command: a verb and its noun phrases. `examine sign` becomes “the examining action, applied to

something the player called *sign*.” At this point “sign” is just words; no particular object is attached yet.

3.2.2 Resolve scope

Next the engine matches those words to actual entities, and only the ones the player can currently perceive. This is **scope**: the welcome sign in the room resolves; an object locked in another room does not. Scope is also what lets the player call things by their aliases, the `aliases` you put on an `IdentityTrait` back in Chapter 2. If nothing matches, the turn ends right here with a “you can’t see any such thing” message.

3.2.3 Run the action: validate, execute, report

With a verb and a resolved object in hand, the matching **action** runs. Every action, the standard ones and any you write later, moves through the same phases:

- **validate**: *can this happen?* Is the sign here, and is it something you can examine? If not, the action is **blocked** and reports why.
- **execute**: *change the world*, if anything changes. (Examining changes nothing; taking moves an item into your hands.)
- **report**: *record what happened*.

Volume III, *Making It Interactive*, takes these phases apart in detail. For now, the shape (check, change, report) is all you need.

3.2.4 Events become text

Here’s the part that surprises newcomers: **actions never print anything**. Instead, the report phase emits **events** (small records of what happened), and each one carries a *message id*, not a finished sentence:

```
context.event('if.event.taken', {  
  messageId: 'if.action.taken.success',  
  params: { item: 'brass key' },  
});
```

(That snippet is illustrative; the zoo has no brass key. It just shows the *shape* of an event: a type, a message id, and parameters.)

If you come from web or app development, set one expectation aside: these aren't notifications dispatched to listeners, and nothing subscribes to them. They're just records the action *reports*, collected as the turn runs.

At the end of the turn the engine's **prose pipeline** consumes the reported events: it looks each message id up in the language layer, runs each through the text formatters, and renders the actual words the player reads.

Why the indirection? Because it keeps every player-facing word in one place. The same event can be rendered in another language, restyled in different prose, or suppressed entirely. When a room is dark, a “you see...” event is filtered out before it ever becomes text (that's *Light & Dark*, later in Volume II). Volume V, *Words*, is devoted to that language layer.

3.2.5 Everyone else takes a turn

A turn isn't only the player's. Once the player's action resolves, the rest of the world gets to move: NPCs act, and timed events (countdowns and background processes) tick forward. The zookeeper's patrol and the parrot's squawk in Volume VI happen here. Then the engine shows the turn's result and returns to the prompt.

3.3 One turn, start to finish

Put it together with a single command in the one-room zoo from Chapter 2:

```
> examine sign
```

- **Parse:** the examining action, applied to “sign.”
- **Scope:** “sign” resolves to the welcome sign in the room.
- **Action:** examining validates (the sign is here and in scope), executes (nothing to change), and reports an event carrying the sign’s description.
- **Text:** the engine renders that event into the sign’s description, and it prints.
- **Other actors:** nothing else lives in this tiny world yet, so the turn ends.
- The prompt returns, and the loop comes back around.

Every command you type follows that same path, whether it’s `look`, `take key`, or a custom verb you invent ten chapters from now.

3.4 Key takeaway

A turn is a short pipeline: parse the text, resolve the words to objects in scope, run the matching action (validate → execute → report), turn the resulting events into text, then let NPCs and timed events move. The engine drives this loop; your job is to supply the world it runs on. Carry one idea forward above the rest: **actions emit events, not text**. The words are produced later, by the language layer, and that single separation is what makes a Sharpee story translatable, restylable, and able to stay quiet in the dark.

VOLUME II — BUILDING A WORLD

4 Rooms & Navigation: Exits Wired in Pairs

One room is a demo, not a game. In this chapter the zoo grows to four locations: the Zoo Entrance, a Main Path, a Petting Zoo, and an Aviary. The player can walk between them with compass directions: north, south, east, west.

Everything from the first chapter is still here. We're adding rooms and the connections between them.

4.1 Exits live on the room

Rooms are connected through **exits** on the `RoomTrait`. Each exit maps a compass direction to a destination, which is just the ID of another room:

```
entranceRoom.exits = {  
    [Direction.SOUTH]: { destination: mainPath.id },  
};
```

Read that as: “when the player types `south` in the entrance, move them to the main path.” The `Direction` enum gives you all the standard IF directions: `NORTH`, `SOUTH`, `EAST`, `WEST`, `UP`, `DOWN`, the four diagonals, and `IN` / `OUT`. The parser already understands the short forms (`n`, `se`, and so on), so you never spell those out yourself.

4.2 Two rules that trip up everyone

Exits are one-way. This is the single most common beginner mistake. If the entrance has a south exit to the main path, the player can walk south, but they *cannot* walk back unless the main path also has a north exit to the entrance. Always wire exits in pairs:

```

entranceRoom.exits = {
  [Direction.SOUTH]: { destination:
    mainPath.id }, // entrance → path
};
mainPathRoom.exits = {
  [Direction.NORTH]: { destination: entrance.id }, // path →
    entrance (the way back)
};

```

Forget the return exit and the player gets stranded.

Create first, connect second. An exit needs its destination’s ID, and you can’t reference a room that doesn’t exist yet. So build every room with empty exits first, then wire the exits once they all exist.

4.3 Getting a trait back with `.get()`

In the first chapter we only ever *added* traits. To wire exits we need to read a trait back off an entity, which is what `.get()` does:

```

const entranceRoom = entrance.get(RoomTrait)!;
entranceRoom.exits = { /* ... */ };

```

The `!` is a non-null assertion: “I know this room has a `RoomTrait`.” If you aren’t certain an entity has the trait, check instead of asserting:

```

const roomTrait = entrance.get(RoomTrait);
if (roomTrait) {
  roomTrait.exits = { /* ... */ };
}

```

4.4 Putting it together

This chapter adds one new import, `Direction`, to the world-model line from Chapter 2:

```
import {  
    WorldModel, IFEntity, EntityType, Direction,  
} from '@sharp/ee/world-model';
```

Here is the complete `initializeWorld` for this version. It replaces the single-room one from Chapter 2: four rooms with full descriptions, the exits wired in both directions, the scenery for every room (your Chapter 2 welcome sign and ticket booth, plus three new objects for the new rooms), and the player placed at the entrance.

```

initializeWorld(world: WorldModel): void {
    // Step 1: create every room first, with empty exits.
    const entrance = world.createEntity('Zoo Entrance',
        EntityType.ROOM);
    entrance.add(new RoomTrait({ exits: {}, isDark: false }));
    entrance.add(new IdentityTrait({
        name: 'Zoo Entrance',
        description:
            'You stand before the gates of the Willowbrook Family Zoo.
            A cheerful ' +
            'welcome sign arches over the entrance, and a small ticket
            booth sits ' +
            'to one side. The main path leads south into the zoo
            grounds.',
        aliases: ['entrance', 'gates', 'gate'],
        article: 'the',
    }));

    const mainPath = world.createEntity('Main Path',
        EntityType.ROOM);
    mainPath.add(new RoomTrait({ exits: {}, isDark: false }));
    mainPath.add(new IdentityTrait({
        name: 'Main Path',
        description:
            'A wide gravel path winds through the heart of the zoo.
            Colorful ' +
            'direction signs point every which way. To the east, a
            white picket ' +
            'fence surrounds the petting zoo. To the west, a tall mesh
            enclosure ' +
            'rises above the treetops, the aviary. The entrance gates
            are back ' +
            'to the north.',
        aliases: ['path', 'main path', 'gravel path'],
        article: 'the',
    }));

    const pettingZoo = world.createEntity('Petting Zoo',
        EntityType.ROOM);
    pettingZoo.add(new RoomTrait({ exits: {}, isDark: false }));
    pettingZoo.add(new IdentityTrait({
        name: 'Petting Zoo',
        description:
            'A cheerful open-air enclosure filled with friendly
            animals. Pygmy ' +

```

```

        'goats trot around nibbling at
        visitors\' shoelaces, while a pair of ' +
        'fluffy rabbits hop near a hay bale. The main path is back
        to the west.',
    aliases: ['petting zoo', 'petting area', 'pen'],
    article: 'the',
  }));

const aviary = world.createEntity('Aviary', EntityType.ROOM);
aviary.add(new RoomTrait({ exits: {}, isDark: false }));
aviary.add(new IdentityTrait({
  name: 'Aviary',
  description:
    'You step inside a soaring mesh dome. Brilliantly colored
    parrots ' +
    'chatter from rope perches, and a toucan eyes you curiously
    from a ' +
    'branch overhead. The exit back to the main path is to the
    east.',
  aliases: ['aviary', 'bird house', 'dome'],
  article: 'the',
}));

// Step 2: wire exits now that every room exists.
entrance.get(RoomTrait)!.exits = {
  [Direction.SOUTH]: { destination: mainPath.id },
};
mainPath.get(RoomTrait)!.exits = {
  [Direction.NORTH]: { destination: entrance.id },
  [Direction.EAST]: { destination: pettingZoo.id },
  [Direction.WEST]: { destination: aviary.id },
};
pettingZoo.get(RoomTrait)!.exits = {
  [Direction.WEST]: { destination: mainPath.id },
};
aviary.get(RoomTrait)!.exits = {
  [Direction.EAST]: { destination: mainPath.id },
};

// Step 3: scenery. The welcome sign and ticket booth from
    Chapter 2 stay
    // in the entrance. The three new rooms get scenery of their
    own. Each is the
    // same pattern you already know: an entity, an IdentityTrait,
    and a moveEntity
    // to place it. The EntityType.SCENERY type makes each one
    fixed.

```

```

const sign = world.createEntity('welcome sign',
    EntityType.SCENERY);
sign.add(new IdentityTrait({
    name: 'welcome sign',
    description: 'A brightly painted wooden sign welcomes you to
        the zoo.',
    aliases: ['sign', 'welcome sign', 'wooden sign'],
    article: 'a',
}));
world.moveEntity(sign.id, entrance.id);

const booth = world.createEntity('ticket booth',
    EntityType.SCENERY);
booth.add(new IdentityTrait({
    name: 'ticket booth',
    description:
        'A small wooden booth with a sliding glass window reading '
        +
        '"Self-Guided Tours / No Ticket Needed Today!'",
    aliases: ['booth', 'ticket booth', 'window'],
    article: 'a',
}));
world.moveEntity(booth.id, entrance.id);

const directionSigns = world.createEntity('direction signs',
    EntityType.SCENERY);
directionSigns.add(new IdentityTrait({
    name: 'direction signs',
    description:
        'A cluster of brightly colored arrow signs nailed to a
        wooden post. ' +
        'They point to: PETTING ZOO (east), AVIARY (west), ' +
        'REPTILE HOUSE (south -> coming soon!), and EXIT (north).',
    aliases: ['signs', 'direction signs', 'arrow signs', 'post'],
    article: 'some',
    grammaticalNumber: 'plural',
}));
world.moveEntity(directionSigns.id, mainPath.id);

const goats = world.createEntity('pygmy goats',
    EntityType.SCENERY);
goats.add(new IdentityTrait({
    name: 'pygmy goats',
    description:
        'Three pygmy goats with stubby legs and rectangular pupils,
        clearly ' +
        'hoping you have food.',

```

```

    aliases: ['goats', 'pygmy goats', 'goat'],
    article: 'some',
    grammaticalNumber: 'plural',
  }));
world.moveEntity(goats.id, pettingZoo.id);

const toucan = world.createEntity('toucan', EntityType.SCENERY);
toucan.add(new IdentityTrait({
  name: 'toucan',
  description:
    'A Toco toucan with an enormous orange-and-black bill. It
    regards ' +
    'you with one intelligent eye.',
  aliases: ['toucan', 'bird', 'toco toucan'],
  article: 'a',
}));
world.moveEntity(toucan.id, aviary.id);

// Step 4: place the player at the entrance, as in Chapter 2.
const player = world.getPlayer();
if (player) world.moveEntity(player.id, entrance.id);
}

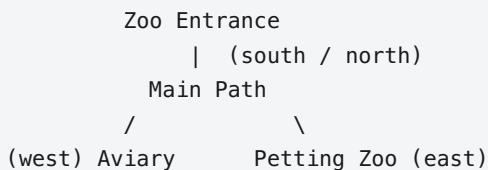
```

The method is complete; nothing is abbreviated with an “as before” comment. The `direction signs` on the Main Path are what `examine signs` in the “Try it” list below reads.

4.5 The going action is free

You don’t write any movement code. Sharpee’s standard library includes a **going** action that, when the player types a direction, looks up the current room’s exits, finds the matching one, moves the player to the destination, and prints the new room’s description. Wiring the exits is the whole job.

4.6 The map



4.7 Try it

> south	Walk to the Main Path
> examine signs	Read the direction signs
> east	Walk to the Petting Zoo
> west	Back to the Main Path
> west	Walk to the Aviary
> east	Back to the Main Path
> north	Back to the Zoo Entrance

4.8 Key takeaway

Rooms are connected by exits on `RoomTrait`, each mapping a `Direction` to a destination room ID. Create every room before connecting them, and always wire exits in both directions, otherwise the player will get stuck.

5 Scenery & Portable Objects: Everything Is Portable by Default

A world you can only walk through is a stage set. In this chapter the zoo gains two kinds of things: **scenery** you can examine but never carry off (fences, benches, animals) and **portable items** the player can take, pocket, and drop (a zoo map, a souvenir penny, a bag of feed). Together they're the difference between a room description and a place you can rummage through.

The surprising part is how little code each one takes. One of them takes *no new trait at all*.

5.1 Everything is portable by default

Here is Sharpee's central rule about objects, and it catches everyone the first time: **an entity is takeable unless you say otherwise**. Create something with an `IdentityTrait` and nothing else, and the player can pick it up, carry it between rooms, and drop it wherever they like. There is no `PortableTrait`, because portability isn't a feature you add. It's the starting state.

So a souvenir penny needs only its identity and a home:

```
const penny = world.createEntity('souvenir penny',
    EntityType.ITEM);
penny.add(new IdentityTrait({
  name: 'souvenir penny',
  description: 'A flattened copper penny stamped with a smiling
    elephant.',
  aliases: ['penny', 'coin', 'souvenir'],
}));
world.moveEntity(penny.id, mainPath.id);
```

That's a complete, takeable object. `EntityType.ITEM` is the type label for a generic portable thing; the `IdentityTrait` gives it a name, description, and aliases. No trait is needed to make it carryable; that's the default.

5.2 EntityType.SCENERY makes a thing fixed

Most of the things in a room are *not* meant to be carried. You don't want the player stuffing a park bench into a backpack or wandering off with the iron fence. Create a fixed thing as `EntityType.SCENERY` and it comes with a `SceneryTrait` already attached. That trait does exactly one thing: it blocks the taking action.

```
const fence = world.createEntity('iron fence',
    EntityType.SCENERY);
fence.add(new IdentityTrait({
  name: 'iron fence',
  description: 'A tall wrought-iron fence with animal
    silhouettes.',
  aliases: ['fence', 'iron fence', 'railing'],
}));
world.moveEntity(fence.id, entrance.id);
```

Now `take fence` gives the player “*iron fence is fixed in place.*” But `examine fence` still works: scenery blocks *taking*, not *looking*. The entity keeps its `IdentityTrait`, so its description is always readable.

The mistake everyone makes once: a fixed thing that *isn't* typed `EntityType.SCENERY`. The scenery type pins it for you, but a container, a supporter, or an animal you made an `ACTOR` is portable by default. If the player can pocket your feed dispenser, it has no `SceneryTrait` and you need to add one by hand.

5.3 When you still add `SceneryTrait` by hand

Typing a thing `EntityType.SCENERY` is the usual way to fix it, and it is enough on its own. You reach for an explicit `SceneryTrait` in only two cases:

- **A fixed thing of another type.** A feed dispenser is a `CONTAINER` and a park bench is a `SUPPORTER`; those types don't arrive fixed, so you add a `SceneryTrait` to pin them in place.
- **A custom refusal.** A plain `SceneryTrait` gives the standard "fixed in place" line; construct it with your own message to say something specific.

Entity type	Fixed by default	Example
<code>EntityType.ITEM</code>	No (portable)	Maps, keys, coins
<code>EntityType.SCENERY</code>	Yes (gets <code>SceneryTrait</code>)	Fences, benches, animals
<code>EntityType.CONTAINER</code> / <code>SUPPORTER</code>	No (add <code>SceneryTrait</code> to fix)	Dispensers, shelves

A rule of thumb: if you'd find it strange for the player to put a thing in their pocket, make it `EntityType.SCENERY`.

5.4 Aliases make objects findable

Whether takeable or fixed, every object should answer to more than its exact name. If a room mentions "a wrought-iron fence," the player might type `examine fence`, `examine railing`, or `examine wrought-iron fence`, and all of them should land:

```
aliases: ['fence', 'iron fence', 'wrought-iron fence', 'railing'],
```

The other easy miss: thin aliases. A player who can see a thing in the description but can't refer to it the way they'd naturally

say it will assume it isn't really there. Be generous: every noun in your prose is a word the player may type.

5.5 What you get for free

Because portability is built in, so are the verbs that go with it. The standard library handles the whole inventory vocabulary without a line of code from you:

Player types What happens

take map	Moves the map from the room into the player's inventory
drop map	Moves the map from inventory to the current room
inventory / i	Lists everything the player is carrying
take all	Takes every portable object in the room
drop all	Drops everything the player is holding

When the player carries an item and walks to a new room, the item travels with them. Carried things live with the player's default ability to 'contain' items, so they go wherever the player goes. Loose portable objects left on the floor are listed after the room description:

Main Path

A wide gravel path winds through the heart of the zoo...

You can see a souvenir penny here.

Scenery is *not* listed this way; it's expected to be named in the room's description prose, where it belongs.

5.6 How taking actually decides

The whole portable-vs-fixed distinction comes down to one check. When the player types `take map`:

1. The **parser** finds the entity named "map" in the current room.

2. The **taking** action asks: does this entity have `SceneryTrait`?
 - If yes → blocked: *“The map is fixed in place.”*
 - If no → it proceeds.
3. The action moves the entity into the player:
`world.moveEntity(map.id, player.id)`.
4. The player sees *“Taken.”*

That’s the entire rule. Portable or not is simply: *does it have SceneryTrait*? Creating a thing as `EntityType.SCENERY` is just the quickest way to give it one.

5.7 Putting it together

Fill each room with scenery for atmosphere, then scatter a few takeable items. Scenery is typed `EntityType.SCENERY`, which fixes it in place; items get nothing extra:

```

// Scenery: the SCENERY type fixes it in place, examinable,
// mentioned in room prose.
const fence = world.createEntity('iron fence',
    EntityType.SCENERY);
fence.add(new IdentityTrait({
    name: 'iron fence',
    description: 'A tall wrought-iron fence with animal
        silhouettes.',
    aliases: ['fence', 'iron fence', 'railing'],
}));
world.moveEntity(fence.id, entrance.id);

// More scenery: a pair of rabbits in the Petting Zoo, beside the
// goats.
const rabbits = world.createEntity('rabbits', EntityType.SCENERY);
rabbits.add(new IdentityTrait({
    name: 'rabbits',
    description:
        'A pair of Holland Lop rabbits with floppy ears and twitching
        noses, ' +
        'one pure white and the other brown and cream.',
    aliases: ['rabbits', 'rabbit', 'bunnies', 'bunny'],
    article: 'some',
    grammaticalNumber: 'plural',
}));
world.moveEntity(rabbits.id, pettingZoo.id);

// A takeable item: no SceneryTrait, so it's portable by default.
const zooMap = world.createEntity('zoo map', EntityType.ITEM);
zooMap.add(new IdentityTrait({
    name: 'zoo map',
    description: 'A colorful folding map of the zoo, a heart drawn
        around the petting zoo in crayon.',
    aliases: ['map', 'zoo map', 'folding map'],
}));
world.moveEntity(zooMap.id, entrance.id);

// A second takeable item, in the Petting Zoo this time.
const animalFeed = world.createEntity('bag of animal feed',
    EntityType.ITEM);
animalFeed.add(new IdentityTrait({
    name: 'bag of animal feed',
    description:
        'A small brown paper bag of dried corn and pellets. The label
        reads ' +
        '"ZOO SNACKS: Safe for goats, rabbits, and birds." It rustles
        invitingly.',

```

```

    aliases: ['feed', 'animal feed', 'bag of feed', 'bag', 'corn',
              'pellets'],
  }));
world.moveEntity(animalFeed.id, pettingZoo.id);

```

The souvenir penny from earlier in the chapter sits on the Main Path, and the **pygmy goats** you placed in the Petting Zoo back in Chapter 4 are scenery, so every object the “Try it” walkthrough touches is now in the world: the map and penny are portable, the feed waits in the Petting Zoo, and the goats stay put.

Plural-named scenery: the rabbits get `grammaticalNumber: 'plural'`. Sharpee’s messages agree in number with the entity, so this is what makes `take rabbits` report “The rabbits are fixed in place.” rather than “is”. Set it on anything with a plural name (*pygmy goats*, *direction signs*, *flower beds*) and the generated prose stays grammatical. (See Chapter 19 for how the message templates choose the verb.) Authors using the `object()` builder write `.plural()` instead.

5.8 Try it

> look	Notice the zoo map on the ground
> take map	Pick up the map
> inventory	See what you're carrying
> examine fence	You can look at scenery...
> take fence	...but "iron fence is fixed in place."
> south	Walk to the Main Path (map comes with you)
> take penny	Pick up the souvenir penny
> drop map	Leave the map here
> look	Map is now on the ground in Main Path
> east	Go to the Petting Zoo
> take feed	Pick up the bag of animal feed
> take goats	Can't: they're scenery!

5.9 Key takeaway

Items are portable by default: `EntityType.ITEM` plus an `IdentityTrait` is a complete takeable object, no special trait required. `SceneryTrait` *removes* portability; it's what makes fences, benches, and animals fixed in place while still examinable. Reach for scenery on anything the player shouldn't pocket, and give every object generous aliases so it can be named the way a player would say it.

6 Containers & Supporters: What Holds What

So far the zoo has things you walk past and things you carry. This chapter adds things that hold *other* things: a red backpack the player can store items inside, a feed dispenser bolted to a post at the petting zoo, and a park bench you can set objects on top of. Two new traits cover both cases, and the difference between them is the difference between *in* and *on*.

6.1 Two kinds of holders

Sharpee gives you two traits for entities that hold other entities:

- **ContainerTrait** : things go *inside*. Backpacks, boxes, drawers, dispensers.
- **SupporterTrait** : things go *on top*. Tables, shelves, benches, pedestals.

The parser sorts them out by preposition. “put X **in** Y” routes to the container; “put X **on** Y” routes to the supporter. You never write that logic; you just declare which kind of holder each object is.

```
> put map in backpack
You put zoo map in backpack.
> put penny on bench
You put souvenir penny on park bench.
```

6.2 ContainerTrait

A container holds entities inside it. You’ve already met this trait once: the player carries an inventory because the player entity itself has a **ContainerTrait**. In this chapter we put it on ordinary objects too.

```

const backpack = world.createEntity('backpack',
    EntityType.CONTAINER);
backpack.add(new IdentityTrait({
    name: 'backpack',
    description: 'A small red canvas backpack.',
    aliases: ['backpack', 'bag', 'pack'],
}));
backpack.add(new ContainerTrait({
    capacity: { maxItems: 5 },    // holds up to 5 things
}));
world.moveEntity(backpack.id, entrance.id);

```

Because we *didn't* add `SceneryTrait`, this backpack is portable, and that's where containers get interesting. (As always, the `moveEntity` is what puts it in the world; without it the backpack exists nowhere and `take backpack` finds nothing.)

6.2.1 Portable vs fixed containers

A container is fixed or portable on exactly the rule from the last chapter: `SceneryTrait` or not.

Container	Portable?	How
Backpack	Yes	No <code>SceneryTrait</code> , so the player can take it
Feed dispenser	No	Has <code>SceneryTrait</code> , so it is fixed to its post

A portable container moves as a unit: the player takes the backpack and everything inside comes with it. A bag of five items counts as **one** item in the player's own inventory. A fixed container, like the dispenser below, is for built-in storage the player can reach into but never walk off with:

```

const dispenser = world.createEntity('feed dispenser',
    EntityType.CONTAINER);
dispenser.add(new IdentityTrait({
    name: 'feed dispenser',
    description:
        'A coin-operated feed dispenser mounted on a wooden post, its
        glass ' +
        'globe half full of pellets.',
    aliases: ['dispenser', 'feed dispenser', 'machine', 'globe'],
}));
dispenser.add(new ContainerTrait({ capacity: { maxItems: 3 } }));
dispenser.add(new SceneryTrait()); // can't take the dispenser
    itself
world.moveEntity(dispenser.id, pettingZoo.id);

```

A fixed container still needs its own `IdentityTrait` (the name and aliases are how the player refers to it, as in `examine dispenser`) and its `moveEntity`, which bolts it into the Petting Zoo.

6.3 SupporterTrait

A supporter is the surface counterpart: things rest *on* it rather than *in* it.

```

const parkBench = world.createEntity('park bench',
    EntityType.SUPPORTER);
parkBench.add(new IdentityTrait({
    name: 'park bench',
    description:
        'A weathered wooden bench worn smooth by decades of
        visitors.',
    aliases: ['bench', 'park bench', 'seat'],
}));
parkBench.add(new SupporterTrait({ capacity: { maxItems: 3 } }));
parkBench.add(new SceneryTrait()); // can't take the bench
    itself
world.moveEntity(parkBench.id, mainPath.id);

```

The bench goes on the Main Path, so `put penny on bench` in the “Try it” walkthrough has a surface to use. The key behavioral difference from a container: supporters are always **open**, so whatever sits on a bench is

visible without any special action. Containers, as the next chapter shows, can be opened and closed to hide their contents.

6.4 Capacity limits

Both traits accept the same `capacity` option:

```
capacity: { maxItems: 5 }
```

Try to put a sixth item in a five-item container and the player gets a “can’t fit” message. Capacity isn’t just bookkeeping. Limited space is the raw material of puzzles, forcing the player to choose what to carry and what to leave behind.

The mistake everyone makes once: leaving `capacity` off. A container or supporter with no `capacity` has *no limit*; it will swallow the entire zoo. If you want a bound, set `maxItems` explicitly.

6.5 Traits are composable

The real lesson of this chapter isn’t either trait on its own. It’s that an entity can wear several traits at once, and they stack cleanly:

- The park bench is a `SupporterTrait` (things go on it) **and** a `SceneryTrait` (you can’t take it).
- The feed dispenser is a `ContainerTrait` (things go in it) **and** a `SceneryTrait` (you can’t take it).

You’re not choosing one behavior per object. You’re assembling each object out of small, single-purpose traits until it does exactly what the world needs. Every chapter from here on is, underneath, more of this same move: combine traits to get new behavior.

6.6 Try it

> take backpack	Pick up the portable container
> take map	Pick up the zoo map
> put map in backpack	Store the map inside
> look in backpack	See what's in the backpack
> inventory	Backpack counts as one item; its contents ride along
> south	Go to the Main Path
> take penny	Pick up the penny
> put penny on bench	Place it on the supporter
> look	The penny is visible on the bench
> east	Go to the Petting Zoo
> take dispenser	Can't: it's scenery
> examine dispenser	But you can look at it

6.7 Key takeaway

`ContainerTrait` holds things *inside*; `SupporterTrait` holds things *on top*; the parser routes “in” and “on” to the right one automatically. Either can be portable or fixed depending on whether it also has

`SceneryTrait`, and a portable container carries its contents with it as a single inventory item. Set `capacity` to bound them, and remember that traits are composable, so you build each object by stacking the small traits it needs.

7 Openable Things, Locked Doors & Keys: Gating the Way Through

The containers in the last chapter gave up their contents freely. Real worlds make you work a little: a lunchbox you must open before you can reach the juice inside, a staff gate that stays shut until you find the right keycard. This chapter adds the *closed* state, first to containers and then to doors, and along the way wires up the zoo's first real puzzle: find a key, unlock a gate, walk through.

It builds in two steps. `OpenableTrait` adds open and closed. `LockableTrait` adds locked and unlocked on top of it. Doors then connect rooms through that machinery.

7.1 OpenableTrait

`OpenableTrait` gives an entity an open/closed state and hooks up the built-in `open` and `close` actions. On its own it just tracks a boolean; its power comes from combining with other traits.

```
lunchbox.add(new OpenableTrait({
  isOpen: false,           // starts closed
  canClose: true,          // the player can close it again
  revealsContents: true,   // opening announces what's inside
}));
```

The three properties:

Property	Default	What it does
<code>isOpen</code>	<code>false</code>	Current state: open or closed
<code>canClose</code>	<code>true</code>	

Property	Default	What it does
		Whether the player can close it again after opening
revealsContents	true	Whether opening prints “Inside you can see...”

7.1.1 Openable + Container = a discoverable container

Put `OpenableTrait` and `ContainerTrait` on the same entity and the closed state starts to matter:

- **Closed:** contents are hidden. `look in lunchbox` reports it’s closed; `put map in lunchbox` is blocked.
- **Open:** contents are visible and every container operation works normally.

This is how you make the player *discover* things: they open something and find items they couldn’t see before.

7.1.2 Stocking a container that starts closed

Here’s a wrinkle you’ll hit during world setup. You want the lunchbox to start closed, but you also want a juice box inside it from the beginning. The engine enforces the same rules on you that it enforces on the player: you can’t `moveEntity` into a closed container. The fix is to open it, place the item, and close it back:

```
lunchbox.get(OpenableTrait)!.isOpen = true;    // temporarily open
world.moveEntity(juice.id, lunchbox.id);        // place the item
inside
lunchbox.get(OpenableTrait)!.isOpen = false;    // close it again
```

The mistake everyone makes once: calling `moveEntity` into a container that’s closed at setup time and wondering why the

item never appears. The engine plays by its own rules during `initializeWorld()`, so open the container, stock it, then close it.

7.2 LockableTrait

A lock is just a gate on opening. `LockableTrait` adds locked/unlocked state, and it's almost always paired with `OpenableTrait`, because the entire point of a lock is to stop something from being opened.

```
staffGate.add(new LockableTrait({
  isLocked: true,      // starts locked
  keyId: keycard.id,   // THIS key unlocks THIS lock
}));
```

The critical property is `keyId`. It wires one specific key entity to one specific lock. When the player types `unlock gate with keycard`, the engine asks: does the player hold an entity whose `id` matches this lock's `keyId`? If so, the unlock succeeds.

7.2.1 Keys are just items

A key needs no special trait. It's an ordinary `EntityType.ITEM` with an `IdentityTrait`, nothing more. The *only* thing that makes it a key is that some lock's `keyId` points at its `id`. Which means anything can be a key: a literal key, a keycard, a gemstone, a spoken word. The lock decides what opens it, not the key.

7.2.2 The unlock sequence

A locked door asks three separate actions of the player, in order:

1. **Find the key:** `take keycard`
2. **Unlock the lock:** `unlock gate with keycard`
3. **Open the door:** `open gate`

Only after all three can they pass. That sequence is a puzzle in miniature, and you get it for free just by combining the traits.

7.3 DoorTrait and the exit via property

A locked lunchbox is one thing; a locked *door between rooms* is what makes the zoo bigger. Two pieces connect a door to the map.

First, `DoorTrait` marks an entity as the connection between two rooms:

```
staffGate.add(new DoorTrait({
    room1: mainPath.id,      // one side
    room2: supplyRoom.id,    // the other side
    bidirectional: true,      // passable both ways
}));
```

Second, and this is the part that's easy to forget, the rooms' exits must route *through* the door using the `via` property:

```
mainPath.get(RoomTrait)!.exits = {
    [Direction.SOUTH]: {
        destination: supplyRoom.id,
        via: staffGate.id,          // must pass through this entity
    },
};
```

Now when the player types `south`, the going action checks the `via` entity before moving them:

- Is it closed? → “*The staff gate is closed.*”
- Is it locked? → “*The staff gate is locked.*”
- Is it open? → the player walks through.

The mistake everyone makes once: giving the gate every trait but forgetting `via` on the exit. Without `via`, the going action never consults the door; the exit is unconditional and the player

strolls through a “locked” gate as if it weren’t there. The door’s state only matters because the exit points at it.

7.4 Wiring it all together

A working locked door is five traits on the door entity, plus a key, plus the `via` on the exit:

Trait	Purpose
IdentityTrait	Name, description, aliases
DoorTrait	Connects two rooms
OpenableTrait	Can be opened and closed
LockableTrait	Can be locked and unlocked with a key
SceneryTrait	Can’t be picked up

Plus a **key** (any portable item whose `id` you put in `LockableTrait.keyId`) and **exits with `via`** pointing at the door from both sides.

This chapter introduces three new traits, so add them to your world-model import:

```
import {  
    OpenableTrait, LockableTrait, DoorTrait,  
} from '@sharpee/world-model';
```

Here is the whole puzzle, in order: a new room behind the gate (with its shelves), the keycard the player finds, the gate itself as an `EntityType.DOOR` wearing all five traits, and the exits wired through it on both sides.

```

// A new room, behind the gate.
const supplyRoom = world.createEntity('Supply Room',
    EntityType.ROOM);
supplyRoom.add(new RoomTrait({ exits: {}, isDark: false }));
supplyRoom.add(new IdentityTrait({
    name: 'Supply Room',
    description:
        'A cluttered storage room behind the staff gate. Metal
        shelves line the ' +
        'walls, stacked with feed sacks, coiled hoses, and cleaning
        supplies.',
    aliases: ['supply room', 'storage room', 'store room'],
    article: 'the',
}));

// The metal shelves: scenery, so "examine shelves" has something
// to find.
const shelves = world.createEntity('metal shelves',
    EntityType.SCENERY);
shelves.add(new IdentityTrait({
    name: 'metal shelves',
    description: 'Industrial steel shelving stacked with feed sacks
        and supplies.',
    aliases: ['shelves', 'metal shelves', 'shelf', 'shelving'],
}));
world.moveEntity(shelves.id, supplyRoom.id);

// The key: an ordinary item, placed at the entrance for the
// player to find.
const keycard = world.createEntity('staff keycard',
    EntityType.ITEM);
keycard.add(new IdentityTrait({
    name: 'staff keycard',
    description:
        'A white plastic keycard reading "WILLOWBROOK ZOO / STAFF
        ONLY," with a ' +
        'faded photo of a smiling zookeeper on the back.',
    aliases: ['keycard', 'key card', 'card', 'key', 'staff
        keycard'],
    article: 'a',
}));
world.moveEntity(keycard.id, entrance.id);

// The gate: type DOOR, wearing all five traits, placed on the
// Main Path.
const staffGate = world.createEntity('staff gate',
    EntityType.DOOR);

```

```

staffGate.add(new IdentityTrait({
    name: 'staff gate',
    description:
        'A sturdy metal gate marked STAFF ONLY, with a card reader
        beside it.',
    aliases: ['gate', 'staff gate', 'metal gate', 'staff door'],
    article: 'a',
}));
staffGate.add(new DoorTrait({ room1: mainPath.id, room2:
    supplyRoom.id, bidirectional: true }));
staffGate.add(new OpenableTrait({ isOpen: false }));
staffGate.add(new LockableTrait({ isLocked: true, keyId:
    keycard.id }));
staffGate.add(new SceneryTrait());
world.moveEntity(staffGate.id, mainPath.id);

// Wire the passage on BOTH sides, each routing through the gate
// with `via`.
// This replaces the Main Path exits from Chapter 4, adding the
// south passage.
mainPath.get(RoomTrait)!.exits = {
    [Direction.NORTH]: { destination: entrance.id },
    [Direction.EAST]: { destination: pettingZoo.id },
    [Direction.WEST]: { destination: aviary.id },
    [Direction.SOUTH]: { destination: supplyRoom.id, via:
        staffGate.id },
};
supplyRoom.get(RoomTrait)!.exits = {
    [Direction.NORTH]: { destination: mainPath.id, via:
        staffGate.id },
};

```

EntityType.D00R is the label that says “this entity is a door”; the SceneryTrait is still what stops the player taking it. With the return exit on the Supply Room also routed via the gate, the player can walk back out the way they came.

7.5 Try it

> take keycard	Pick up the keycard
> south	Go to the Main Path
> south	"The staff gate is locked."
> unlock gate with keycard	Unlock it
> open gate	Open it
> south	Walk through to the Supply Room
> examine shelves	Look around the supply room
> north	Back through the open gate

7.6 Key takeaway

`OpenableTrait` adds open/closed state and the `open` / `close` actions; combined with `ContainerTrait` it hides contents until opened.

`LockableTrait` adds a lock on top, with `keyId` wiring one key to one lock, and keys are just ordinary items. A door between rooms needs

`DoorTrait` *and* a `via` on the exits pointing at it, or the going action will never check the door at all.

8 Light & Dark: What the Player Can See

South of the supply room lies a nocturnal animals exhibit, and it is pitch black. Walk in without a light and you can't see a thing: no description, no animals, nothing to interact with but the way back out. The fix is sitting in the supply room: a flashlight. Switch it on, carry it in, and the darkness lifts to reveal sugar gliders, bush babies, and a barn owl.

Darkness is one of the oldest mechanics in interactive fiction, and it's really two ideas working together: rooms that can be dark, and objects that can light them.

8.1 Dark rooms

Any room becomes dark by setting `isDark: true` on its `RoomTrait`. Here's the one line that matters on the nocturnal exhibit (we build the whole room at the end of the chapter):

```
nocturnalExhibit.add(new RoomTrait({
  exits: {},
  isDark: true,      // this room is pitch black
}));
```

Enter a dark room with no light and the player sees a darkness message instead of the room description. They can't examine, take, or touch anything; the only move available is to leave. The objects are still there; they're just unreachable until there's light.

8.2 LightSourceTrait

`LightSourceTrait` marks an entity as something that can illuminate a dark room:

```
flashlight.add(new LightSourceTrait({
    brightness: 8,    // how powerful, 1-10
    isLit: false,    // starts unlit
}));
```

When the player is carrying an entity whose `isLit` is `true`, dark rooms light up: the description appears normally and every object becomes accessible again.

8.3 SwitchableTrait

`SwitchableTrait` gives an entity an on/off state and the `switch on` / `switch off` actions:

```
flashlight.add(new SwitchableTrait({
    isOn: false,    // starts off
}));
```

On its own it just tracks on/off. Combined with `LightSourceTrait`, flipping the switch is what lights the device.

8.4 The flashlight pattern

A flashlight is three traits stacked, the composability lesson from earlier chapters applied again:

Trait	What it provides
<code>SwitchableTrait</code>	On/off toggle via <code>switch on</code> / <code>switch off</code>
<code>LightSourceTrait</code>	Illumination for dark rooms
<code>IdentityTrait</code>	Name, description, aliases

When the player switches it on:

1. `SwitchableTrait.isOn` becomes `true`.

2. `LightSourceTrait.isLit` becomes `true`; the engine links the two.
3. Any dark room the player carries it into is now lit.

```
const flashlight = world.createEntity('flashlight',
    EntityType.ITEM);
flashlight.add(new IdentityTrait({
    name: 'flashlight',
    description: 'A heavy rubberized flashlight with a bright
        halogen bulb.',
    aliases: ['flashlight', 'torch', 'light'],
}));
flashlight.add(new SwitchableTrait({ isOn: false }));
flashlight.add(new LightSourceTrait({ brightness: 8, isLit:
    false }));
world.moveEntity(flashlight.id, supplyRoom.id);
```

The mistake everyone makes once: expecting `SwitchableTrait` alone to banish the dark. A switch with no `LightSourceTrait` just toggles on and off and lights nothing, and a `LightSourceTrait` with no switch is *always* lit. A controllable light needs both.

8.5 Other light-source patterns

The flashlight is the simplest case. The same trait covers others by changing which pieces you include:

Always-on light (a glowing gem, an enchanted sword): no switch, just lit:

```
gem.add(new LightSourceTrait({ isLit: true, brightness: 5 }));
```

Consumable light (a candle or torch that burns down):


```
candle.add(new LightSourceTrait({
  isLit: false,
  brightness: 3,
  fuelRemaining: 50,      // burns for 50 turns
  fuelConsumptionRate: 1, // one fuel per turn
}));
```

Adjustable light (a lantern with a dimmer): set `brightness` high and let story code change it dynamically:

```
lantern.add(new LightSourceTrait({ brightness: 10 }));
```

8.6 Darkness as a gate

Objects inside a dark room exist the whole time; they're simply inaccessible until there's light. That makes darkness a natural gating mechanism: put something worth finding behind it, and the light source becomes the key that opens it. The flashlight here, a candle elsewhere, a magic spell in another game: same shape, different flavor.

8.7 Wiring it into the zoo

Two new traits arrive this chapter, so add them to your world-model import:

```
import { LightSourceTrait, SwitchableTrait } from '@sharpee/world-
model';
```

The dark exhibit hangs off the Supply Room from Chapter 7. Build the room in full, connect it south of the supply room (with the way back north), and populate it with the animals the flashlight will reveal:

```

const nocturnalExhibit = world.createEntity('Nocturnal Animals
    Exhibit', EntityType.ROOM);
nocturnalExhibit.add(new RoomTrait({ exits: {}, isDark: true }));
nocturnalExhibit.add(new IdentityTrait({
    name: 'Nocturnal Animals Exhibit',
    description:
        'A hushed, cavern-like hall lit by faint blue moonlight
        panels. Sugar ' +

            'gliders leap between branches, wide-eyed bush babies
            cling to a rope, ' +

            'and an enormous barn owl perches motionless on a stump.',
    aliases: ['nocturnal exhibit', 'nocturnal animals', 'dark
        exhibit', 'exhibit'],
    article: 'the',
}));

// Connect it south of the Supply Room, with the way back north.
// This adds the
// south passage to the Supply Room exits from Chapter 7.
supplyRoom.get(RoomTrait)!.exits = {
    [Direction.NORTH]: { destination: mainPath.id, via:
        staffGate.id },
    [Direction.SOUTH]: { destination: nocturnalExhibit.id },
};
nocturnalExhibit.get(RoomTrait)!.exits = {
    [Direction.NORTH]: { destination: supplyRoom.id },
};

// The animals: scenery, examinable only once the room is lit.
const sugarGliders = world.createEntity('sugar gliders',
    EntityType.SCENERY);
sugarGliders.add(new IdentityTrait({
    name: 'sugar gliders',
    description: 'A family of tiny sugar gliders with enormous dark
        eyes, gliding between branches.',
    aliases: ['sugar gliders', 'gliders', 'sugar glider'],
    article: 'some',
}));
world.moveEntity(sugarGliders.id, nocturnalExhibit.id);

const bushBabies = world.createEntity('bush babies',
    EntityType.SCENERY);
bushBabies.add(new IdentityTrait({
    name: 'bush babies',
    description: 'Two bush babies with impossibly large round eyes,
        clinging to a rope.',

```

```

    aliases: ['bush babies', 'bush baby', 'galagos'],
    article: 'some',
  }));
world.moveEntity(bushBabies.id, nocturnalExhibit.id);

const barnOwl = world.createEntity('barn owl',
  EntityType.SCENERY);
barnOwl.add(new IdentityTrait({
  name: 'barn owl',
  description: 'An enormous barn owl with a heart-shaped white
    face, watching you without blinking.',
  aliases: ['barn owl', 'owl'],
  article: 'a',
}));
world.moveEntity(barnOwl.id, nocturnalExhibit.id);

```

The flashlight from earlier in the chapter sits in the Supply Room, ready to carry in, so `examine owl` and `examine gliders` in the walkthrough below both resolve once the light is on.

8.8 Try it

> take keycard	Get the key
> south	Main Path
> unlock gate with keycard	Unlock the staff gate
> open gate	Open it
> south	Supply Room
> take flashlight	Grab the flashlight
> south	Nocturnal Exhibit, dark!
> look	"It is pitch dark..."
> north	Retreat to the Supply Room
> switch on flashlight	Let there be light
> south	Nocturnal Exhibit, now lit!
> examine owl	Look at the barn owl
> examine gliders	Look at the sugar gliders
> switch off flashlight	Darkness returns
> look	Dark again

8.9 Key takeaway

`isDark: true` on a `RoomTrait` makes a room pitch black, locking out interaction until light arrives. `LightSourceTrait` lets an entity illuminate the dark, and `SwitchableTrait` adds the on/off control. A flashlight is just an item carrying both: switch it on, take it in, and the darkness lifts. Vary which traits you include for always-on, consumable, or adjustable lights.

9 The Map & Regions: Grouping Rooms

You’ve quietly been building a map since Chapter 4. Every `RoomTrait` you gave an `exits` table added another connection, and together the zoo’s six rooms (entrance, main path, petting zoo, aviary, and the supply room and nocturnal exhibit behind the staff gate) form a small but complete map. This chapter steps back to look at that map as a whole, and introduces **regions**: a way to treat a group of rooms as a single named place.

9.1 The map is the exits

There is no separate “map” object in Sharpee. The map is the set of exits you declared on each room: a graph of rooms joined by directions. A couple of habits keep that graph sane as it grows:

- **Make exits reciprocal.** If the main path leads east to the petting zoo, the petting zoo should lead west back. Sharpee won’t add the return exit for you, and a one-way passage the player can’t retrace is almost always a bug.
- **Keep directions consistent.** If north takes the player somewhere, south should bring them back. Players build a mental map from your compass; contradicting it is disorienting.
- **Route doored connections through the door.** The staff gate connection uses `via` so the door is actually checked (Chapter 7); the map and the barrier on it are the same link.

For six rooms you can hold the whole map in your head. For sixty, you’ll want a way to talk about *areas* rather than individual rooms. That’s what regions are for.

9.2 Regions: grouping rooms

A **region** is a named area that owns a set of rooms. The zoo divides naturally into two: the public area the visitor wanders freely (entrance, main path, petting zoo, aviary) and the staff area behind the gate (supply room, nocturnal exhibit). Regions let you name that division and act on it.

Create regions in `initializeWorld()`, before the rooms that belong to them:

```
world.createRegion('reg-public', {
  name: 'Public Zoo',
});

world.createRegion('reg-staff', {
  name: 'Staff Area',
  ambientSmell: 'disinfectant and animal feed',
});
```

By convention region IDs take a `reg-` prefix, to tell them apart from room IDs at a glance.

Then, after the rooms exist, assign each one to its region:

```
world.assignRoom(entrance.id, 'reg-public');
world.assignRoom(mainPath.id, 'reg-public');
world.assignRoom(pettingZoo.id, 'reg-public');
world.assignRoom(aviary.id, 'reg-public');

world.assignRoom(supplyRoom.id, 'reg-staff');
world.assignRoom(nocturnalExhibit.id, 'reg-staff');
```

9.3 Region-wide properties

The reason to group rooms is that a region can carry properties its rooms inherit. A `RegionOptions` object accepts a few:

Property	What it does
<code>name</code>	The region's human-readable name (required)
<code>defaultDark</code>	Rooms in the region start dark unless they say otherwise
<code>ambientSound</code>	A region-wide sound (dripping water, distant traffic)
<code>ambientSmell</code>	A region-wide smell
<code>parentRegionId</code>	Nest this region inside another

Setting `defaultDark: true` on, say, a cave region saves you marking every room `isDark` by hand: a property that belongs to the *area* lives on the area.

9.4 Crossing the boundary

The real power shows up when the player moves *between* regions. When a `go` command carries the player from a room in one region to a room in another, the engine emits two events automatically:

- `if.event.region_exited`, fired once for each region being left,
- `if.event.region_entered`, fired once for each region being entered.

You react to them exactly the way you'll react to any event in Volume IV, by registering a handler:

```
world.registerEventHandler('if.event.region_entered', (event,
    world) => {
  if (event.data.regionId === 'reg-staff') {
    // The visitor just slipped into the staff area: flavor, a
    // warning,
    // a scoring hook, whatever the moment calls for.
  }
});
```

This is the natural home for “as you enter the old town, the noise of the market swells”: atmosphere keyed to an area instead of bolted onto every room’s description.

9.5 Nesting and querying

Regions can nest. Give one a `parentRegionId` and a room in the child counts as being in the parent too:

```
world.createRegion('reg-underground', { name: 'The Underground',
                                         defaultDark: true });
world.createRegion('reg-mine', { name: 'Coal Mine',
                                parentRegionId: 'reg-underground' });
// a room in reg-mine answers true for reg-underground as well
```

And you can ask the world about membership at any time:

`world.isInRegion(roomId, 'reg-staff')` gives a yes/no, or, if you add the optional `@sharpee/queries` package, its entity-query API lists every room in an area: `world.rooms.inRegion('reg-staff', world).toArray()`. (`world.rooms` comes from that package; the plain `WorldModel` gives you `isInRegion`.)

9.6 Key takeaway

The map is nothing more than the exits you declare on each room: keep them reciprocal and consistent and the graph stays trustworthy. When a map grows past what you can hold in your head, **regions** group rooms into named areas that can share properties (`defaultDark`, ambient sound and smell) and, best of all, fire `if.event.region_entered` / `region_exited` as the player crosses between them, the hook for area-wide atmosphere and events. The zoo is small enough to skip regions entirely; reach for them when your world gets big enough to think about in neighborhoods.

VOLUME III — MAKING IT INTERACTIVE

10 The Standard Actions: The Four-Phase Model

So far the zoo has been a place to *be*: rooms to walk through, objects to examine, containers to open. Everything the player could do already worked: `take`, `drop`, `open`, `go`, `examine`. You never wrote a line of code to make it so. This volume is about how a world *responds*, and it starts with the machinery you’ve been leaning on all along: the standard action library, and the four-phase model every action in Sharpee obeys.

10.1 The verbs you got for free

Sharpee’s standard library (`@sharpee/stdlib`) ships a full vocabulary of interactive-fiction verbs. The moment you created a room and an object, the player could already act on them. The standard actions fall into three groups:

- **World-changing actions** alter the game state: `take` and `drop`, `open` and `close`, `go`, `put` and `insert`, `lock` and `unlock`, `switch on/off`, `wear` and `remove`, `eat` and `drink`, `push`, `pull`, `give`, `enter` and `exit`.
- **Query actions** only look: `examine`, `look`, `read`, `search`, `inventory`. They report what’s there without changing anything.
- **Meta actions** speak to the game rather than the world: `save`, `restore`, `quit`, `wait`, `again`, `help`, `score`.

You don’t register any of these; they come with the platform, wired to the traits you’ve already met. `OpenableTrait` is what makes `open` work on the lunchbox; `SwitchableTrait` is what makes `switch on` work on the flashlight. Add the trait, and the matching verb lights up. (The full catalog lives in Appendix B.)

10.2 One shape for every action

Every action, the standard ones above and the custom ones you’ll write in Volume IV, has the same four-part structure. Chapter 3 sketched it as “check, change, report”; here is the whole contract:

```
const someAction: Action = {
  id: 'if.action.taking',

  validate(context): ValidationResult { /* can this happen? */ },
  execute(context): void { /* change the world */ },
  report(context): ISemanticEvent[] { /* record what happened
    */ },
  blocked(context, result): ISemanticEvent[] { /* explain the
    refusal */ },
};
```

The engine calls these in a fixed order, and each has exactly one job.

10.2.1 validate: can this happen?

`validate` is the gatekeeper. It checks preconditions and returns either `{ valid: true }` or `{ valid: false, error: '...' }`. Taking something checks that it’s here, that it’s not already in your hands, that it’s not scenery bolted to the floor. `validate` decides; it never changes anything.

10.2.2 execute: change the world

If validation passed, `execute` runs and performs the actual mutation, which moves the item into the player’s inventory, flipping `isOpen` to `true`. This is the *only* phase that changes game state, and it’s meant to be small. The real work usually lives in a **behavior** (more on those in Volume IV), with `execute` just coordinating it.

10.2.3 report: record what happened

`report` produces the `events` for the turn: the `if.event.taken` and `if.event.opened` records you met in Chapter 3, each carrying a message id rather than text. It generates events; it doesn't mutate. Anything `execute` learned that `report` needs is handed forward through `context.sharedData`, never smuggled onto the context itself.

10.2.4 blocked: explain the refusal

If `validate` returns `{ valid: false }`, the engine skips `execute` and `report` and calls `blocked` instead. Its job is to turn the validation error into an event the player can understand, such as “You can't take that.” A refused action is still a complete, well-formed turn; it simply reports a different outcome.

10.3 Why the discipline matters

Splitting an action into four phases with strict rules (*mutations only in execute, events only in report*) is what makes the whole system predictable and extensible:

- **Validation can be trusted.** Because `validate` never changes anything, the engine can ask “would this work?” without side effects, which is how the parser can disambiguate and how a command can be checked before it commits.
- **Every action reads the same way.** Learn the shape once and you can read any action in the library, and write your own that slots in beside them.
- **The world stays inspectable.** State changes happen in one phase; the record of them happens in another. That clean seam is what lets event handlers (Chapter 13) react to what an action did, and what makes saved games and replays reliable.

You won't write an action in this chapter, because the standard library already covers the zoo. But the four-phase model is the backbone of everything ahead: custom actions in Chapter 14 fill in all four phases by hand, capability dispatch in Chapter 15 delegates them per entity, and event handlers in Chapter 13 hang new consequences off the events `report` emits.

10.4 Key takeaway

The standard library gives you a complete set of IF verbs for free, each wired to the traits you add to entities, with no registration required. Behind every verb is the same four-phase contract: **validate** (can it happen?), **execute** (change the world), **report** (record what happened as events), and **blocked** (explain a refusal). Mutations live only in `execute`, events only in `report`, and data passes between them through `context.sharedData`. That single, uniform shape is what makes Sharpee actions predictable to read, safe to validate, and ready to extend.

11 Scope & Visibility: What the Player Can Reach

When the player types `take key`, two quiet questions get answered before anything happens: *which* key, and *can the player refer to it at all?* A key in the room is fair game; a key locked inside a closed box, or sitting in the next room, is not, even though both exist in the world. The system that draws that line is **scope**, and you've been relying on it since your very first room without touching it.

11.1 What scope is

Scope is the set of entities a command can refer to and act on *right now*. It's computed fresh each turn from where the player is and what they can perceive: the room they're in, the things in it, what they're carrying, what's inside open containers nearby. When the parser resolves the word "key," it searches scope, not the whole world, which is why `examine sign` found the welcome sign in Chapter 2 but `examine sign` from a different room would not.

Think of scope as the answer to "what could the player plausibly mean?" Everything outside it is, as far as this command is concerned, invisible.

11.2 Degrees of access

Not everything in scope is equally available. Sharpee distinguishes a few degrees, because different verbs need different things:

- **In the room:** present in the player's location.
- **Visible:** can be *seen* (subject to light; see below).
- **Reachable:** can be physically *touched*.
- **Carried / worn:** already in the player's hands or on their body.

- **Audible:** can be *heard* even if not seen.

These overlap but aren't identical, and the gap between them is where good puzzles live. You can *see* a fish through the glass of an aquarium but not *reach* it. You can *hear* the parrot squawking from the next room without it being *visible*. Most of the time the standard actions pick the right degree for you (`examine` wants visibility, `take` wants reach), so you rarely think about it. You reach for the distinction yourself only when you write a custom action whose verb needs a different degree.

11.3 Sight and darkness

Visibility isn't free: it depends on light. In a dark room, such as the nocturnal exhibit from *Light & Dark*, the player can't see, so visual scope collapses to almost nothing. The objects are still physically present, but a `look` or `examine` can't reach them; the engine's perception layer steps in and replaces "you see..." output with the darkness message instead. Bring a lit flashlight and visibility (and the room's contents) snap back.

This is why darkness works as a gate: it doesn't remove objects, it removes the player's *perception* of them. Sight is the sense most puzzles gate on, but the same rule covers the others: a sound the player can't hear is out of scope too.

11.4 Permissive parser, strict action

The parser is deliberately **permissive**, and this is worth understanding even though you rarely configure it: when it resolves a noun, it accepts anything the player could plausibly be touching, and leaves the harder judgment to the action. The action's `validate` phase then applies the *strict* rule: does this verb actually require sight? reach?

The payoff is commands like attacking, pushing, or grabbing in the dark: the parser still resolves "the lever" by touch, and the action decides whether doing it blind is allowed. If the parser demanded full visibility up

front, every in-the-dark interaction would fail with “you can’t see any such thing” before the action ever got a say. (When you write your own grammar in Volume V, you can set a pattern’s scope explicitly; until then, the standard verbs already strike this balance.)

Inside an action, the checks are simple helpers, `context.canSee(entity)` and `context.canReach(entity)`, so a custom action can be as strict or as lenient as its verb demands.

11.5 Key takeaway

Scope is the per-turn set of things a command can refer to, computed from the player’s location and senses. The parser searches it, not the whole world, which is how `take key` finds the right key and ignores the one locked away. Within scope, verbs care about different degrees of access: **visible**, **reachable**, **carried**, **audible**. Visibility depends on light, so darkness shrinks sight-based scope without deleting the objects. And because the parser resolves permissively while each action validates strictly, the player can still act in the dark when the verb allows it.

12 Readable Objects & Switchable Devices: Things That Carry State

A zoo is full of things to read. Brass plaques by the enclosures, a glossy brochure at the entrance, a yellow warning sign outside the nocturnal exhibit. Each sign *says* something the player wants to take in, separate from what it *looks* like. And tucked on a shelf in the supply room is a battered radio that clicks on and off but sheds no light at all. This chapter covers two small, self-contained traits that round out an ordinary world: `ReadableTrait` for things with words, and `SwitchableTrait` for devices with an on/off state.

`ReadableTrait` is new this chapter; `SwitchableTrait` you met in *Light & Dark*. Both come from the world-model package:

```
import { ReadableTrait, SwitchableTrait } from '@sharpee/world-model';
```

The snippets below go in `initializeWorld`, alongside the rooms you've built since Chapter 4. `entrance`, `pettingZoo`, and `supplyRoom` are the same room entities from earlier chapters.

12.1 ReadableTrait: what an object says

`ReadableTrait` gives an entity text that the `read` action displays:

```
plaque.add(new ReadableTrait({
  text: 'PYGMY GOATS: These Nigerian Dwarf goats are gentle,
        curious, ' +
        'and always hungry.',
}));
```

`read` and `examine` are different verbs that pull from different traits, `ReadableTrait.text` versus `IdentityTrait.description`:

Command	Trait used	What it shows
examine plaque	<code>IdentityTrait.description</code>	What the object looks like
read plaque	<code>ReadableTrait.text</code>	What the object says

A brass plaque *looks like* “a brass plaque mounted on a wooden post.” It *says* “PYGMY GOATS: These Nigerian Dwarf goats are gentle, curious, and always hungry.” Two different strings, two different verbs.

The test for whether something wants a `ReadableTrait`: would a real person say “I want to *read* this”? A sign, a book, a letter, a label: yes. A rock, a fence, a tree: no; those just need a description.

12.2 Readable scenery: the info plaque

Plaques are scenery you can read but can’t take. Each includes `IdentityTrait`, `ReadableTrait`, and `SceneryTrait`.

```

const pettingPlaque = world.createEntity('info plaque',
    EntityType.SCENERY);
pettingPlaque.add(new IdentityTrait({
    name: 'info plaque',
    description:
        'A brass plaque mounted on a wooden post near the petting zoo
        ' +
        'gate. It has text etched into the metal.',
    aliases: ['plaque', 'info plaque', 'brass plaque'],
    properName: false,
    article: 'an',
}));
pettingPlaque.add(new ReadableTrait({
    text:
        'PYGMY GOATS: These Nigerian Dwarf goats are gentle, curious,
        ' +
        'and always hungry. They can eat up to 3% of their body
        weight ' +
        'daily. Please use only zoo-approved feed from the dispensers.
        \n\n' +
        'HOLLAND LOP RABBITS: Known for their floppy ears and calm ' +
        'temperament. Our pair, Biscuit and Marmalade, were born
        right ' +
        'here at Willowbrook in 2023.',
}));
world.moveEntity(pettingPlaque.id, pettingZoo.id);

```

Note the `\n\n` between the two animals. Readable text is shown as-is, so line breaks are yours to place.

The mistake everyone makes once: writing one long unbroken paragraph and wondering why it reads like a wall. Use `\n` (or `\n\n`) to break readable text into lines and stanzas the way a real sign or page would.

12.3 Readable items: the brochure

`ReadableTrait` works on portable things too. Leave off `SceneryTrait` and the player can pick the object up and read it anywhere:

```

const brochure = world.createEntity('zoo brochure',
    EntityType.ITEM);
brochure.add(new IdentityTrait({
    name: 'zoo brochure',
    description:
        'A glossy tri-fold brochure with "WILLOWBROOK FAMILY ZOO" on
        ' +
        'the cover in cheerful yellow letters.',
    aliases: ['brochure', 'zoo brochure', 'pamphlet', 'leaflet'],
    properName: false,
    article: 'a',
}));
brochure.add(new ReadableTrait({
    text:
        'WILLOWBROOK FAMILY ZOO: Your Guide\n\n' +
        'EXHIBITS:\n' +
        '  Petting Zoo ..... East from Main Path\n' +
        '  Aviary ..... West from Main Path\n' +
        '  Nocturnal Animals ..... Staff Area (ask a keeper!)\n\n' +
        '"Where every visit is a wild adventure!"',
}));
world.moveEntity(brochure.id, entrance.id);

```

Readable scenery (plaques, warning signs) and readable items (brochures, letters, books) are the same trait; the only difference is whether the thing is fixed in place.

12.4 SwitchableTrait: a device with on/off state

Back in *Light & Dark* the flashlight combined `SwitchableTrait` with `LightSourceTrait`. But `SwitchableTrait` stands perfectly well on its own, for any device with an on/off state that isn't a light. The supply-room radio is exactly that:

```

const radio = world.createEntity('radio', EntityType.ITEM);
radio.add(new IdentityTrait({
  name: 'radio',
  description:
    'A battered portable radio held together with duct tape. A
    faded ' +
    'sticker on the side reads "ZOO FM | All Animals, All The
    Time."',
  aliases: ['radio', 'portable radio'],
  properName: false,
  article: 'a',
}));
radio.add(new SwitchableTrait({ isOn: false })); // starts off
radio.add(new SceneryTrait()); // bolted to
    the shelf
world.moveEntity(radio.id, supplyRoom.id);

```

The player can switch on radio, switch off radio, turn on radio, or turn off radio; the stdlib handles all four phrasings and reports the toggle. The radio has no LightSourceTrait, so switching it on changes its state but illuminates nothing. The SceneryTrait means it can't be carried off.

The mistake everyone makes once: expecting a bare SwitchableTrait to do something dramatic. On its own it just tracks isOn. Combine it with LightSourceTrait for a controllable light, or react to its state with an event handler (the next volume of the book) when you want flipping the switch to do something.

12.5 Switchable vs. openable

SwitchableTrait and OpenableTrait look like twins (both hold a boolean, both have paired verbs), but they model different kinds of object, and the parser keeps their verbs apart:

Trait	Verbs	Models	Examples
SwitchableTrait			

Trait	Verbs	Models	Examples
	switch on / off, turn on / off	Devices, electronics	Radio, flashlight, alarm, fan
OpenableTrait	open / close	Physical barriers	Door, container, book, lid

You'd never "switch on" a door or "open" a radio. Pick the trait whose verbs match how a person would actually talk about the object.

12.6 Try it

> take brochure	Pick up the zoo brochure
> take keycard	Grab the staff keycard; it's here at the entrance
> read brochure	Read the guide, different from examine!
> examine brochure	See the physical brochure
> south	Main Path
> east	Petting Zoo
> read plaque	Read about the goats and rabbits
> examine plaque	See the brass plaque itself
> west	Back to Main Path
> unlock gate with keycard	Open the staff area
> open gate; south	Supply Room
> examine radio	See the battered radio
> switch on radio	Click, it's on
> switch off radio	Click, off again
> take radio	Can't, it's scenery

12.7 Key takeaway

`ReadableTrait` separates what an object *says* (`read`) from what it *looks like* (`examine`); type fixed plaques and signs `EntityType.SCENERY` and leave portable brochures and books as plain items, and use `\n` to shape the text. `SwitchableTrait` gives any device an on/off toggle through the `switch` / `turn` verbs, alone for a plain device like the radio, or paired with another trait when the switch should drive something. It's the

sibling of `OpenableTrait` : same shape, different verbs, different kind of object.

VOLUME IV — CUSTOM BEHAVIOR

13 Event Handlers: Reacting to What Happens

So far every object in the zoo has *been* something: scenery, a container, a readable sign. This chapter is where the world starts to *react*. Drop the bag of feed in the petting zoo and the goats rush over to devour it. Feed a souvenir penny into the press in the gift shop and it comes out flattened and embossed. Neither of those is a new verb. The player is using ordinary `drop` and `put in`. The reaction comes from an **event handler** listening for those actions and adding something on top.

This is the workhorse pattern for puzzles and special effects: let the standard actions do their job, then hook the events they emit.

13.1 Every action emits an event

When a standard action succeeds, it announces what happened by emitting an event. You react by registering a handler for that event type:

Event	Fired when
<code>if.event.taken</code>	Player took an item
<code>if.event.dropped</code>	Player dropped an item
<code>if.event.put_in</code>	Player put an item in a container
<code>if.event.put_on</code>	Player put an item on a supporter
<code>if.event.opened</code>	Player opened something
<code>if.event.closed</code>	Player closed something
<code>if.event.locked</code>	Player locked something
<code>if.event.unlocked</code>	Player unlocked something

...and many more. The list grows with the `stdlib`, but the shape is always the same: an action happens, an event fires, your handler gets a look.

13.2 Two kinds of handler

There are two ways to listen, and which you choose depends on whether you want the player to *see* anything.

Silent handlers mutate world state and produce no text. Register them with `world.registerEventHandler()`:

```
world.registerEventHandler('if.event.dropped', (event, world) => {  
  // Set a flag, move an item, change state; but no visible text  
  world.setStateValue('item-was-dropped', true);  
});
```

Chain handlers return an event that turns into visible text. Register them with `world.chainEvent()`. The handler returns either an event (which gets dispatched and rendered) or `null` to stay quiet:

```
world.chainEvent(  
  'if.event.dropped',  
  (event, w) => {  
    const data = event.data as Record<string, any>;  
    if (data.itemId !== feedId) return null; // not our item,  
      ignore  
    return {  
      id: `goats-react-${Date.now()}`,  
      type: 'zoo.event.goats_react',  
      timestamp: Date.now(),  
      entities: {},  
      data: { text: 'The goats rush over and devour the feed!' },  
    };  
  },  
  { key: 'zoo.chain.goats-eat-feed' },  
);
```

Use `registerEventHandler()` for bookkeeping the player never sees; use `chainEvent()` when something visible should happen.

The mistake everyone makes once: reaching for `type: 'game.message'` in a chain handler. The event processor treats a `game.message` returned from a handler as an *override* of the

original action’s text, so instead of adding your reaction it replaces the “You drop the feed.” line. Use a custom event type like `zoo.event.goats_react` with a `text` field instead; the renderer displays any event that carries `text`, and the original action message survives.

13.3 Reading the event data

Each event carries a `data` object describing what happened. The fields depend on the event type:

```
// if.event.dropped
{ item: string, itemId: EntityId, toLocation: EntityId }

// if.event.put_in
{ itemId: EntityId, targetId: EntityId, preposition: 'in' }
```

Note `item` is the item’s *name* and `itemId` is its *entity ID*. Compare against `itemId`. Names aren’t unique; IDs are.

13.4 Setting up: the gift shop, the press, and remembering IDs

The reactions in this chapter need three things the world doesn’t have yet: a Gift Shop room, the souvenir press to put the penny in, and a way for a handler to *refer* to specific entities long after `initializeWorld` has run. The story remembers the IDs it cares about in two class fields, and the handler signatures pull in a few new types:

```

import { GameEngine } from '@sharpee/engine';
import { ISemanticEvent } from '@sharpee/core';
import { IWorldModel } from '@sharpee/world-model';

class FamilyZooStory implements Story {
  config = config;

  private roomIds: { giftShop: string; pettingZoo: string } =
    { giftShop: '', pettingZoo: '' };
  private entityIds: { animalFeed: string; penny: string;
    souvenirPress: string } =
    { animalFeed: '', penny: '', souvenirPress: '' };

  // createPlayer / initializeWorld / onEngineReady ...
}

```

In `initializeWorld`, add the Gift Shop west of the Aviary and the press inside it, then record the IDs the handlers will match against (the `penny` and `animalFeed` entities were created back in Chapter 5):

```

const giftShop = world.createEntity('Gift Shop', EntityType.ROOM);
giftShop.add(new RoomTrait({ exits: {}, isDark: false }));
giftShop.add(new IdentityTrait({
  name: 'Gift Shop',
  description:
    'A small zoo gift shop crammed with stuffed animals and
    postcards. A large ' +

    'souvenir penny press stands near the door. The aviary is
    back to the east.',
  aliases: ['gift shop', 'shop', 'store'],
  article: 'the',
}));
// Connect it west of the Aviary (and back east). This replaces
  the Aviary
// exits from Chapter 4, adding the west passage.
aviary.get(RoomTrait)!.exits = {
  [Direction.EAST]: { destination: mainPath.id },
  [Direction.WEST]: { destination: giftShop.id },
};
giftShop.get(RoomTrait)!.exits = {
  [Direction.EAST]: { destination: aviary.id },
};

const souvenirPress = world.createEntity('souvenir press',
  EntityType.CONTAINER);
souvenirPress.add(new IdentityTrait({
  name: 'souvenir press',
  description:
    'A heavy cast-iron machine with a crank handle and a slot
    that accepts ' +
    'pennies. A sign reads: "INSERT PENNY, TURN HANDLE, KEEP
    FOREVER!"',
  aliases: ['press', 'souvenir press', 'penny press', 'machine'],
  article: 'a',
}));
souvenirPress.add(new ContainerTrait({ capacity: { maxItems:
  1 } }));
souvenirPress.add(new SceneryTrait());
world.moveEntity(souvenirPress.id, giftShop.id);

// Remember the IDs the event handlers will match against.
this.roomIds.giftShop = giftShop.id;
this.roomIds.pettingZoo = pettingZoo.id;
this.entityIds.animalFeed = animalFeed.id;

```

```
this.entityIds.penny = penny.id;  
this.entityIds.souvenirPress = souvenirPress.id;
```

The handlers themselves are registered in `onEngineReady`, which the engine calls once the world is fully built. The two reaction sections that follow both live inside it:

```
onEngineReady(engine: GameEngine): void {  
    const world = engine.getWorld();  
    // the chainEvent registrations below go here  
}
```

13.5 Reaction pattern: the goats eat the feed

Putting it together: when the player drops the feed in the petting zoo, the goats react, but only once:

```

const feedId = this.entityIds.animalFeed;
const pettingZooId = this.roomIds.pettingZoo;

world.chainEvent(
  'if.event.dropped',
  (event: ISemanticEvent, w: IWorldModel): ISemanticEvent | null
    => {
    const data = event.data as Record<string, any>;

    // Is it the feed, dropped in the petting zoo?
    if (data.itemId !== feedId || data.toLocation !==
      pettingZooId) {
      return null;
    }

    // Only react once
    if (w.getStateValue('goats-fed')) return null;
    w.setStateValue('goats-fed', true);

    return {
      id: `zoo-goats-eat-${Date.now()}`,
      type: 'zoo.event.goats_react',
      timestamp: Date.now(),
      entities: {},
      data: {
        text:
          'The pygmy goats spot the bag of feed and rush over! They' +
          'crowd around, bleating excitedly, and devour the' +
          'corn and ' +
          'pellets in seconds. The smallest goat looks up at' +
          'you with ' +
          'big grateful eyes.',
      },
    };
  },
  { key: 'zoo.chain.goats-eat-feed' },
);

```

The `getStateValue` / `setStateValue` flag is the guard that keeps the goats from re-staging their feast every time the feed touches the ground.

13.6 Transformation pattern: put A in, get B out

A classic puzzle shape: the player puts one item into a machine and a different item comes out. The souvenir press swallows a plain penny and produces a pressed one:


```

const pennyId = this.entityIds.penny;
const pressId = this.entityIds.souvenirPress;

world.chainEvent(
  'if.event.put_in',
  (event: ISemanticEvent, w: IWorldModel): ISemanticEvent | null
    => {
    const data = event.data as Record<string, any>;
    if (data.itemId !== pennyId || data.targetId !== pressId)
      return null;

    // 1. Destroy the input
    w.removeEntity(pennyId);

    // 2. Create the output
    const pressedPenny = w.createEntity('pressed penny',
      EntityType.ITEM);
    pressedPenny.add(new IdentityTrait({
      name: 'pressed penny',
      description: 'A flattened oval of copper with an embossed
        toucan.',
      aliases: ['pressed penny', 'pressed coin', 'souvenir'],
      properName: false,
      article: 'a',
    }));

    // 3. Hand it to the player
    const player = w.getPlayer();
    if (player) w.moveEntity(pressedPenny.id, player.id);

    // 4. Tell them what happened
    return {
      id: `zoo-press-penny-${Date.now()}`,
      type: 'zoo.event.penny_pressed',
      timestamp: Date.now(),
      entities: {},
      data: {
        text: 'CLUNK! CRUNCH! WHIRRR! The souvenir press swallows
          the penny ' +
            'and spits out a beautiful pressed penny with an
          embossed ' +
            'toucan design. You pocket it proudly.',
      },
    };
  },
);

```

```
{ key: 'zoo.chain.penny-press' },  
);
```

Remove the old entity, create the new one, move it to the player, return the text. That four-step shape covers a surprising number of machines, ovens, forges, and vending slots.

The `{ key: '...' }` option gives each handler a unique identifier, which the engine needs to manage handlers across saves and reloads.

13.7 Try it

> south; east	Go to the Petting Zoo
> take feed	Pick up the bag of animal feed
> drop feed	The goats rush over!
> west	Back to Main Path
> take penny	Grab the souvenir penny
> west	Aviary
> west	Gift Shop
> examine press	See the souvenir press
> put penny in press	CLUNK! CRUNCH! WHIRRR!
> inventory	You're holding a pressed penny

13.8 Key takeaway

Event handlers let standard actions do the work while you react to what they emit. `world.registerEventHandler()` runs silently for state bookkeeping; `world.chainEvent()` returns an event with a `text` field to show the player something. Match on `itemId` / `targetId` (not names), guard one-time reactions with a state flag, and never return `game.message` from a chain handler. Use a custom event type so your reaction adds to the action's text instead of replacing it.

14 Custom Actions: Teaching the Parser New Verbs

Event handlers let you react to verbs the `stdlib` already knows. But sometimes the verb itself doesn't exist yet. There's no `feed` action in the standard library, no `photograph`. When you need a brand-new verb, you write a **custom action**, and wiring one up means touching every layer of Sharpee at once: the action logic, the grammar that recognizes the words, and the language that holds the text. This chapter feeds the goats and snaps photos to show all three.

Wiring an action touches several packages, so this chapter's imports span a few:

```
import { Action, ActionContext, ValidationResult } from '@sharpee/stdlib';
import type { Parser } from '@sharpee/parser-en-us';
import type { LanguageProvider } from '@sharpee/lang-en-us';
import { ISemanticEvent } from '@sharpee/core';
import { IdentityTrait, IFEntity, EntityType } from '@sharpee/world-model';
```

The action objects below are top-level `const`s. The three registration methods (`getCustomActions`, `extendParser`, `extendLanguage`) are members of your `FamilyZooStory` class, alongside `initializeWorld`.

14.1 The four-phase action

Every action, standard or custom, implements the same four-phase pattern you met in *The Standard Actions*. A custom action is just an `Action` object with those four methods:

```

const feedAction: Action = {
  id: 'zoo.action.feeding',
  group: 'interaction',

  // Phase 1: can the action proceed?
  validate(context: ActionContext): ValidationResult {
    if (!hasRequiredItem) return { valid: false, error:
      'no_feed' };
    return { valid: true };
  },

  // Phase 2: mutate the world (only runs if valid)
  execute(context: ActionContext): void {
    context.world.setStateValue('item-used', true);
  },

  // Phase 3: success events (text output)
  report(context: ActionContext): ISemanticEvent[] {
    return [context.event('zoo.event.fed', { messageId:
      'fed_goats' })];
  },

  // Phase 4: failure events (runs instead of execute/report if
  // invalid)
  blocked(context: ActionContext, result: ValidationResult):
    ISemanticEvent[] {
    return [context.event('zoo.event.feeding_blocked',
      { messageId: result.error })];
  },
};

```

The engine runs them in order: `validate()` first; if it returns `{ valid: false }` it jumps straight to `blocked()` ; otherwise it calls `execute()` then `report()` . Validation checks, world mutation, success text, and failure text each live in their own phase, never tangled together.

14.2 A complete custom action: feeding

Here's the feed action in full. It confirms the player is carrying feed, that the target is a feedable animal, and that the animal hasn't already eaten:

```

const FEED_ACTION_ID = 'zoo.action.feeding';

const FeedMessages = {
  NO_FEED: 'zoo.feeding.no_feed',
  NOT_AN_ANIMAL: 'zoo.feeding.not_animal',
  ALREADY_FED: 'zoo.feeding.already_fed',
  FED_GOATS: 'zoo.feeding.fed_goats',
  FED_RABBITS: 'zoo.feeding.fed_rabbits',
  FED_GENERIC: 'zoo.feeding.fed_generic',
} as const;

const feedAction: Action = {
  id: FEED_ACTION_ID,
  group: 'interaction',

  validate(context: ActionContext): ValidationResult {
    const target = context.command.directObject?.entity;

    // Player must be carrying the feed
    const inventory =
      context.world.getContents(context.player.id);
    const hasFeed = inventory.some(item =>
      item.get(IdentityTrait)?.aliases?.includes('feed'));
    if (!hasFeed) return { valid: false, error:
      FeedMessages.NO_FEED };

    // There must be a target, and it must be feedable
    if (!target) return { valid: false, error:
      FeedMessages.NOT_AN_ANIMAL };
    const name = target.get(IdentityTrait)?.name?.toLowerCase()
      || '';
    const feedable = ['pygmy goats', 'rabbits'].some(a =>
      name.includes(a));
    if (!feedable) return { valid: false, error:
      FeedMessages.NOT_AN_ANIMAL };

    // Not already fed
    if (context.world.getStateValue(`fed-${target.id}`)) {
      return { valid: false, error: FeedMessages.ALREADY_FED };
    }

    // Hand the target to the later phases
    context.sharedData.feedTarget = target;
    return { valid: true };
  },

```

```

execute(context: ActionContext): void {
    const target = context.sharedData.feedTarget as IFEntity;
    if (target) context.world.setStateValue(`fed-${target.id}`,
        true);
},

report(context: ActionContext): ISemanticEvent[] {
    const target = context.sharedData.feedTarget as IFEntity;
    const name = target?.get(IdentityTrait)?.name?.toLowerCase()
        || '';

    let messageId: string = FeedMessages.FED_GENERIC;
    if (name.includes('goats')) messageId =
        FeedMessages.FED_GOATS;
    else if (name.includes('rabbits')) messageId =
        FeedMessages.FED_RABBITS;

    return [context.event('zoo.event.fed', {
        messageId,
        params: { animal: target?.get(IdentityTrait)?.name ||
            'animal' },
    })];
},

blocked(context: ActionContext, result: ValidationResult):
    ISemanticEvent[] {
    return [context.event('zoo.event.feeding_blocked', {
        messageId: result.error || FeedMessages.NOT_AN_ANIMAL,
    })];
},
};

```

14.2.1 Passing data between phases

Notice `context.sharedData.feedTarget.validate()` already did the work of finding and checking the target, so it stashes the result in `sharedData` for `execute()` and `report()` to reuse. This is the sanctioned way to carry data forward: don't recompute the target in every phase, and don't smuggle it onto the context object itself.

14.3 A second action: photographing

`getCustomActions` and the grammar further down both reference a `photographAction`; here it is in full. It's simpler than feeding: it checks the player is carrying a camera, then reports a photo of whatever they aimed at.

```

const PHOTOGRAPH_ACTION_ID = 'zoo.action.photographing';

const PhotoMessages = {
  NO_CAMERA: 'zoo.photo.no_camera',
  TOOK_PHOTO: 'zoo.photo.took_photo',
} as const;

const photographAction: Action = {
  id: PHOTOGRAPH_ACTION_ID,
  group: 'interaction',

  validate(context: ActionContext): ValidationResult {
    const inventory =
      context.world.getContents(context.player.id);
    const hasCamera = inventory.some(item =>
      item.get(IdentityTrait)?.aliases?.includes('camera'));
    if (!hasCamera) return { valid: false, error:
      PhotoMessages.NO_CAMERA };

    const target = context.command.directObject?.entity;
    if (target) context.sharedData.photoTarget = target;
    return { valid: true };
  },

  execute(_context: ActionContext): void {
    // Photographs are cosmetic; nothing in the world changes.
  },

  report(context: ActionContext): ISemanticEvent[] {
    const target = context.sharedData.photoTarget as IEntity |
      undefined;
    const name = target?.get(IdentityTrait)?.name || 'the
      scenery';
    return [context.event('zoo.event.photographed', {
      messageId: PhotoMessages.TOOK_PHOTO,
      params: { target: name },
    })];
  },

  blocked(context: ActionContext, result: ValidationResult):
    ISemanticEvent[] {
    return [context.event('zoo.event.photographing_blocked', {
      messageId: result.error || PhotoMessages.NO_CAMERA,
    })];
  }
};

```



```
    },  
  };
```

The camera it looks for is an ordinary item. Add it to the gift shop (the room from Chapter 13) in `initializeWorld`:

```
const camera = world.createEntity('disposable camera',  
    EntityType.ITEM);  
camera.add(new IdentityTrait({  
    name: 'disposable camera',  
    description: 'A cheap yellow disposable camera with "ZOO  
        MEMORIES" on the side.',  
    aliases: ['camera', 'disposable camera'],  
    article: 'a',  
}));  
world.moveEntity(camera.id, giftShop.id);
```

14.4 Telling the engine: `getCustomActions`

An `Action` object does nothing until the engine knows about it. Return your actions from `getCustomActions()` on the story:

```
getCustomActions(): any[] {  
    return [feedAction, photographAction];  
}
```

14.5 Teaching the parser: `extendParser`

The engine now *has* the action, but the parser still doesn't recognize the word "feed." Map verb patterns to your action IDs in `extendParser()`:

```

extendParser(parser: Parser): void {
    const grammar = parser.getStoryGrammar();

    grammar.define('feed :thing').mapsTo(FEED_ACTION_ID).withPriority(150).build()

    grammar.define('photograph :thing').mapsTo(PHOTOGRAPH_ACTION_ID).withPriority(150)
    grammar.define('photo :thing').mapsTo(PHOTOGRAPH_ACTION_ID).withPriority(150)
    grammar.define('snap :thing').mapsTo(PHOTOGRAPH_ACTION_ID).withPriority(150)
}

```

A `:slot` (here `:thing`) is an argument the parser resolves to an entity. You can register several patterns for the same action; `photo` and `snap` are aliases for `photograph`. Use `withPriority(150)` (or higher) so your story patterns win over any stdlib defaults. Grammar gets its own chapter in Volume V; this is the minimum to make a custom verb fire.

14.6 Supplying the text: `extendLanguage`

The action returns *message IDs*, not sentences. Register the actual text in `extendLanguage()`:

```

extendLanguage(language: LanguageProvider): void {
    // Feed action: every FeedMessages id needs text, or the player
    // sees raw ids.
    language.addMessage(FeedMessages.NO_FEED,
        "You don't have any animal feed.");
    language.addMessage(FeedMessages.NOT_AN_ANIMAL,
        "That's not something you can feed.");
    language.addMessage(FeedMessages.ALREADY_FED,
        "You've already fed them. They look contentedly full.");
    language.addMessage(FeedMessages.FED_GOATS,
        'You scatter some feed on the ground. The pygmy goats rush
        over, ' +
        'bleating excitedly, and devour the corn and pellets in
        seconds.');
```

```

    language.addMessage(FeedMessages.FED_RABBITS,
        'You sprinkle some pellets near the rabbits. They hop over
        cautiously, ' +
        'then munch away happily, their little noses twitching.');
```

```

    language.addMessage(FeedMessages.FED_GENERIC,
        'You offer some feed. The animal eats it gratefully.');
```

```

    // Photograph action.
    language.addMessage(PhotoMessages.NO_CAMERA,
        "You don't have a camera. There's one in the gift shop.");
    language.addMessage(PhotoMessages.TOOK_PHOTO,
        "Click! You snap a photo of {target}. That one's going on the
        fridge.");
}

```

A `{param}` placeholder in the text is filled from the `params` object the action passed, so `params: { target: name }` substitutes into `{target}`. Keeping text out of the action and in the language layer is what lets a story be translated or re-voiced without touching its logic.

The mistake everyone makes once: wiring up only part of the chain. A custom verb needs *all three* registrations: the action from `getCustomActions()`, the pattern from `extendParser()`, and the message text from `extendLanguage()`. Miss the grammar and the parser says it doesn't understand; miss the language and the player sees a raw message ID like `zoo.feeding.fed_goats`.

14.7 Try it

> south; east	Go to the Petting Zoo
> take feed	Get the bag of feed
> feed goats	The goats eat happily
> feed goats	"You've already fed them."
> photograph goats	"You don't have a camera." (blocked)
> west; west; west	Head to the Gift Shop
> take camera	Grab the camera
> photograph press	Click! Photo taken

14.8 Key takeaway

A custom action is an `Action` object with four phases. **Validate** checks whether the action can run. If it can, **execute** changes the world model as directed and **report** adds an event message to the turn's output; if it can't, **blocked** produces the failure message instead. Wiring it up takes three registrations: the action, its parser pattern, its language text. The object `context.sharedData` carries results between phases. Miss any one registration and the verb won't work.

15 Capability Dispatch: One Verb, Many Rules

The custom feed action in the last chapter did the same thing to every animal: check for feed, mark it fed, print a message. But some verbs mean different things depending on *what* you apply them to. Petting the goats is affectionate so they lean into your hand. Petting the parrot is a mistake because it bites. The verb is the same; the outcome belongs to the animal. That's **capability dispatch**: one verb, many behaviors, and the entity decides which one runs.

This chapter pulls in the capability-dispatch toolkit from the world-model, plus the action types from the last chapter:

```
import {
  ITrait, IFEntity,
  CapabilityBehavior, CapabilityValidationResult,
    CapabilitySharedData,
  CapabilityEffect, createEffect,
  registerCapabilityBehavior, hasCapabilityBehavior,
  findTraitWithCapability, getBehaviorForCapability,
} from '@sharp/ee/world-model';
import { Action, ActionContext, ValidationResult } from '@sharp/ee/stdlib';
import { ISemanticEvent } from '@sharp/ee/core';
```

15.1 When to reach for it

Pattern

Custom action (last chapter)

Capability dispatch (this chapter)

Use when

A new verb with one fixed behavior for every target

The same verb, but each entity responds differently

If “feed” always does the same thing, a plain custom action is right. If “pet” needs to bleat for goats and bite for parrots, you want dispatch.

15.2 The three pieces

Capability dispatch is built from a custom **trait** that declares a capability, a **behavior** that implements it, and a **registration** that links the two.

15.2.1 1. A trait that declares a capability

A custom trait lists the action IDs it responds to in a static `capabilities` array. `PettableTrait` also carries an `animalKind` so one trait type can stand in for several different animals:

```
class PettableTrait implements ITrait {
    static readonly type = 'zoo.trait.pettable' as const;
    static readonly capabilities = ['zoo.action.petting'] as const;
    readonly type = PettableTrait.type;

    readonly animalKind: 'goats' | 'rabbits' | 'parrot' | 'snake';
    constructor(kind: 'goats' | 'rabbits' | 'parrot' | 'snake') {
        this.animalKind = kind;
    }
}
```

Add it to entities like any other trait: `goats.add(new PettableTrait('goats'))`, `parrot.add(new PettableTrait('parrot'))`.

15.2.2 2. A behavior that implements the capability

A `CapabilityBehavior` follows the same four-phase shape as an action, but its methods receive the target `entity` directly. Because the registry allows **one behavior per trait type + capability**, the single `pettingBehavior` dispatches internally on `animalKind`:

```

const PetMessages = {
  PET_GOATS: 'zoo.petting.goats',
  PET_RABBITS: 'zoo.petting.rabbits',
  PET_PARROT: 'zoo.petting.parrot',
  CANT_PET: 'zoo.petting.cant_pet',
} as const;

const pettingBehavior: CapabilityBehavior = {
  validate(_entity, _world, _actorId, _shared):
    CapabilityValidationResult {
    return { valid: true };           // every pettable animal
                                     accepts a pet
  },

  execute(_entity, _world, _actorId, _shared): void {
    // no world mutation; petting is cosmetic
  },

  report(entity, _world, _actorId, _shared): CapabilityEffect[] {
    const pettable = entity.get(PettableTrait);
    let messageId: string = PetMessages.CANT_PET;
    switch (pettable?.animalKind) {
      case 'goats':   messageId = PetMessages.PET_GOATS; break;
      case 'rabbits': messageId = PetMessages.PET_RABBITS; break;
      case 'parrot':  messageId = PetMessages.PET_PARROT; break;
    }
    return [createEffect('zoo.event.pettled', {
      messageId,
      params: { target: entity.name },
    })];
  },

  blocked(entity, _world, _actorId, error, _shared):
    CapabilityEffect[] {
    return [createEffect('zoo.event.petting_blocked', {
      messageId: error,
      params: { target: entity.name },
    })];
  },
};

```

`report()` and `blocked()` return `CapabilityEffect[]`, built with the `createEffect()` helper, rather than events directly. The dispatch action turns those effects into semantic events.

The mistake everyone makes once: trying to register a separate behavior for each animal under the same trait and capability. The registry holds exactly one behavior per trait type + capability; a second registration replaces the first. Put the per-entity differences in the trait's own data (here `animalKind`) and branch on it inside the one behavior.

15.2.3 3. Registration that links trait to behavior

`registerCapabilityBehavior()` connects a trait type, a capability (action ID), and the behavior that handles them. Guard it with `hasCapabilityBehavior()` so it only registers once:

```
if (!
    hasCapabilityBehavior(PettableTrait.type,
        PETTING_ACTION_ID)) {
    registerCapabilityBehavior(
        PettableTrait.type,      // which trait
        PETTING_ACTION_ID,      // which capability
        pettingBehavior,        // which behavior
    );
}
```

15.3 The dispatch action

Something still has to receive the `pet` verb and route it to the right behavior. Writing it by hand shows exactly what dispatch does: find the trait that claims the capability, look up its behavior, and delegate each phase.


```

const PETTING_ACTION_ID = 'zoo.action.petting';

const pettingAction: Action = {
  id: PETTING_ACTION_ID,
  group: 'interaction',

  validate(context: ActionContext): ValidationResult {
    const entity = context.command.directObject?.entity;
    if (!entity) return { valid: false, error:
      PetMessages.CANT_PET };

    // Find the trait on the target that claims this capability
    const trait = findTraitWithCapability(entity,
      PETTING_ACTION_ID);
    if (!trait) return { valid: false, error:
      PetMessages.CANT_PET };

    // Look up the behavior registered for that trait + capability
    const behavior = getBehaviorForCapability(trait,
      PETTING_ACTION_ID);
    if (!behavior) return { valid: false, error:
      PetMessages.CANT_PET };

    // Delegate validation to the behavior
    const sharedData: CapabilitySharedData = {};
    const result = behavior.validate(entity, context.world,
      context.player.id, sharedData);
    if (!result.valid) return { valid: false, error:
      result.error };

    // Carry the resolved behavior into the later phases
    context.sharedData.capEntity = entity;
    context.sharedData.capBehavior = behavior;
    context.sharedData.capSharedData = sharedData;
    return { valid: true };
  },

  execute(context: ActionContext): void {
    const entity = context.sharedData.capEntity as IEntity;
    const behavior = context.sharedData.capBehavior as
      CapabilityBehavior;
    const shared = context.sharedData.capSharedData as
      CapabilitySharedData;
    if (entity && behavior) behavior.execute(entity,
      context.world, context.player.id, shared);
  },

```

```

report(context: ActionContext): ISemanticEvent[] {
  const entity = context.sharedData.capEntity as IFEntity;
  const behavior = context.sharedData.capBehavior as
    CapabilityBehavior;
  const shared = context.sharedData.capSharedData as
    CapabilitySharedData;
  if (!entity || !behavior) return [];
  const effects = behavior.report(entity, context.world,
    context.player.id, shared);
  return effects.map(effect => context.event(effect.type,
    effect.payload));
},

blocked(context: ActionContext, result: ValidationResult):
  ISemanticEvent[] {
  return [context.event('zoo.event.petting_blocked', {
    messageId: result.error || PetMessages.CANT_PET,
  })];
},
};

```

An entity with no `PettableTrait` falls out at `findTraitWithCapability()` and gets the `CANT_PET` message; petting the hay bale just tells you that you can't.

Worth knowing: the `stdlib` ships

`createCapabilityDispatchAction()`, a factory that builds exactly this find-trait-then-delegate action for you from a few options. The zoo writes it out by hand so the mechanism is visible, but in a real story you'd usually let the factory generate it.

This action is registered the same three ways as the feed action in the last chapter. Add it to `getCustomActions`:

```

getCustomActions(): any[] {
  return [feedAction, photographAction, pettingAction];
}

```

Give it grammar patterns in `extendParser`:

```
grammar.define('pet :thing').mapsTo(PETTING_ACTION_ID).withPriority(150).build
grammar.define('stroke :thing').mapsTo(PETTING_ACTION_ID).withPriority(150).build
```

And register its four message ids in `extendLanguage`. Without these, `petting` prints raw ids like `zoo.petting.goats`:

```
language.addMessage(PetMessages.PET_GOATS,
    'You pet the nearest goat. It leans into your hand and bleats happily; the ' +
    'others crowd around demanding equal attention.');
```

```
language.addMessage(PetMessages.PET_RABBITS,
    'You gently stroke one of the rabbits. Its fur is incredibly soft, and it ' +
    'twitches its nose at you contentedly.');
```

```
language.addMessage(PetMessages.PET_PARROT,
    'You reach toward the parrot. CHOMP! It nips your finger. "NO TOUCHING!" it ' +
    'squawks indignantly.');
```

```
language.addMessage(PetMessages.CANT_PET,
    "You can't pet that.");
```

Finally, the registration block from “3. Registration that links trait to behavior” runs once at the end of `initializeWorld`, after the animals exist.

15.4 Making the zoo’s animals pettable

The behavior and action are wired; now the animals need the trait. The goats (from Chapter 4) and rabbits (from Chapter 5) are already scenery in the Petting Zoo, so give each a `PettableTrait` carrying its kind. The parrot is new: add it to the Aviary as a perched bird. For now it just sits there (and bites); in Chapter 20 it becomes a full NPC that squawks and moves.

```
// The petting-zoo animals, already in the world, now pettable.
goats.add(new PettableTrait('goats'));
rabbits.add(new PettableTrait('rabbits'));

// A new resident of the Aviary: a scarlet macaw with a temper.
const parrot = world.createEntity('parrot', EntityType.ACTOR);
parrot.add(new IdentityTrait({
  name: 'parrot',
  description:
    'A magnificent scarlet macaw perched on a rope, watching you
    with one ' +
    'bright, calculating eye.',
  aliases: ['parrot', 'macaw', 'scarlet macaw'],
  article: 'a',
}));
parrot.add(new ActorTrait({ isPlayer: false }));
parrot.add(new PettableTrait('parrot'));
world.moveEntity(parrot.id, aviary.id);
```

With `goats`, `rabbits`, and `parrot` all carrying a `PettableTrait`, every `pet` command in the walkthrough below resolves, and each animal's `animalKind` selects its own outcome from the single behavior. (`ActorTrait` you met on the player in Chapter 2; here it simply marks the parrot as a character rather than an object.)

15.5 How it fits together

Player types: "pet goats"

- ↓ parser matches "pet :thing" → `zoo.action.petting`, `target = goats`
- ↓ `pettingAction.validate()`:
 - findTraitWithCapability(`goats`, `'zoo.action.petting'`) → `PettableTrait`
 - getBehaviorForCapability(`trait`, `'zoo.action.petting'`) → `pettingBehavior`
 - `behavior.validate()` → { `valid: true` }
- ↓ `pettingAction.execute()` → `behavior.execute()` (nothing)
- ↓ `pettingAction.report()` → `behavior.report()` → "The goat leans into your ha"

Type `pet parrot` and the same path runs, but `animalKind` is `'parrot'`, so the behavior returns the bite message instead. One verb, the entity decides.

15.6 Try it

> south; east	Go to the Petting Zoo
> pet goats	They lean in, bleating happily
> pet rabbits	Soft and fuzzy
> pet dispenser	"You can't pet that." (no PettableTrait)
> west; west	Aviary
> pet parrot	CHOMP! It bites!

15.7 Key takeaway

Capability dispatch lets each entity carry its own rule for a verb. A custom trait declares the capability, a `CapabilityBehavior` implements the four phases over that trait's data, and `registerCapabilityBehavior()` links them. The dispatch action (yours, or one built by `createCapabilityDispatchAction()`) finds the trait claiming the capability and delegates to its behavior, so entities without the trait get the can't-do-that message for free.

16 Custom Traits & Behaviors: Data and Logic, Kept Apart

Every object in the zoo has been assembled from traits, including:

`IdentityTrait`, `RoomTrait`, `ContainerTrait`, `LightSourceTrait`.

You’ve even written one: `PettableTrait`, back in *Capability Dispatch*.

This chapter steps back to the layer those traits live in, the world model, and shows how to build your own trait, and the **behavior** that owns the rules around it, when the standard kit doesn’t carry the state your story needs.

16.1 Traits are data; behaviors are rules

The world model draws a sharp line:

- A **trait** is *data*. It holds the state an entity carries (a name, a capacity, whether a light is lit) and nothing else. No logic.
- A **behavior** is *rules*. It’s the code that reads and changes trait data, enforcing whatever constraints apply.

Every built-in pair follows this split. `LightSourceTrait` is just fields:

`brightness`, `isLit`, `fuelRemaining`. The matching

`LightSourceBehavior` is where the *logic* lives: lighting fails when the fuel is gone, extinguishing flips the flag, “is it lit?” falls back to the switch. The trait never decides anything; the behavior decides everything.

This is the same principle you met in Chapter 10 from the action side: *behaviors own mutations, actions coordinate them*. Here you see the other half: the trait that holds the state, and the behavior that guards it.

16.2 Defining a custom trait

A trait is a small class that implements `ITrait`. It needs a `type` string (both as a static, so code can refer to it, and as an instance field, so the engine can identify it on an entity) and whatever data fields it carries. By convention, custom trait types take a story-specific prefix, `zoo.trait...`, to stay clear of the platform's built-ins.

Suppose the feed dispenser in the petting zoo should run dry after a few uses. The state it needs, a count of charges, isn't in any standard trait, so you write one:

```
import { ITrait } from '@sharpee/world-model';

export class DispenserTrait implements ITrait {
  static readonly type = 'zoo.trait.dispenser';
  readonly type = DispenserTrait.type;

  /** How many servings remain. */
  chargesRemaining = 3;

  constructor(data?: Partial<DispenserTrait>) {
    if (data) Object.assign(this, data);
  }
}
```

That's the whole trait: pure data and a constructor that copies in any overrides, exactly like the built-ins. You add it to an entity the same way you add any trait:

```
dispenser.add(new DispenserTrait({ chargesRemaining: 5 }));
```

16.3 Defining the behavior

The trait holds the count; the *rule* (“you can dispense only while charges remain, and each use spends one”) belongs in a behavior. A behavior is

typically a class of static methods that take the entity, fetch the trait, and change it:

This is the first time we import from one of our *own* files rather than a `@sharpee/*` package. The project's TypeScript is configured for Node's ESM resolution, which requires a `.js` extension on relative imports (the `.js` points at the compiled output of `dispenser-trait.ts`):

```
import { IEntity } from '@sharpee/world-model';
import { DispenserTrait } from './dispenser-trait.js';

export class DispenserBehavior {
  /** Spend one charge. Returns false if the dispenser is already empty. */
  static dispense(dispenser: IEntity): boolean {
    const trait = dispenser.get(DispenserTrait);
    if (!trait || trait.chargesRemaining <= 0) return false;
    trait.chargesRemaining -= 1;
    return true;
  }

  static isEmpty(dispenser: IEntity): boolean {
    const trait = dispenser.get(DispenserTrait);
    return !trait || trait.chargesRemaining <= 0;
  }
}
```

Notice what the behavior does and doesn't do. It performs the mutation (`chargesRemaining -= 1`) and returns a small result (`true / false`); it doesn't print anything or emit events. That keeps it pure and testable: you can call `DispenserBehavior.dispense(d)` in a test and assert the count dropped, with no parser, no turn, no text in the way. (The platform's own behaviors share this shape; many extend a small `Behavior` base class that adds helpers like a trait-required check, but the essence is just static methods over trait data.)

16.4 Putting the pair to work

A trait and behavior are inert on their own; something has to *call* the behavior. That caller is one of the coordination tools from earlier volumes:

```
// inside a custom "operate dispenser" action's execute phase, or
// an event handler:
if (DispenserBehavior.dispense(dispenser)) {
    // success: hand out a serving of feed
} else {
    // empty: report that it's out
}
```

The action or handler decides *when* to act and *what to say*; the behavior decides *what changes*. The trait just remembers. Each layer has one job.

16.5 Which tool, when?

Custom traits and behaviors are powerful, but they're not always the answer. Before reaching for a new trait, run down the lighter options, most of which you've already seen:

You want to...

React to something that already happened

Add a brand-new verb

Make one verb behave differently per entity

Give entities genuinely new *state* and *rules*

Reach for

An **event handler** (Chapter 13)

A **custom action** (Chapter 14)

Capability dispatch (Chapter 15)

A **custom trait + behavior** (this chapter)

The deciding question is whether your idea is really new *state on entities*. If an object needs to remember something the standard traits don't model (charges, a temperature, a loyalty score) and needs rules around how that

something changes, that's a trait-and-behavior. If you only need to react, or to add a verb, the lighter tools fit better.

16.6 Key takeaway

The world model keeps data and logic apart: a **trait** holds state and nothing else, while a **behavior** is pure static methods that read and mutate that state. Behaviors emit no text or events, which is what keeps them testable. Add a trait with `entity.add(...)` and namespace its type (`zoo.trait...`), then let actions or event handlers call the behavior to decide *when* and *what to say*. Reach for a custom trait only when your story genuinely needs new state and logic. For reactions, verbs, or per-entity behavior, the lighter tools from earlier chapters fit better.

VOLUME V — WORDS

17 Extending the Grammar: Teaching New Sentence Shapes

A parser's whole job is to make sense of words: to take a line a player typed and decide which action it means and what it acts on. You've already taught it a few new words: `feed`, `photograph`, `pet`, each wired up with a couple of grammar lines in `extendParser`. This volume is about words, and it starts where the player's words enter the system: the `grammar`.

17.1 How the parser matches

The parser holds a set of **patterns**, which are word-shapes paired with action IDs. When the player types a line, it finds the best-matching pattern, binds the pattern's slots to entities that are in scope (Chapter 11), and hands the result to the engine as a command. `take lamp` matches the pattern `take :item`, binds `:item` to the lamp in the room, and runs the taking action.

You don't touch the standard patterns. `take`, `drop`, `go`, and the rest come wired up. What you add are patterns for *your* verbs.

17.2 Where patterns go

Story grammar is registered in the `extendParser` hook on your story, through the `story grammar` builder:

```

extendParser(parser: Parser): void {
  const grammar = parser.getStoryGrammar();

  grammar
    .define('feed :thing')
    .mapsTo('zoo.action.feeding')
    .withPriority(150)
    .build();
}

```

Read that as a sentence: *define* the pattern `feed :thing`, map it to the feeding action, give it a *priority*, and *build* it (which registers it). Four calls, one pattern.

17.3 Slots

The `:thing` in `feed :thing` is a **slot**, a placeholder the parser fills with an entity from scope. The slot's name is how the action gets it back: a single slot becomes the command's *direct object*, which the action reads as `context.command.directObject?.entity`, exactly the line you saw in the feeding action in Chapter 14. Name a slot whatever reads well; `:thing`, `:item`, `:target`, `:animal` are all fine.

17.4 Aliases: many patterns, one action

Players say the same thing different ways. Register a pattern for each phrasing, all mapping to the same action:

```

grammar.define('photograph :thing').mapsTo('zoo.action.photographing').withPriority(150);
grammar.define('photo :thing').mapsTo('zoo.action.photographing').withPriority(150);
grammar.define('snap :thing').mapsTo('zoo.action.photographing').withPriority(150);

```

Now `photograph toucan`, `photo toucan`, and `snap toucan` all reach the same verb.

17.5 Prepositions and two slots

A pattern can carry a preposition and a second slot, for verbs that take both a direct and an indirect object. Suppose feeding should name *both* the food and the animal:

```
grammar
  .define('feed :food to :animal')
  .mapsTo('zoo.action.feeding')
  .withPriority(150)
  .build();
```

The first slot becomes the direct object, the second (after the preposition) the indirect object, which is

`context.command.indirectObject?.entity` in the action. This is the shape behind built-in commands like `unlock :door with :key` and `put :item in :container`.

Multi-word verbs, phrasal verbs like `pick up :item`, also go through `.define`, since the verb itself is more than one word.

17.6 Constraining a slot

By default a slot resolves to anything the player could plausibly mean. You can narrow it with `.where`, giving the slot a scope rule:

```
grammar
  .define('feed :animal')
  .where('animal', (scope: any) => scope.touchable())
  .mapsTo('zoo.action.feeding')
  .withPriority(150)
  .build();
```

The `(scope: any)` annotation on the callback is there to satisfy the strict `tsconfig.json` that `sharpee init` generates: `.where` accepts more than one kind of constraint, so TypeScript can't infer the

parameter's type on its own and `noImplicitAny` flags it. Annotating it keeps the build clean.

Keep these rules **permissive**, `touchable` rather than `visible`, for the reason from Chapter 11: let the parser resolve the noun, and let the action's `validate` phase make the strict call about whether sight (or anything else) is truly required. A grammar that demands full visibility forecloses perfectly good in-the-dark commands before the action ever runs.

17.7 Priority

Why `withPriority(150)`? When more than one pattern could match a line, the highest-priority one wins. The standard library's patterns sit at the default level, so giving your story patterns a priority of 150 (or higher) ensures a verb you define takes precedence over any `stdlib` pattern that might otherwise catch the same words. For brand-new verbs it rarely matters; for verbs that overlap a standard one, it's what puts your version in charge.

A note on the standard grammar. The platform defines its own verbs with an action-centric builder, `grammar.forAction('if.action.pushing').verbs(['push', 'press', 'shove']).pattern(':target')`, which generates a pattern per verb alias at once. As a story author you'll almost always use `.define` inside `extendParser` instead; `forAction` is how the library wires its built-ins.

17.8 Key takeaway

The parser matches the player's line against **patterns** that map word-shapes to action IDs. Add yours in `extendParser` via

```
parser.getStoryGrammar(), building each with  
.define(pattern).mapsTo(actionId).withPriority(150).build().
```

A `:slot` is filled from scope and reaches the action as its direct (or, after

a preposition, indirect) object; register several patterns to give one action multiple phrasings; constrain a slot with `.where` but keep it permissive; and use a priority of 150+ so your verbs outrank the standard ones. Grammar decides *which* action and *what* it acts on. The next chapter decides what the game *says* back.

18 The Language Layer: Messages & Message IDs

Chapter 3 promised that **actions emit events, not text**, that the words the player reads are produced later, somewhere else. This is that somewhere else. The **language layer** is the single place every user-facing string lives, and the bridge between an action's intent and the sentence the player sees is a **message ID**.

18.1 Intent here, words there

When the feeding action succeeds, it doesn't say "You scatter some feed." It emits an event carrying a *message ID* and some parameters:

```
context.event('zoo.event.photographed', {  
  messageId: 'zoo.photo.took_photo',  
  params: { target: name },  
});
```

`zoo.photo.took_photo` is not text. It's a name for a piece of text. At the end of the turn, the engine's prose pipeline takes that ID to the language layer and asks: *what does this say, in this language, with these parameters?* The answer is what prints.

The standard library works exactly the same way. Every built-in verb emits IDs like `if.action.taking.taken`; the English language package maps each to its prose. Nothing in the engine, `stdlib`, or world model ever hardcodes a sentence. It all flows through IDs.

18.2 Registering your text

You met the registration side back in *Custom Actions*. A story supplies text for its IDs in the `extendLanguage` hook, with `addMessage(id, template)`:

```
extendLanguage(language: LanguageProvider): void {
  language.addMessage('zoo.feeding.fed_goats',
    'You scatter some feed on the ground. The pygmy goats rush
      over, bleating ' +
    'excitedly, and devour the corn and pellets in seconds.');
```



```
  language.addMessage('zoo.photo.took_photo',
    "Click! You snap a photo of {target}. That one's going on the
      fridge.");
}
```

Each call ties one ID to one template. When the engine later looks up `zoo.feeding.fed_goats`, it finds this string and renders it.

18.3 Parameters

A template can carry **placeholders** in curly braces, filled from the `params` the event supplied. The photograph action passed `params: { target: name }`, and the template's `{target}` is where that value lands, so photographing the toucan reads “Click! You snap a photo of the toucan.” Placeholders are how one message adapts to many situations without a separate string for each.

(There's more to placeholders than plain substitution: when a parameter is an *entity*, the language layer can render its name with the right article and capitalization: “the toucan,” “a flashlight,” “Some feed.” That machinery is the **formatter chain**, and it's the whole of the next chapter.)

18.4 Naming message IDs

IDs are just strings, but a consistent scheme keeps them legible. The convention:

- Built-in messages use the `if.*` namespace, as in `if.action.taking.taken`.
- Your story's messages take a story prefix, such as `zoo.feeding.fed_goats`, `zoo.photo.no_camera`.

A descriptive, namespaced ID reads almost like documentation at the call site, and keeps your messages from colliding with the platform's.

18.5 Why route everything through IDs

The indirection buys real things, all of which come from keeping text in one layer instead of scattered through logic:

- **Translation.** A French language package maps the same IDs to French prose; the story's code never changes.
- **Restyling.** Terse or florid, second person or third: swap the templates, keep the behavior.
- **Consistency.** Every “you can't see that” reads the same because it's one string, registered once.
- **Overrides.** Supplying your own text for an existing ID replaces what the player sees there, a way to reskin a standard response in your story's voice without touching the action behind it.

Wherever your story produces player-facing words, whether in actions or in the event handlers from Chapter 13, prefer a message ID over an inline string, so the text stays in the layer built to hold it.

18.6 Key takeaway

All user-facing text lives in the language layer or in your story; code refers to messages by **ID**, never a literal string. Actions emit an ID plus

`params` , and the prose pipeline resolves it to a template at turn end. Registering text means `addMessage` in `extendLanguage` , namespacing your IDs beside the platform's, and reusing an ID to override a standard message. That separation keeps intent in the code and the words in the language layer, and it is what makes a Sharpee story translatable, restyleable, and consistent.

19 The Formatter Chain: Grammar in the Template, Not the Text

The last chapter left a thread hanging: when a message parameter is an *entity*, the language layer renders its name with the right article and capitalization: “the toucan,” “a flashlight,” “Some feed.” It does that with the **formatter chain**, a small grammar inside your message templates. This short chapter pulls that thread.

19.1 The problem with hardcoded articles

Suppose you write a template by hand:

```
You pick up the {item}.
```

It reads fine for “the brass key,” but the moment the object is a flashlight you wanted “a flashlight,” or a proper name (“you pick up Captain,” no article at all), or a mass noun (“you scatter some feed”), the baked-in “the” is wrong. English articles depend on the noun, and you don’t want a separate message for every object. The formatter chain moves that decision out of the literal text and into a **formatter**.

19.2 Formatter syntax

Inside a template, a placeholder can name a formatter before the parameter, separated by a colon: `{formatter:param}`. The **article formatters** pick the correct word for you:

You pick up {the:item}.	→ You pick up the brass key.
You scatter {some:item}.	→ You scatter some feed.
There is {a:item} here.	→ There is a flashlight here.

`{a:item}` even chooses *a* versus *an* by the word that follows: “a flashlight,” “an owl.” The full set of article formatters is `a / an`, `the`, `some` (for mass nouns), and `your`.

19.3 Why pass the entity, not its name

An article formatter can only choose correctly if it knows what kind of noun it’s dealing with: is it a proper name? a mass noun? a plural? That information lives on the entity (the `properName`, `article`, and `noun-type` fields you set on an `IdentityTrait` back in Volume II). So when an action supplies a parameter that the template will article-format, it passes the **entity’s information**, not a bare string. Hand the formatter a plain string and it can only guess from spelling; hand it the entity and it knows that “Captain” takes no article and “feed” takes “some.”

This is the practical reason the earlier chapters were careful about `IdentityTrait`’s `properName` and `article` fields: the formatter chain is what finally reads them.

19.4 Capitalization and other text formatters

Articles aren’t the only thing that varies by position. A name at the start of a sentence needs a capital, even though “the toucan” is lowercase mid-sentence. That’s another formatter, and formatters **chain**: list several, separated by colons, and they apply in turn:

```
{the:cap:item} blocks your way. → The toucan blocks your way.
```

`{the:cap:item}` builds “the toucan” and then capitalizes it. The text formatters are `cap` (capitalize the first letter), `upper`, `lower`, and `title`.

19.5 Verb agreement

A verb has to agree with its subject: “the toucan **is** fixed in place,” but “the pygmy goats **are** fixed in place.” Rather than hardcode `is`, a template keys a **verb formatter** to the same entity:

```
{the:cap:item} {is:item} fixed in place.  
→ The toucan is fixed in place.      (singular)  
→ The pygmy goats are fixed in place. (plural)
```

`{is:item}` emits “is” or “are” depending on the entity’s number, the same `grammaticalNumber: 'plural'` flag you set back in Chapter 5 (or `.plural()` on the `object()` builder). The companions `{was:item}` (was/were) and `{has:item}` (has/have) work the same way. An entity with no number metadata is treated as singular, so existing singular objects read exactly as before. This is why marking plural-named scenery matters: the platform’s standard messages already use these formatters, so one flag on the entity keeps every generated line grammatical.

19.6 Lists

The chain also handles collections. `{list:items}` turns an array of entities into a natural English list. It gives each one its article, groups identical items into counts, and joins them with commas and a final “and”:

```
You see {list:items} here.  
→ You see a goat, two rabbits, and a parrot here.
```

One formatter does all of it: the articles (“a goat”), the grouping (“two rabbits”), and the join. Use `{the-list:items}` for the definite form (“the goat, the rabbit, and the parrot”), and `{count:items}` for a bare quantity (“three coins”). The serial (Oxford) comma is on by default, and a story can turn it off.

19.7 Key takeaway

The formatter chain keeps English grammar in the template's placeholders, not your literal text. Article formatters (`{a:item}` , `{the:item}`) pick the right word from the entity's own metadata, which is why you pass the *entity*, not a bare name. Text formatters like `{cap:...}` handle capitalization, and you chain them with colons to stack more than one on a value, as in `{the:cap:item}` . Write one message, and it reads correctly for every object it's ever handed. With grammar, language, and formatting covered, the words side of Sharpee is complete.

VOLUME VI — LIVING WORLDS

20 Non-Player Characters: Actors That Take Turns

Until now the zoo has been still. Animals are scenery, signs wait to be read, machines wait to be used. Nothing moves unless the player moves it. A **non-player character** changes that. Sam the zookeeper walks a patrol between the main path, the petting zoo, and the aviary. A scarlet macaw sits on its perch and squawks at random, and greets you when you walk in. The world starts to feel inhabited.

Sharpee's NPC system has three parts that work together:

1. **NpcTrait** : the trait that marks an entity as an NPC.
2. **NpcBehavior** : an object that decides what the NPC does each turn.
3. **NpcPlugin** : an engine plugin that gives NPCs their own phase in the turn.

These span four packages: the engine, the world-model, the NPC plugin, and the stdlib.

```
import { GameEngine } from '@sharpee/engine';
import { NpcTrait } from '@sharpee/world-model';
import { NpcPlugin } from '@sharpee/plugin-npc';
import { NpcBehavior, NpcContext, NpcAction,
  createPatrolBehavior } from '@sharpee/stdlib';
```

`parrotBehavior` further down is a top-level `const`; the entity creation and `onEngineReady` are members of your `FamilyZooStory` class.

20.1 Creating an NPC entity

An NPC is an actor, not an item. It needs three traits: `IdentityTrait` for name and description, `ActorTrait` with `isPlayer: false` to mark

it as a character rather than the player, and `NpcTrait` to connect it to a behavior:

```
const zookeeper = world.createEntity('zookeeper',
    EntityType.ACTOR);

zookeeper.add(new IdentityTrait({
  name: 'zookeeper',
  description:
    'A friendly zookeeper in khaki overalls and a wide-brimmed hat, ' +
    'carrying a bucket of mixed animal feed. A name tag reads "Sam."',
  aliases: ['keeper', 'zookeeper', 'sam'],
  properName: false,
  article: 'a',
}));

zookeeper.add(new ActorTrait({ isPlayer: false }));

zookeeper.add(new NpcTrait({
  behaviorId: 'zoo-keeper-patrol', // must match the behavior's
    id
  canMove: true, // allowed to change rooms
  isAlive: true,
  isConscious: true,
}));

world.moveEntity(zookeeper.id, mainPath.id);
```

The `behaviorId` is the crucial link: it must exactly match the `id` of a behavior you register later. `canMove` decides whether this NPC is allowed to walk between rooms. The parrot, which stays put, sets it to `false`.

The mistake everyone makes once: a `behaviorId` that doesn't match any registered behavior's `id`. The NPC exists and you can examine it, but it never acts, because the NPC service can't find a behavior to run for it. Keep the two strings identical.

20.1.1 The parrot becomes an NPC

The parrot already exists; you created it in Chapter 15 as a pettable actor in the Aviary. Turning it into an NPC is one more trait on that same entity, linking it to the behavior we write below:

```
// `parrot` is the entity from Chapter 15 (Aviary, already an
    ACTOR).
parrot.add(new NpcTrait({
  behaviorId: 'zoo-parrot',    // matches parrotBehavior.id, below
  canMove:   false,           // it stays on its perch
  isAlive:   true,
  isConscious: true,
}));
```

So the zookeeper is a brand-new NPC while the parrot is an existing actor promoted to one. Both routes end at the same place: an actor with an `NpcTrait` whose `behaviorId` names a behavior.

20.2 Built-in behaviors

The stdlib ships several behaviors ready to use, so common NPCs need no custom code:

Behavior	What it does
<code>createPatrolBehavior({ route, loop, waitTurns })</code>	Walk a fixed route of rooms
<code>createWandererBehavior({ moveChance })</code>	Move randomly between rooms
<code>createFollowerBehavior({ immediate })</code>	Follow the player
<code>guardBehavior</code>	Stand guard, block passage, fight
<code>passiveBehavior</code>	Do nothing (react-only NPCs)

The zookeeper uses `createPatrolBehavior`: give it a route of room IDs and it walks them in order, finding the exits on its own.

20.3 Writing a custom behavior

When the built-ins don't fit, implement the `NpcBehavior` interface yourself. Its only required hook is `onTurn`, called every turn; the others fire on specific events. The parrot squawks a random phrase when the player is present and greets them on arrival:

```

const PARROT_PHRASES = [
  'Polly wants a cracker!',
  'SQUAWK! Pretty bird! Pretty bird!',
  'Pieces of eight! Pieces of eight!',
  "Who's a good bird? WHO'S A GOOD BIRD?",
  'BAWK! Welcome to the zoo!',
];

const parrotBehavior: NpcBehavior = {
  id: 'zoo-parrot',
  name: 'Parrot Behavior',

  // Called every turn, whether or not the player is here.
  onTurn(context: NpcContext): NpcAction[] {
    if (!context.playerVisible) return []; // no audience,
    stay quiet

    if (context.random.chance(0.5))
    {
      // 50% chance to squawk
      const phrase = context.random.pick(PARROT_PHRASES);
      return [{
        type: 'speak',
        messageId: 'npc.speech',
        data: { npcName: 'parrot', text: phrase },
      }];
    }
    return [];
  },

  // Called once when the player walks into the parrot's room.
  onPlayerEnters(context: NpcContext): NpcAction[] {
    return [{
      type: 'emote',
      messageId: 'npc.emote',
      data: {
        npcName: 'parrot',
        text: 'The parrot ruffles its feathers and eyes you with
        interest.',
      },
    },
  ]];
  },
};

```

`NpcContext` hands the behavior everything it needs: `playerVisible` (is the player in this room?), `random` for chance and selection, and access to the NPC and the world. Each hook returns an array of `NpcAction` describing what the NPC does this turn:

Action	What it does
<code>{ type: 'move', direction: Direction.NORTH }</code>	Walk to a connected room
<code>{ type: 'speak', messageId, data }</code>	Say something (visible text)
<code>{ type: 'emote', messageId, data }</code>	Do something visible
<code>{ type: 'wait' }</code>	Do nothing this turn
<code>{ type: 'take', target } /</code> <code>{ type: 'drop', target }</code>	Pick up / drop an item

Return an empty array for a turn where the NPC does nothing.

The `npc.speech` and `npc.emote` message ids the behavior emits are provided by the platform's language layer (`@sharpee/lang-en-us`). You don't register them in `extendLanguage` . They render verbatim the `text` you pass in each action's `data` .

20.4 Registering the plugin and behaviors

NPC behaviors don't fire until the `NpcPlugin` is registered with the engine. That happens in `onEngineReady()` , the story hook called after the engine is fully built, which is where any plugin needing the engine reference is set up. The patrol route references `this.roomIds` , the field you started in Chapter 13; make sure `initializeWorld` records `mainPath` , `pettingZoo` , and `aviary` there so the route can name them:

```

onEngineReady(engine: GameEngine): void {
    // 1. Create and register the plugin: gives NPCs a turn phase
    const npcPlugin = new NpcPlugin();
    engine.getPluginRegistry().register(npcPlugin);

    // 2. Get the NPC service from the plugin
    const npcService = npcPlugin.getNpcService();

    // 3. Build the zookeeper's patrol from a route of room IDs
    const keeperPatrol = createPatrolBehavior({
        route: [this.roomIds.mainPath, this.roomIds.pettingZoo,
            this.roomIds.aviary],
        loop: true,           // Main Path → Petting Zoo → Aviary → Main
                             // Path → ...
        waitTurns: 1,        // pause one turn at each stop
    });

    // The factory's default id is 'patrol'; override it to match
    // NpcTrait.behaviorId
    keeperPatrol.id = 'zoo-keeper-patrol';
    npcService.registerBehavior(keeperPatrol);

    // 4. Register the parrot's custom behavior (its id already
    // matches)
    npcService.registerBehavior(parrotBehavior);
}

```

Two registrations are needed and both matter: register the *plugin* (without it, no NPC acts at all) and register each *behavior* (without it, an NPC with that `behaviorId` has nothing to run).

Note the patrol factory: it returns a behavior whose `id` defaults to `'patrol'`, so the zookeeper's `behaviorId: 'zoo-keeper-patrol'` wouldn't match until you override `keeperPatrol.id`. The parrot needs no override; `parrotBehavior` was defined with `id: 'zoo-parrot'` to begin with.

20.5 Try it

> south	Walk to the Main Path, where Sam patrols
> examine zookeeper	See Sam's description
> wait	Sam patrols on toward the petting zoo
> wait	...and on toward the aviary
> west	Aviary, meet the parrot
> examine parrot	See the macaw
> wait	The parrot might squawk
> wait	...or not; it's a coin flip each turn

20.6 Key takeaway

An NPC is an actor carrying `IdentityTrait`, `ActorTrait`, and `NpcTrait`, with a `behaviorId` that matches a registered behavior, whether you use a built-in such as `createPatrolBehavior` or write your own `NpcBehavior`, whose `onTurn` and `onPlayerEnters` return `NpcAction[]`. Nothing acts until `onEngineReady()` does *both*: registers the `NpcPlugin` with the engine and registers each behavior with its service.

21 Scenes: Named Windows of Story Time

A world that only reacts command-by-command stays flat. Stories have *phases*: a storm that rolls in and passes, a market that's busy by day, a tense stretch while an alarm blares. Sharpee models these as **scenes**: named windows of story time that switch on and off based on conditions you write, with the engine watching the clock for you.

21.1 What a scene is

A scene is a phase of the story with a beginning and an end. You give it two conditions, when it should *begin* and when it should *end*, and the engine checks them every turn, flipping the scene between three states:

```
waiting —[begin() is true]—> active —[end() is true]—> ended
```

While a scene is **active**, you can key behavior to it; once it's **ended**, that behavior stops. You don't poll or schedule anything yourself. You describe the window, and the engine manages the lifecycle.

21.2 Creating a scene

Scenes are created in `initializeWorld()` with `world.createScene()`. The scene methods (`createScene`, `isSceneActive`, `hasSceneHappened`, `hasSceneEnded`) are all on the `WorldModel` you already have in scope; the one symbol you import is `SceneTrait` (used later to read a scene's `activeTurns`), from `@sharpee/world-model`. The `begin` and `end` options are predicates over the world; each returns `true` when its moment has come. Here's a scene that's active only while the visitor is in the petting zoo:

```
world.createScene('scene-petting-zoo', {
  name: 'Among the Animals',
  begin: (w) => w.getLocation(w.getPlayer()!.id) ===
    pettingZoo.id,
  end: (w) => w.getLocation(w.getPlayer()!.id) !==
    pettingZoo.id,
  recurring: true,
});
```

`recurring: true` lets the scene reactivate: leave the petting zoo and come back, and it begins again. Without it (the default), a scene fires once and stays ended.

21.3 Querying a scene

Anywhere in your story logic you can ask the world about a scene's state:

```
if (world.isSceneActive('scene-petting-zoo')) {
  // the player is among the animals right now
}

if (world.hasSceneHappened('scene-finale')) {
  // the finale has run at least once
}
```

`isSceneActive` is the common one; `hasSceneHappened` and `hasSceneEnded` let you check whether a phase has already played out.

21.4 Reacting to transitions

The real power is reacting to the *edges*: the moment a scene begins or ends. You write those reactions as `onBegin` and `onEnd` callbacks, right next to the `begin` and `end` conditions in `createScene`. Each callback returns the text the player should see at that edge, either prose directly (`{ text }`) or a message id resolved through your language file (`{ messageId }`):

```

world.createScene('scene-petting-zoo', {
  name: 'Among the Animals',
  begin: (w) => w.getLocation(w.getPlayer()!.id) ===
    pettingZoo.id,
  end: (w) => w.getLocation(w.getPlayer()!.id) !==
    pettingZoo.id,
  recurring: true,
  onBegin: () => ({ text: 'A waft of hay and warm fur greets
    you.' }),
  onEnd: () => ({ text: 'The animal sounds fade behind you.' }),
});

```

The callback receives a typed context (`sceneId` , `sceneName` , `turn` , the `world` , and, on `onEnd` , `totalTurns` , how long the scene ran), so you can vary the line:

```

onEnd: (ctx) => ({ text: `You spent ${ctx.totalTurns} turns
  among the animals.` }),

```

Return nothing for a state-only beat (a scene whose edges drive logic but print no line). To return more than one line, return an array of reactions. This is where atmosphere and staged events live: open a sequence when a scene begins, close it down when the scene ends; and because the reaction is part of the scene definition, the condition and its payoff sit together.

The engine still emits `if.event.scene_began` / `if.event.scene_ended` as observable facts (for perception, tooling, and transcript tests), but author reactions go through `onBegin` / `onEnd` , not the event-handler bus.

21.5 Common shapes

Most scenes are one of a few patterns, all expressed through `begin` / `end` :

- **Location-based:** active while the player is somewhere, as above; `begin` and `end` test the player's room. Usually `recurring` .

- **One-shot trigger:** `begin` watches a flag (`w.getStateValue('alarmTripped')`) and `end` fires after a turn or two, so the beat plays once and never returns.
- **Timed:** `end` checks how long the scene has run. A scene's `SceneTrait` tracks `activeTurns`, so a storm can last a fixed stretch:

```
world.createScene('scene-storm', {
  name: 'Thunderstorm',
  begin: (w) => w.getStateValue('stormTriggered') === true,
  end:   (w) => (w.getEntity('scene-
    storm')?.get(SceneTrait)?.activeTurns ?? 0) >= 15,
});
```

21.6 Scenes versus timed events

The next chapter covers **daemons** and **fuses**: per-turn machinery for things that *do* something each turn or fire after a countdown. Scenes sit a level above that: a scene answers “*is this phase of the story on right now?*” It’s state, not action. A common division of labor is a scene that defines the window and a daemon that does the per-turn work *while* that window is open; the daemon simply checks `world.isSceneActive(...)` in its condition. Reach for a scene when you’re thinking in story beats; reach for a daemon or fuse when you’re thinking in turns.

21.7 Key takeaway

A scene is a named phase of the story with `begin` and `end` conditions the engine checks each turn, cycling `waiting` → `active` → `ended` (and back to `waiting` when `recurring`). Create them with `world.createScene()` in `initializeWorld()`, query state with `world.isSceneActive()` / `hasSceneHappened()`, and react to the edges with the scene’s own `onBegin` / `onEnd` callbacks. Reach for a scene when a phase of the story needs a defined window; let a daemon or fuse handle whatever happens turn-by-turn inside it.

22 Turns, Timed Events & Daemons: Giving the World a Clock

A living world doesn't only react to the player. It has a clock of its own. Announcements crackle over the zoo PA as closing time approaches. Feeding time arrives on a schedule, and if you miss it the goats bleat for a few turns until they're fed or give up. None of this is triggered by a command; it happens *because time passes*. Sharpee provides that clock through the **SchedulerPlugin**, which after every player turn runs two kinds of timed event: **daemons** and **fuses**.

22.1 How the scheduler ticks

Register the `SchedulerPlugin` and, after each player turn, it “ticks”: it runs every active daemon and counts down every active fuse. A **daemon** is a function that runs each turn: a background process, a ticking clock. A **fuse** is a countdown timer that fires once when it reaches zero, optionally re-arming to fire again.

Registration follows the same `onEngineReady()` pattern as the NPC plugin, and in fact lives in the *same* `onEngineReady`, alongside the NPC registration from Chapter 20. The daemon `run` functions return `ISemanticEvent[]`, and one reads an `IdentityTrait`, so the imports grow a little:

```

import { SchedulerPlugin } from '@sharpee/plugin-scheduler';
import type { Daemon, Fuse, SchedulerContext } from '@sharpee/
  plugin-scheduler';
import { ISemanticEvent } from '@sharpee/core';
import { IdentityTrait } from '@sharpee/world-model';

onEngineReady(engine: GameEngine): void {
  // ... the NPC plugin registration from Chapter 20 stays here ...

  const schedulerPlugin = new SchedulerPlugin();
  engine.getPluginRegistry().register(schedulerPlugin);
  const scheduler = schedulerPlugin.getScheduler();

  scheduler.registerDaemon(createPAAnnouncementDaemon());
  scheduler.setFuse(createFeedingTimeFuse());
  scheduler.registerDaemon(createGoatBleatingDaemon());
}

```

The daemons and fuse below emit message ids from a `TimedMessages` table; declare it once, near the top of your story module:

```

const TimedMessages = {
  PA_CLOSING_3: 'zoo.pa.closing_3',
  PA_CLOSING_2: 'zoo.pa.closing_2',
  PA_CLOSING_1: 'zoo.pa.closing_1',
  PA_CLOSED: 'zoo.pa.closed',
  FEEDING_TIME: 'zoo.feeding_time.announced',
  GOATS_BLEATING: 'zoo.goats.bleating',
} as const;

```

Both daemons and fuses receive a `SchedulerContext` (`{ world, turn, random, playerLocation, playerId }`), giving them the turn number, the world, and where the player is.

22.2 Daemons: run every turn

A `Daemon` has an `id`, a `name`, a `run` function that returns events, and an optional `condition` that gates when it runs. Here's the PA

announcer, which fires every fifth turn and walks through a sequence of closing announcements:

```
function createPAAnnouncementDaemon(): Daemon {
  let announcementCount =
    0;           // internal state, kept across turns

  return {
    id: 'zoo.daemon.pa_announcements',
    name: 'Zoo PA Announcements',
    priority: 5,           // low; runs after more
                          // important daemons

    // Only run every 5th turn, and only four times total
    condition: (ctx: SchedulerContext): boolean =>
      ctx.turn > 0 && ctx.turn % 5 === 0 && announcementCount < 4,

    run: (ctx: SchedulerContext): ISemanticEvent[] => {
      announcementCount++;
      let messageId: string;
      switch (announcementCount) {
        case 1: messageId = TimedMessages.PA_CLOSING_3; break;
        case 2: messageId = TimedMessages.PA_CLOSING_2; break;
        case 3: messageId = TimedMessages.PA_CLOSING_1; break;
        default: messageId = TimedMessages.PA_CLOSED; break;
      }
      return [{
        id: `zoo-pa-${ctx.turn}`,
        type: 'game.message',
        timestamp: Date.now(),
        entities: {},
        data: { messageId },
        narrate: true,
      }];
    },

    // Save/restore the internal counter so it survives a save/
    // load
    getRunnerState() { return { announcementCount }; },
    restoreRunnerState(state) { announcementCount =
      (state.announcementCount as number) ?? 0; },
  };
}
```


The daemon keeps its own state (`announcementCount`) in the closure, and exposes `getRunnerState` / `restoreRunnerState` so that state survives a save and reload. Daemon events here use `type:` `'game.message'` with a `messageId` and `narrate: true`, which is the right form for scheduler output, which the engine narrates as ambient text. (Contrast this with the chain handlers in *Event Handlers*, which must avoid `game.message` because there it would override the action's own text. Different context, different rule.)

The mistake everyone makes once: writing a daemon with no `condition`. A daemon without one runs *every single turn*, which is rarely what you want for something like an announcement. Use a `condition` to control timing: a turn modulus, a world-state flag, whatever fits.

22.3 Conditional daemons: react to state

A daemon's `condition` can read world state, not just the turn count. The goat bleating daemon only runs while feeding time is active and there are bleats left, counting itself down and stopping:

```

function createGoatBleatingDaemon(): Daemon {
  return {
    id: 'zoo.daemon.goat_bleating',
    name: 'Goat Bleating',
    priority: 3,

    condition: (ctx: SchedulerContext): boolean => {
      const active =
        ctx.world.getStateValue('zoo.feeding_time_active') as
        boolean;
      const left =
        ctx.world.getStateValue('zoo.bleat_turns_remaining') as
        number;
      return active === true && (left ?? 0) > 0;
    },

    run: (ctx: SchedulerContext): ISemanticEvent[] => {
      const left =
        (ctx.world.getStateValue('zoo.bleat_turns_remaining') as
        number) ?? 0;
      if (left <= 1) {
        ctx.world.setStateValue('zoo.feeding_time_active', false);
        ctx.world.setStateValue('zoo.bleat_turns_remaining', 0);
      } else {
        ctx.world.setStateValue('zoo.bleat_turns_remaining', left
        - 1);
      }

      // Ambient sound, only heard if the player is in the petting
      zoo

      const room = ctx.world.getEntity(ctx.playerLocation);
      if ((room?.get(IdentityTrait)?.name ||
        '').includes('Petting Zoo')) {
        return [{
          id: `zoo-bleat-${ctx.turn}`, type: 'game.message',
          timestamp: Date.now(), entities: {},
          data: { messageId: TimedMessages.GOATS_BLEATING },
          narrate: true,
        }];
      }
      return [];
    },
  },
};
}

```

Returning an empty array is how a daemon stays silent on a given turn while still having run. Here, the goats only bleat *aloud* if the player is around to hear them.

22.4 Fuses: count down and fire

A `Fuse` ticks down `turns` each turn and calls `trigger` when it hits zero. Set `repeat: true` with an `originalTurns` value and it re-arms after firing. The feeding-time fuse first fires at turn 10, then every 8 turns after that, and each time it primes the bleating daemon:

```
function createFeedingTimeFuse(): Fuse {
  return {
    id: 'zoo.fuse.feeding_time',
    name: 'Feeding Time',
    turns: 10,           // first feeding time at turn 10
    repeat: true,        // keep re-arming
    originalTurns: 8,    // subsequent feedings every 8 turns
    priority: 10,

    trigger: (ctx: SchedulerContext): ISemanticEvent[] => {
      ctx.world.setStateValue('zoo.feeding_time_active', true);
      ctx.world.setStateValue('zoo.bleat_turns_remaining', 3);
      return [{
        id: `zoo-feeding-${ctx.turn}`, type: 'game.message',
        timestamp: Date.now(), entities: {},
        data: { messageId: TimedMessages.FEEDING_TIME }, narrate:
          true,
      }];
    },
  };
}
```

This is a clean two-part collaboration: the fuse marks the schedule and sets the state; the conditional daemon does the per-turn reaction until the state clears.

The mistake everyone makes once: expecting a fuse with `turns: 10` to fire exactly ten turns after you set it. A newly set

fuse skips its first tick, so it fires about *eleven* ticks after registration. If precise timing matters, count from the skip, or test it and adjust `turns`.

22.5 Giving the announcements their words

Every `TimedMessages` id the daemons emit needs text in `extendLanguage`, or the PA just narrates raw ids:

```
extendLanguage(language: LanguageProvider): void {
    language.addMessage(TimedMessages.PA_CLOSING_3,
        '*DING DONG* "Attention visitors! The zoo closes in three
            hours. Please ' +
            'visit all exhibits before closing time!";
    language.addMessage(TimedMessages.PA_CLOSING_2,
        '*DING DONG* "Two hours until closing. Don\'t forget the gift
            shop!";
    language.addMessage(TimedMessages.PA_CLOSING_1,
        '*DING DONG* "One hour until closing. Please make your way
            toward the exit.";
    language.addMessage(TimedMessages.PA_CLOSED,
        '*DING DONG* "The zoo is now closed. Thank you for
            visiting!";
    language.addMessage(TimedMessages.FEEDING_TIME,
        '*DING DONG* "It\'s FEEDING TIME at the Petting Zoo! Come
            watch the goats ' +
            'and rabbits enjoy their snacks!";
    language.addMessage(TimedMessages.GOATS_BLEATING,
        'The pygmy goats are bleating loudly and headbutting the
            fence. They seem ' +
            'very hungry!');
}
```

(If your story already has an `extendLanguage` from earlier chapters, add these lines to it; a story has just one.)

22.6 Try it

```
> wait                (repeat ~5 times; the first PA announcement crackles)
> wait                ...closing announcements count down
> south; east         Petting Zoo
> wait                (repeat until "FEEDING TIME" is announced)
> wait                The goats start bleating
> take feed           Grab the feed
> feed goats          Feed them, but the bleating runs on its own timer
> wait                ...a turn or two later the bleating stops on its own
```

The bleating ends when the daemon's three-turn countdown reaches zero, *not* because you fed the goats. The feeding action (Chapter 14) records that the goats were fed; it never touches `zoo.feeding_time_active`, which is the only state the daemon watches. If you *wanted* feeding to silence them early, you'd add an event handler on the feed action that clears that flag. That's a nice exercise, but the scheduler's own countdown is doing the stopping here, exactly as the conditional daemon above ("counting itself down and stopping") was built to do.

22.7 Key takeaway

The `SchedulerPlugin` gives the world a clock once you register it in `onEngineReady()`. **Daemons** run every turn; gate them with a `condition`, and expose `getRunnerState` / `restoreRunnerState` so their closure state survives a save. **Fuses** count down and fire once (re-arming with `repeat`), but skip their first tick. Both narrate through `game.message` events, and both can cooperate through world state: a fuse can set the stage for a daemon to play out.

23 Scoring & Endgame: Winning the Game

A game needs a way to keep score and a way to end. The zoo gives points for seeing the sights and doing the activities (visiting each exhibit, feeding the animals, pressing a souvenir penny, reading the brochure), and when the player has done it all, the game declares victory. This final mechanic chapter ties together much of what came before: a **score ledger** that records achievements, event handlers that award points as the player plays, and a **victory daemon** that watches for the win and triggers the ending.

23.1 The score ledger

Sharpee tracks score with `world.awardScore()`. Each award has a unique ID, and that ID is what makes scoring safe: awarding the same ID twice does nothing the second time. You never have to worry about a player re-entering a room and scoring for it again.

```
// Set the maximum in initializeWorld() so "score" can show "X out
  of Y"
world.setMaxScore(75);

// Award points (idempotent): the same ID never scores twice
const awarded = world.awardScore(
  'zoo.visit.petting_zoo', // unique ID
  5,                       // points
  'Visited the petting zoo' // description (for debugging /
                           transcripts)
);
// awarded === true the first time, false on every call after

// Read the score back
const current = world.getScore();
const max = world.getMaxScore();
```

The mistake everyone makes once: reusing a score ID for two different achievements. Because awarding is idempotent on the ID, the second achievement silently never scores; the ledger thinks it's already been counted. Give every achievement its own descriptive, unique ID.

23.2 Defining the scoring table

It pays to declare all the IDs and point values up front, as constants, rather than scattering string literals through the code:

```

const MAX_SCORE = 75;

const ScoreIds = {
  VISIT_PETTING_ZOO: 'zoo.visit.petting_zoo',
  VISIT_AVIARY: 'zoo.visit.aviary',
  VISIT_GIFT_SHOP: 'zoo.visit.gift_shop',
  VISIT_SUPPLY_ROOM: 'zoo.visit.supply_room',
  VISIT_NOCTURNAL: 'zoo.visit.nocturnal',
  FEED_GOATS: 'zoo.action.fed_goats',
  FEED_RABBITS: 'zoo.action.fed_rabbits',
  COLLECT_MAP: 'zoo.collect.map',
  COLLECT_PRESSED_PENNY: 'zoo.collect.pressed_penny',
  PHOTOGRAPH_ANIMAL: 'zoo.action.photographed',
  PET_ANIMAL: 'zoo.action.petted',
  READ_BROCHURE: 'zoo.action.read_brochure',
} as const;

const ScorePoints: Record<string, number> = {
  [ScoreIds.VISIT_PETTING_ZOO]: 5,
  [ScoreIds.VISIT_AVIARY]: 5,
  [ScoreIds.VISIT_GIFT_SHOP]: 5,
  [ScoreIds.VISIT_SUPPLY_ROOM]: 5,
  [ScoreIds.VISIT_NOCTURNAL]: 5,
  [ScoreIds.FEED_GOATS]: 10,
  [ScoreIds.FEED_RABBITS]: 10,
  [ScoreIds.COLLECT_MAP]: 5,
  [ScoreIds.COLLECT_PRESSED_PENNY]: 10,
  [ScoreIds.PHOTOGRAPH_ANIMAL]: 5,
  [ScoreIds.PET_ANIMAL]: 5,
  [ScoreIds.READ_BROCHURE]: 5,
};

```

Achievement	Points
Visit each of the five exhibits	5 each (25)
Feed the goats / the rabbits	10 each (20)
Collect the map / press a penny	5 / 10
Photograph something	5
Pet an animal	5
Read the brochure	5

Achievement	Points
Total	75

Two more tables finish the setup: one maps room names to their visit-score id (the room-visit handler reads it), and one holds the endgame message id.

```
const ROOM_SCORE_MAP: Record<string, string> = {
  'Petting Zoo': ScoreIds.VISIT_PETTING_ZOO,
  'Aviary': ScoreIds.VISIT_AVIARY,
  'Gift Shop': ScoreIds.VISIT_GIFT_SHOP,
  'Supply Room': ScoreIds.VISIT_SUPPLY_ROOM,
  'Nocturnal Animals Exhibit': ScoreIds.VISIT_NOCTURNAL,
};

const ScoreMessages = {
  VICTORY: 'zoo.victory',
} as const;
```

The `chainEvent` handlers below use the same `ISemanticEvent / IWorldModel` types and the `this.entityIds` field you established in *Event Handlers* (Chapter 13).

23.3 Awarding points as the player plays

Scoring hooks into the action layers you already know. Some awards live inside a custom action or capability behavior; petting an animal awards its points right in the behavior's `execute()`:

```
execute(_entity, world, _actorId, _shared): void {
  world.awardScore(ScoreIds.PET_ANIMAL,
    ScorePoints[ScoreIds.PET_ANIMAL], 'Petted an animal');
},
```

Others ride on standard-action events through `chainEvent`. Room visits award points on `if.event.actor_moved`, looking the destination up in a

room-to-score-ID map; because `awardScore` is idempotent, re-entering a scored room is harmless:

```
world.chainEvent('if.event.actor_moved', (event, w) => {
  const data = event.data as Record<string, any>;
  const toRoom = data.toRoom || data.destination;
  if (!toRoom) return null;

  const roomName = w.getEntity(toRoom)?.get(IdentityTrait)?.name
    || '';
  const scoreId = ROOM_SCORE_MAP[roomName];
  if (!scoreId) return null;

  w.awardScore(scoreId, ScorePoints[scoreId], `Visited ${roomName}`);
  return null; // scoring is silent; no custom event
}, { key: 'zoo.chain.room-visit-scoring' });
```

The same shape covers collecting the map and reading the brochure. Each matches the event against the entity id recorded in `this.entityIds` (Chapter 13), awards the points, and returns `null` to stay silent:

```

const mapId = this.entityIds.zooMap;
world.chainEvent('if.event.taken', (event: ISemanticEvent, w:
    IWorldModel) => {
    const data = event.data as Record<string, any>;
    if (data.itemId === mapId) {
        w.awardScore(ScoreIds.COLLECT_MAP,
            ScorePoints[ScoreIds.COLLECT_MAP],
            'Collected the zoo map');
    }
    return null;
}, { key: 'zoo.chain.take-scoring' });

const brochureId = this.entityIds.brochure;
world.chainEvent('if.event.read', (event: ISemanticEvent, w:
    IWorldModel) => {
    const data = event.data as Record<string, any>;
    if (data.entityId === brochureId || data.targetId ===
        brochureId) {
        w.awardScore(ScoreIds.READ_BROCHURE,
            ScorePoints[ScoreIds.READ_BROCHURE],
            'Read the zoo brochure');
    }
    return null;
}, { key: 'zoo.chain.read-scoring' });

```

(`this.entityIds.zooMap` and `.brochure` are recorded in `initializeWorld` when you create those items, the same way Chapter 13 stored the feed and penny ids.)

That covers eight of the twelve awards (40 of the 75 points). The remaining four ride the very same two patterns, so wire them up the same way. Feeding the goats or rabbits and photographing an animal award inside their custom actions' `execute()` (like petting, above); pressing a souvenir penny awards in the penny-press chain from Chapter 13 (like collecting the map):

```

// inside the feeding action's execute(), keyed on which animal
// was fed:
world.awardScore(ScoreIds.FEED_GOATS,
    ScorePoints[ScoreIds.FEED_GOATS], 'Fed the goats');
// ...and ScoreIds.FEED_RABBITS the same way when the rabbits are
// fed.

// inside the photograph action's execute():
world.awardScore(ScoreIds.PHOTOGRAPH_ANIMAL,
    ScorePoints[ScoreIds.PHOTOGRAPH_ANIMAL], 'Photographed an
    animal');

// in the penny-press chain (Chapter 13), the same shape as the
// map award:
w.awardScore(ScoreIds.COLLECT_PRESSED_PENNY,
    ScorePoints[ScoreIds.COLLECT_PRESSED_PENNY],
    'Pressed a souvenir penny');

```

With all twelve awards in place the scores sum to the full 75, so the victory daemon below has a target it can actually reach. Leave any of these four out and the game caps at 40 (or wherever you stopped) and the win never fires, a useful reminder that the max score and the awarding code have to agree.

23.4 The victory daemon

The win condition is checked by a daemon, exactly the scheduler pattern from the last chapter. It watches the score each turn and, when the maximum is reached, marks the game over and emits the victory message. Its `priority: 100` makes it run *last*, after every other daemon and all the turn's scoring has settled:

```

function createVictoryDaemon(): Daemon {
  let victoryTriggered = false;

  return {
    id: 'zoo.daemon.victory',
    name: 'Victory Check',
    priority: 100, // run last, after all scoring for the turn

    condition: (ctx: SchedulerContext): boolean => {
      if (victoryTriggered) return false;
      return ctx.world.getScore() >= MAX_SCORE;
    },

    run: (ctx: SchedulerContext): ISemanticEvent[] => {
      victoryTriggered = true;
      ctx.world.setStateValue('game.victory', true);
      ctx.world.setStateValue('game.ended', true);
      return [{
        id: `zoo-victory-${ctx.turn}`, type: 'game.message',
        timestamp: Date.now(), entities: {},
        data: { messageId: ScoreMessages.VICTORY }, narrate: true,
      }];
    },

    getRunnerState() { return { victoryTriggered }; },
    restoreRunnerState(state) { victoryTriggered =
      (state.victoryTriggered as boolean) ?? false; },
  };
}

```

Register it like any daemon, alongside the others:

```

scheduler.registerDaemon(createVictoryDaemon());

```

And give `ScoreMessages.VICTORY` its text in `extendLanguage`, or the win narrates a raw id:

```
language.addMessage(ScoreMessages.VICTORY,
    "Congratulations! You've earned your JUNIOR ZOOKEEPER
    badge. You visited " +
    'every exhibit, fed the animals, collected souvenirs, and made
    memories to ' +
    'last a lifetime.\n\n*** You have won ***');
```

The mistake everyone makes once: giving the victory check a low priority, so it runs *before* the turn's scoring daemons and event handlers. It then sees a stale score, misses the moment the player crosses the line, and only fires a turn late. Keep the victory daemon at high priority so it evaluates after everything else has had its say.

23.5 Try it

```
> score           Check the score: 0 out of 75
> south; east     Visit the petting zoo, +5
> score           Now 5 out of 75
> take feed
> feed goats      +10
> ... visit every exhibit, feed, pet, photograph, press a penny, read ...
> score           75 out of 75, victory!
```

23.6 Key takeaway

`world.awardScore(id, ...)` records an achievement, and the unique `id` makes it idempotent, so the same award never counts twice. Hang awards wherever the achievement actually happens, whether in custom actions, capability behaviors, or standard-action events via `chainEvent`, and let a high-priority **victory daemon** watch `getScore()` and trigger the ending when the maximum is reached. With scoring and an endgame in place, the zoo is a complete, winnable game.

VOLUME VII — PRESENTATION

24 Channels: The Universal UI Surface

You’ve built a complete game, but the book has quietly treated “what the player sees” as one thing: prose. A running story shows far more: where you are, your score, the turn count, a command prompt, and, in a rich client, images and sound. How does all of that travel from the engine to the screen, the same way whether the game runs in a terminal, a browser, or a multi-user server? Through **channels**, the foundation this whole volume builds on.

24.1 One surface for everything

A **channel** is a named stream of signals from the story to the UI. The key idea is that *everything* the player perceives travels over a channel, not just the prose. The narrative is the `main` channel; the location and score are the `location` and `score` channels; the command prompt is the `prompt` channel; images and sound are media channels. There is no special path for prose and a separate one for the status bar. One mechanism carries it all, which is why channels are called the universal UI surface.

24.2 A turn produces a packet

Each turn, the engine asks every channel “what do you have this turn?” and assembles the answers into a **turn packet**: the set of channels that emitted, each with its value. On the other side, the client hands each channel’s payload to a matching **renderer** that updates the corresponding piece of UI: the prose window, the status line, the score display.

Because the packet is just data, the same turn packets can drive an in-process browser client *and* a multi-user server with one renderer per

connected player. The engine produces signals; how (and where) they're drawn is entirely the client's business.

24.3 Channel modes

Every channel has a **mode** that tells the renderer how its value behaves from turn to turn:

Mode	Meaning	Examples
replace	A single current value that supersedes the last	location, score, prompt
append	New entries each turn that accumulate	main (the prose log)
event	A one-off, transient signal	death, endgame, a sound

The mode is what lets the score display *overwrite* while the prose window *grows* and a death notice *fires once*. It's the first thing you decide when defining a channel.

24.4 The channels you get for free

The standard library registers a full set of channels, fed by the same world and events the rest of your story already produces. You wire none of them:

Channel	Mode	Carries
main	append	the prose-pipeline text blocks
location	replace	the player's current room name
score	replace	the score (from the ledger in Volume VI)

Channel	Mode	Carries
turn	replace	the turn count
prompt	replace	the command prompt
info, ifid	replace	story metadata
death, endgame, score_notify	event	endgame and scoring signals

The `score` channel reads the ledger you set up in *Scoring & Endgame*; the `location` channel reads the player’s room; the `main` channel carries the blocks the prose pipeline rendered. Channels are the seam where everything you’ve already built becomes something a client can show.

24.5 Capability negotiation

Not every client can display everything. At startup the client declares its **capabilities**, and the engine replies with a **manifest** listing the channels available to *that* client; a text-only terminal simply never sees the media channels. After the manifest, the per-turn packets flow. As an author you rarely touch this. It’s the machinery that lets one story serve a bare terminal and a graphical browser from exactly the same code.

24.6 Defining your own channel

When your story has a UI signal the standard channels don’t cover, such as an ambient mood line, a custom HUD value, or a trigger for a story-specific overlay, you define your own **IOChannel** in the `registerChannels` hook. A channel is an object with an `id`, a `contentType`, a `mode`, an `emit` policy, and a `produce` closure:

```

// A mood line per room; rooms not listed clear the line.
const AMBIENCE_BY_ROOM: Record<string, string> = {
  'Aviary': 'The air is alive with birdsong and the rustle of
    wings.',
  'Nocturnal Animals Exhibit': 'Your eyes strain against the warm
    red dark.',
};

registerChannels(registry: IChannelRegistry): void {
  registry.add({
    id: 'zoo.ambience',
    contentType: 'text',
    mode: 'replace',
    emit: 'sparse',           // only re-emit when the value
                             changes
    produce: (ctx) => {
      const world = ctx.world as WorldModel;
      const room =
        world.getEntity(world.getLocation(world.getPlayer()!.id)!);
      // a mood line for the current room, or '' to clear the line
      return room ? AMBIENCE_BY_ROOM[room.name] ?? '' : '';
    },
  });
}

```

`produce` receives a context with the turn's `world`, `events`, `blocks`, `turn` number, and the channel's `prevValue`. Return a value to emit it, or `undefined` to stay silent. The `emit` policy decides idle turns: `sparse` emits only when the value changes; `always` emits every turn. To *override* a standard channel, register one with the same `id`. Last write wins.

One subtlety to internalize, because it bites everyone once: on a `sparse` `replace` channel, `undefined` means “no change this turn,” **not** “clear the line.” The channel doesn’t re-emit, so whatever it last showed stays on screen. If you returned `undefined` for “rooms without a mood,” the previous room’s line would follow the player around. To actually blank the line you must emit a *different* value, here the empty string `''`, which is a real transition the renderer paints as blank (and `sparse` then stays quiet until the mood changes again). Reach for `undefined` only when you genuinely want the current value to persist untouched.

Crucially, a channel emits **data** (text, a number, JSON), never UI. The value says *what*; the renderer (next chapter) decides *how* it looks. That data-only wire is what keeps presentation in the client's hands, where an author can restyle or replace it per story.

Family Zoo v18 ships exactly this `zoo.ambience` channel, a one-line mood description for each area, and its browser entry registers a renderer that creates a dedicated page element and paints the line into it. The chapters ahead build on that concrete example.

24.7 Key takeaway

Channels are the universal UI surface: every story-to-player signal (prose, status, prompt, media) travels as a named channel, and each turn the engine emits a packet of the ones that changed for the client to render. A channel's **mode** (`replace` / `append` / `event`) tells the renderer how its value behaves; the standard channels come free, and you add your own `IOChannel` in `registerChannels` , returning data, never UI. Because the wire is data-only, one story drives a terminal, a browser, or a multi-user server unchanged. That portability is the subject of the chapters ahead.

25 The Web Client: A Framework-Free UI

The last chapter ended with a promise: a channel emits *data*, and something on the other side decides how it looks. That something is the **client**. This chapter walks through Sharpee’s reference browser client, including: how it connects to the engine, turns turn packets into a living page, and why it’s built from plain HTML and CSS instead of a framework.

25.1 What the client is responsible for

The engine produces signals; it knows nothing about screens. The browser client, `@sharpee/platform-browser`, is the piece that owns the page: it holds the DOM, runs the input box, draws the menus and save dialogs, and, most importantly, receives each turn packet and paints it. Everything visible is the client’s job. The engine never reaches for an element; it only emits channels.

The orchestrator is `BrowserClient`. A story’s browser entry point creates one, hands it the page’s elements, connects the engine, and starts:

```

const client = new BrowserClient({
  storagePrefix: 'familyzoo-',
  defaultTheme: 'zoo-sunny',           // the theme applied on
                                     first load / restore
  // The clickable theme menu is generated at build time from your
  // package.json
  // `sharpee.themes` (Chapter 26); this array is metadata the
  // generator fills in.
  themes: [
    { id: 'zoo-sunny', name: 'Zoo Sunny' },
    { id: 'modern-dark', name: 'Modern Dark' },
    { id: 'paper', name: 'Paper' },
  ],
  storyInfo: {
    title: 'Family Zoo',
    authors: 'You',
    version: '1.0.0',
    engineVersion: '', // filled by `sharpee build`; '' is a
                      valid placeholder
    buildDate: '',
  },
});

client.initialize(elements);           // page elements (after
                                     DOMContentLoaded)
client.connectEngine(engine, world);  // wire the engine
await client.start();                 // boot, restore autosave,
                                     first look

```

You rarely write this by hand. `sharpee build --browser` generates the entry point and the host page for you. But knowing the three calls demystifies what the bundle is doing: `initialize` learns the DOM, `connectEngine` subscribes to the engine, and `start` runs the opening turn.

25.2 How a turn reaches the screen

Inside `connectEngine`, the client builds a **renderer** (the consumer-side host from the previous chapter) and subscribes to exactly two engine signals:

```
engine.on('channel:manifest', (cmgt) => renderer.applyCmgt(cmgt));
engine.on('channel:packet', (packet) =>
  renderer.applyTurnPacket(packet));
```

That's the whole rendering path. At startup the engine emits one **manifest** (the capability-filtered list of channels this client gets); thereafter it emits one **packet per turn**. The renderer dispatches each channel in the packet to the `ChannelRenderer` registered for it: the `main` channel's renderer appends prose, the `location` renderer rewrites the status line, the `score` renderer updates the score. There is no second path. Prose and status and media all arrive the same way.

The client registers a full set of platform-default renderers in one call, `registerDefaultBrowserRenderers`, which covers `main`, `prompt`, `location`, `score`, `turn`, the notification channels, and every media channel. Those defaults are what give you a working page with zero rendering code.

25.3 Commands flow back the same way

Rendering is only half a loop; the player has to type. The input box feeds commands to `engine.executeTurn(command)`, and the engine runs a turn, which produces the next packet, which the renderer paints. UI *gestures* close the same loop: when a clickable hotspot or a menu item fires, it synthesizes the equivalent typed command and runs it through `executeTurn`, so a click and a typed verb are indistinguishable to the engine. The menu's **Help** and **About** entries, for instance, are wired straight to `engine.executeTurn('help')` and `engine.executeTurn('about')`.

25.4 Why no framework

Open the reference client and you will not find React, Vue, or a web-component library. The UI is plain HTML elements styled by CSS classes. Dialogs are native `<dialog>` elements opened with `showModal()`; the

menu bar is a `<nav>` with `.sharpee-menu-bar-item` rows; the prose window is a scrolling `<div>`. State that *would* be component props lives instead in `--modifier` classes and standard ARIA attributes: an open menu carries `--open` and `aria-expanded`, a checked theme carries `--checked`.

The framework-free build is deliberate: a framework would put a runtime between the author and the page and impose its own idioms for overriding a view. Sharpee's bet is the opposite: the page is just HTML and CSS, so an author restyles it with CSS and replaces a renderer with a function. There is no framework API to learn.

25.5 Overriding a renderer

Because each channel maps to one registered renderer, customizing the UI is *re-registering*. After the platform defaults are in place, available from `connectEngine` onward, a story grabs the renderer and registers its own. There are two cases, and they differ in one way: whether the channel already has a place on the page.

Replacing an existing channel's renderer. A standard channel like `score` already renders into a platform element, the status line. To change *how* it looks, re-register against the same id and write into that same element. Registration is last-write-wins, so your renderer simply replaces the platform's for that channel, without touching any other:

```
const renderer = client.getChannelRenderer();
renderer.registerRenderer('score', {
  onValue: (value) => {
    const { current } = value as { current: number };
    const el = document.getElementById('score-turns'); // the
    platform status element
    if (el) el.textContent = `★ ${current}`;
  },
});
```


You're not adding anything to the page. The score element is already there; you're only changing what gets written into it.

Rendering a channel you invented. A channel you created in `registerChannels` (the `zoo.ambience` channel from the last chapter) has no place on the page yet. The platform doesn't know it exists, so left alone its value falls to the renderer's JSON-tree fallback. Its renderer therefore makes its own element the first time it runs and reuses it after. This is exactly how the platform's built-in renderers work: they create DOM nodes and append them into the page's containers. Create once, reuse thereafter:

```
renderer.registerRenderer('zoo.ambience', {
  onValue: (value) => {
    const main = document.getElementById('main-window'); // a
    // stable platform container
    if (!main) return;
    let line = document.getElementById('zoo-ambience');
    if (!line) {
      line = document.createElement('div');
      line.id = 'zoo-ambience';
      main.prepend(line); // a mood line above the prose
    }
    line.textContent = String(value ?? '');
  },
});
```

The renderer owns the element, so nothing needs to be added to the host page and it survives every rebuild. Style it from your override stylesheet (Chapter 26) by its id or a class you give it.

25.6 Save, restore, and theme: for free

The client ships the surrounding chrome too. Saving routes the engine's complete `ISaveData` into a browser envelope persisted in `localStorage`; an `autosave` piggy-backs on the per-turn packet, so every turn boundary is captured without any story code. Restore unwraps the envelope and hands the save back to the engine, which rebuilds the

world. Theme switching is one attribute flip on the document, the subject of the next chapter. All of it is configured through `BrowserClientConfig`; none of it is something you implement.

25.7 Key takeaway

The web client owns the page; the engine only emits channels.

`BrowserClient` wires them with three calls and drives the screen from just two signals (one manifest, then one packet per turn) dispatched to per-channel renderers, with commands flowing back through `engine.executeTurn`. Because the UI is framework-free, you customize it the web-native way: restyle with CSS, or replace a view by re-registering a `ChannelRenderer`; save, restore, and theming come built in. With data flowing to a rendered page, the next two chapters turn to *how it looks*: decoration and theming, then media and audio.

26 Decoration & Theming: Style Without HTML on the Wire

Channels carry data, and the client paints it, but the data still has to *say* something about emphasis, and the page still has to *look* like something. This chapter covers the three pieces that turn bare text into a styled screen: **decoration** (how prose carries styling without carrying HTML), **theming** (how the whole page changes skin), and the **status line** (how location, score, and turn become a bar at the top).

26.1 Decoration: styling without HTML on the wire

Volume V's formatter chain handled *grammar*: articles and capitalization inside a message template. Decoration handles *style*: making a word italic, marking a name, flagging a command. The rule Sharpee commits to is that **no HTML travels on the wire**. A message never contains `<em>` or an inline `style`. Instead, templates use a bracket syntax:

```
This is Flood Control Dam #3 in the center of [em:Zork].
```

`[name:content]` wraps its content in a decoration. The prose pipeline turns each bracket into a structured decoration node, so `[em:Zork]` becomes `{ className: 'sharpee-em', content: ['Zork'] }`, and the browser renderer turns *that* into `<span class="sharpee-em">Zork</span>`. Brackets nest: `[em:[strong:bold italic]]` produces a `sharpee-strong` span inside a `sharpee-em` span.

The point of the indirection is the same as HTML-plus-CSS: the markup says *what kind* of thing this is; the stylesheet says how it looks. A span with a class, never a baked-in color.

26.2 Platform vocabulary and the sharpee-namespace

The platform reserves the `sharpee-` prefix and ships a small vocabulary, each name mapping to one CSS class with a default rule:

Decoration Class		Renders as
<code>em</code>	<code>.sharpee-em</code>	italic
<code>strong</code>	<code>.sharpee-strong</code>	bold
<code>code</code>	<code>.sharpee-code</code>	monospace
<code>item</code>	<code>.sharpee-item</code>	a referenced object
<code>npc</code>	<code>.sharpee-npc</code>	a character
<code>room</code>	<code>.sharpee-room</code>	a location
<code>command</code>	<code>.sharpee-command</code>	a verb the player can type

Write `[em:...]` and you inherit the platform's `.sharpee-em` rule. The resolver only prefixes names it recognizes. Write a name the platform *doesn't* know, such as `[thief-taunt:Hold still!]`, and it emits `<span class="thief-taunt">` verbatim, with no prefix, and your story's CSS owns the styling completely. That's the one firm rule for authors: don't name your own classes with `sharpee-`; that namespace is the platform's.

26.3 Theming: one DOM, many skins

Decoration styles the prose; **theming** styles the whole page. Sharpee's theming model (ADR-170) rests on two commitments. First, a stable **component vocabulary**: a fixed set of class names the DOM always uses, regardless of theme:

Component	Class
Window shell	<code>.sharpee-window</code>
Menu bar	<code>.sharpee-menu-bar</code>
Status bar	<code>.sharpee-status-bar</code>

Component Class

Prose pane	.sharpee-prose-pane
Input bar	.sharpee-input-bar
Dialog	.sharpee-dialog

Second, and this is the part ADR-188 settled, the platform ships a **theme engine**, and a theme is *data*. The engine (in `@sharpee/platform-browser`) is one un-scoped layer of component rules that reads a set of `--theme-*` custom properties and paints every component class once:

```
/* the engine, shipped by the platform, written once, never per theme */  
body { background: var(--theme-bg); color: var(--  
  theme-text); }  
.sharpee-status-bar { background: var(--theme-accent); color:  
  var(--theme-accent-text); }  
/* ...every .sharpee-* component, all consuming var(--theme-*) ... */
```

The engine also ships the default token values on `:root`, the classic white-on-blue palette, so the page is **always** skinned, even with no theme selected. A **theme**, then, is nothing but a block that overrides those tokens:

```
[data-theme="modern-dark"] {  
  --theme-bg: #1e1e2e;  
  --theme-text: #cdd6f4;  
  --theme-accent: #89b4fa;  
  --theme-font: "Inter", system-ui, sans-serif;  
}
```

The `--theme-*` set is the published contract. Sixteen properties the engine knows how to consume: `--theme-bg`, `--theme-bg-alt`, `--theme-text`, `--theme-text-muted`, `--theme-accent`, `--theme-accent-text`, `--theme-border`, `--theme-input-bg`, `--theme-menu-bg`, `--theme-menu-hover`, `--theme-desktop-bg`, `--theme-font`, `--theme-font-body`, `--theme-font-chrome`, `--theme-font-size`, `--theme-line-height`. A theme sets the ones it cares about; the rest fall back to the `:root` default.

So the contract between platform and author is now the **tokens**, not a pile of per-theme component rules. The variables aren't a convenience *inside* a theme, they *are* the theme. Switching is still one attribute flip, from `data-theme="modern-dark"` to `data-theme="zoo-sunny"`, and the whole page re-skins, because the engine re-reads the tokens. (Theme CSS loads *after* the engine: `:root` and `[data-theme="x"]` have equal specificity, so source order decides, and the theme must win.) The fallback is automatic and needs no code: an unset token, an unknown `data-theme`, or a selected theme that isn't wired into the build all resolve to the `:root` default, so the page is never unskinned. The `ThemeManager` handles the flip and remembers the choice in `localStorage`.

26.4 Applying themes

You wire themes into a story by listing them in `package.json` → `sharpee.themes`. The build reads that list, copies in any theme CSS, links it after the engine, and builds the theme menu: the `classic` default plus one entry per theme you list. Discovery is never magic: the build wires exactly what you name and does **not** scan `node_modules`.

26.4.1 Built-in themes: list the id

The platform ships a set of built-in themes with `@sharpee/platform-browser`: `modern-dark`, `retro-terminal`, `paper`, and `system-6` (plus `classic`, the white-on-blue default, which is always present). To offer some of them, just list their **ids**, with no installs and no extra dependencies:

```
// the story's package.json
"sharpee": { "themes": ["modern-dark", "paper", "system-6"] }
```

`sharpee build-browser` copies each listed built-in's CSS out of `platform-browser` into `dist/web/themes/<id>.css`, links it, and adds it to the menu.

26.4.2 Your own theme: three lines of CSS and one list entry

Because a theme is just a token block, shipping your own takes two small pieces:

1. Write a `[data-theme]` token block in your author override stylesheet, `browser/<story-id>.css` (the build links it *last*, so it wins the cascade):

```
/* browser/familyzoo.css */
[data-theme="zoo-sunny"] {
  --theme-bg: #ffffaf0;           /* warm cream */
  --theme-text: #2f2a24;
  --theme-accent: #4a9d52;       /* zoo green */
  --theme-font: "Nunito", "Segoe UI", system-ui, sans-serif;
}
```

2. List it inline in `sharpee.themes`, giving it an `id` (matching the `[data-theme]` value) and a menu `name`:

```
"sharpee": { "themes": [
  "modern-dark", "paper",
  { "id": "zoo-sunny", "name": "Zoo Sunny" }
] }
```

That's it. The build adds *Zoo Sunny* to the menu; selecting it flips `data-theme="zoo-sunny"`, and the engine paints the window, menu, status bar, prose pane, dialogs, and input from your four variables (and the `:root` defaults for the rest).

If you *want* to push past the tokens (Family Zoo, for instance, deliberately puts its green on the title and menu bars instead of the engine's default, the status bar), add a few **flourish** rules under the same `[data-theme]` selector in that same stylesheet:

```
[data-theme="zoo-sunny"] .sharpie-menu-bar { background: var(--  
  theme-menu-bg); }  
[data-theme="zoo-sunny"] .sharpie-status-bar { background: var(--  
  theme-bg-alt); }
```

Flourishes are optional polish; the token block is what makes it a theme. (The author override stylesheet is also where anything that *isn't* a theme lives: a one-off tweak to a single component, an extra class your prose uses.)

Sharing a theme across stories. A theme that lives only in one story's override stylesheet can't be reused elsewhere. The same `[data-theme]` block can instead be published as a small npm package other authors install and list by name, the path the built-ins themselves use internally. That's an advanced step; for one story, the override stylesheet is all you need.

26.5 The status line

The bar across the top, *Toucan Enclosure · Score: 12 · Turns: 47*, is not a special widget. It's three channels rendered into the status bar: `location`, `score`, and `turn`, all `replace-mode` (each shows a single current value that supersedes the last). Each turn, the engine reads the player's room, the scoring ledger from Volume VI, and the turn count, and emits them; the client's status renderers write them into `.sharpie-status-bar`.

Because they're ordinary channels, the status line is as customizable as anything else. Don't want a turn counter? Restyle or hide `.sharpie-status-bar` in your theme. Want the score as a star badge instead of a number? Re-register the `score` renderer, exactly as the previous chapter showed. Nothing about the status line is privileged. It's the universal channel mechanism pointed at a strip of the page.

26.6 Key takeaway

Style reaches the screen without ever putting HTML on the wire.

Decoration marks prose with `[name:content]` brackets that become `sharpee-`-prefixed spans; markup says *what*, CSS says *how*. **Theming** paints a stable component vocabulary from sixteen `--theme-*` tokens, so a theme is just a `[data-theme]` block of variables, selected by a single flip. Offer a built-in by id in `sharpee.themes`, or ship your own in your override stylesheet. Even the status line is just the `location / score / turn` channels rendered into a bar you can restyle. Text and chrome covered, the final chapter adds the senses Sharpee has saved for last: images and sound.

27 Media & Audio: Sight and Sound as Channels

Every chapter so far has reached the player through words and the chrome around them. This last chapter of the volume adds the other senses: a picture behind the prose, a sound when a door slams, music that swells at the climax. They arrive the same way everything else does, over channels and capability-gated, so a text-only client never even hears about them.

27.1 Media is just more channels

There is no separate media engine. Images and audio are channels like `main` and `score`, registered among the platform defaults and rendered by the browser client. The standard media channels are:

Channel	Carries
<code>image:background</code>	a full-bleed image behind the prose
<code>image:main</code>	a primary illustration in the content flow
<code>image:overlay</code>	an image layered above the scene
<code>image:preload</code>	assets to fetch ahead of time
<code>sound</code>	one-off sound effects
<code>music</code>	the background track
<code>ambient:*</code>	layered environmental loops (wind, machinery, ...)

A story drives these channels by firing **`media.*` events**:

`media.image.show`, `media.sound.play`, `media.music.play`, `media.ambient.play` (and their `.hide` / `.stop` partners). The standard channels listen for those events on the turn's event stream and project them; the browser renderers do the rest. That's the through-line of this chapter: *you emit a `media.*` event, and the channel surface turns it into*

something the player sees or hears. Because these are ordinary channels, an image's hotspot can carry a `command` that the client routes back through `engine.executeTurn`, a clickable region that plays exactly like a typed verb.

27.2 Capability gating

The reason a terminal never mis-renders an image is **capability gating**, the mechanism from the Channels chapter. At startup the client declares what it can do. The browser declares a full graphical profile, with `images`, `sound`, `music`, `animations`, and more all `true`, so every media channel appears in its manifest. A text-only client declares them `false`, and the engine simply leaves the media channels out of *that* client's manifest. The story emits the same signals regardless; the gate decides who receives them. You never write “if the client supports images.” The manifest already did.

27.3 The audio model

Audio is just more `media.*` events, each projected onto a standard channel:

- **`media.sound.play`**: a one-off sound effect (`src`, optional `volume`, `pan`). Read by the `sound` channel.
- **`media.music.play` / `media.music.stop`**: start or crossfade the `music` channel's track, or stop it. Music loops by default.
- **`media.ambient.play` / `media.ambient.stop`**: start or stop a loop on an `ambient:<id>` channel. Reusing the same `channel id` replaces its source, so a soundscape swaps as the player moves room to room.
- **`media.image.show` / `media.image.hide`**: show or hide an image on an `image:<layer>` channel (`background`, `main`, `overlay`).

Every duration is in milliseconds and every volume runs 0.0–1.0.

A note on `@sharpee/media`. Sharpee also ships a `@sharpee/media` package (ADR-138) with an older `audio.*` event vocabulary and an `AudioRegistry`. That vocabulary predates the channel surface; the events the channels actually consume today are the `media.*` set above. The `AudioRegistry` is still useful, not as an emitter but as a *data store* for room atmospheres, which we use below.

27.4 Supplying your own assets

Sharpee ships no audio or images. The `src` of every `media.*` event is a path **you provide**. You source the files, drop them in one place, and the build bundles them.

Put assets under an `assets/` directory at your project root, in whatever subfolders your `src` paths use:

```
my-zoo/  
  assets/  
    audio/aviary-birdsong.mp3  
    images/aviary.jpg  
  src/  
    browser/
```

`sharpee build --browser` copies the contents of `assets/` into the web bundle, so a `src` like `audio/aviary-birdsong.mp3` resolves at the page root, `dist/web/audio/aviary-birdsong.mp3`. There's no magic mapping: the folder layout you choose under `assets/` is the layout your `src` paths reference, copied across as-is.

Sourcing the files is your job, and so is their licensing. Use audio and images you have the right to ship. Public-domain (CC0) material is the least friction, with nothing to attribute inside a bundled game, and there are well-known CC0 sound and image collections to draw from. Whatever you choose, keep a note of each file's source and license; if a license asks for credit, surface it in your About text or an on-page credit.

A `src` with no file behind it simply fails to load: the channel still fires and the renderer still runs, the browser just 404s the missing file. So a story that *declares* a soundscape but ships no audio is silent, not broken. Wire the channels first and drop the real assets in later.

27.5 Fades, not cuts

On the browser side, the `AudioManager` plays these events through the Web Audio API with sample-accurate **fades** (ADR-169): music crossfades, ambient loops ramp in and out, so the soundscape never snaps. Sound effects are the deliberate exception: they fire instantly, because a door slam shouldn't fade in.

One browser rule shapes the wiring: audio can't start until the player interacts with the page. The client unlocks the audio context on the first command and queues any events that fired before then, so the opening turn's music waits for the player's first keystroke rather than being silently dropped.

27.6 Room atmospheres in practice

Scattering raw file paths through your story ages badly. The `AudioRegistry` lets you declare each room's **atmosphere** once, its ambient layers and an optional music track, with a fluent builder, and look it up by room later. Family Zoo v18 does exactly this:

```
const audio = new AudioRegistry();

audio.atmosphere(aviaryId)
  .ambient('audio/aviary-birdsong.mp3', 'environment', 0.4)
  .build();
audio.atmosphere(nocturnalId)
  .ambient('audio/night-crickets.mp3', 'environment', 0.3)
  .build();
```

A room-entry handler turns that data into channel signals. It's registered on the event processor and returns `Effect[]` (from `@sharpee/event-processor`). Two small helpers keep the body readable: `mediaEvent` builds a `media.*` semantic event, and `emit` wraps it in the `Effect` shape the processor expects:

```
import type { Effect } from '@sharpee/event-processor';
import { ISemanticEvent } from '@sharpee/core';

let mediaCounter = 0;
function mediaEvent(type: string, data: Record<string, unknown>):
  ISemanticEvent {
  return { id: `zoo-media-${++mediaCounter}`, type, timestamp:
    Date.now(),
    entities: {}, data };
}
function emit(type: string, data: Record<string,
  unknown>): Effect {
  return { type: 'emit', event: mediaEvent(type, data) };
}
```

On `if.event.actor_moved` the handler looks up the destination's atmosphere, emits the `media.*` events, and stops the loop for rooms that have none, building up the `effects` array it returns:

```
const atmosphere = audio.getAtmosphere(toRoom);
if (atmosphere) {
  for (const a of atmosphere.ambient) {
    effects.push(emit('media.ambient.play', {
      src: a.src, channel: a.channel, volume: a.volume, loop:
        true,
    }));
  }
} else {
  effects.push(emit('media.ambient.stop', { channel:
    'environment' }));
}
```

Sound effects are simpler still: a one-off `media.sound.play` straight from the action that causes them. In v18 the feed action emits a crunch and the photograph action a shutter click, right alongside their prose.

When the zoo closes, the after-hours daemon emits one `media.music.play` and a theme fades in. Throughout, the story only ever declares intent as a `media.*` event; the client, gated by what it can do, decides what the player actually perceives.

27.7 Key takeaway

Media are expressed as channels: images and audio ride `image:*`, `sound`, `music`, and `ambient:*`. Media channels are **capability-gated**, so a text-only client never receives them, and you never branch on client support. You drive them by firing `media.*` events, and declare each room's atmosphere once with the `AudioRegistry`, emitting it on entry. With sight and sound in place, every signal rides the one universal surface: the channel system.

VOLUME VIII — SHIPPING

28 The Multi-File Story: Putting It All Together

Every chapter so far has shown a fragment: a trait here, a daemon there, a custom action on its own. A real story is all of it at once, and by now the Family Zoo has grown past what one file should hold. This chapter is the turn from *learning Sharpee* to *shipping a Sharpee game*. It starts where every growing project does: splitting one long file into many.

28.1 When one file stops working

Through most of this book the zoo lived in a single source file because each version added just one idea. By version 17 that file holds rooms, items, characters, four custom actions, a scheduler full of daemons, an NPC plugin, scoring rules, and every line of player-facing prose. A single file that large is hard to navigate and harder to change. Touching the scoring rules means scrolling past the map, the items, and the parser grammar to find them.

The fix is the same one every codebase reaches for: split by **concern**. The guiding question is “what changes together?” The rooms change together; the scoring rules change together; the prose changes together. Each becomes a file.

28.2 Organizing by concern, not by code type

Family Zoo splits into seven files, each owning one slice of the world. All seven live in the companion repository at [tutorials/famylzoo/src/ch28-multi-file/](https://github.com/famylzoo/famylzoo/tree/master/src/ch28-multi-file); this chapter walks their structure rather than reprinting every line, so open them on GitHub alongside as we go:

File	Owns
<code>zoo-map.ts</code>	rooms, exits, scenery: the physical zoo
<code>zoo-items.ts</code>	the objects players pick up and use
<code>characters.ts</code>	the zookeeper, the parrot, pettable animals, their NPC behaviors
<code>events.ts</code>	PA announcements, feeding time, the after-hours daemons
<code>scoring.ts</code>	point values, score IDs, the victory condition
<code>language.ts</code>	every line of player-facing prose
<code>index.ts</code>	the Story class that wires them all together

Notice what the split is *not*: there's no `traits.ts`, no `actions.ts`, no `behaviors.ts`. Sharpee doesn't ask you to group code by its type. The petting feature's trait, its capability behavior, and its prose can each live near the rest of *their* concern (the animal in `characters.ts`, the message in `language.ts`) because that's what you'll edit together when you change how petting works. Group by the part of the *world* a file describes, and a change stays in one place.

28.3 Each file exports a builder and its IDs

The files don't reach into each other's internals. Each exposes a small, intentional interface: a function that builds its part of the world, and a typed set of the IDs it created so other files can refer to them.

```
// zoo-map.ts
export interface RoomIds {
  entrance: string;
  pettingZoo: string;
  aviary: string;
  // ...
}

export function createZooMap(world: WorldModel): { rooms:
  RoomIds; /* ... */ } {
  // create rooms, wire exits, add scenery
  // return the IDs the rest of the story needs
}
```

`createZooItems(world, rooms)` then takes the room IDs it needs and returns its own `ItemIds`; `createCharacters(world, rooms)` returns `CharacterIds`. IDs flow forward through the build, each file handling the next exactly what it needs and nothing more. This is the “clear boundaries” discipline from the start of the book made concrete: the map doesn’t know the items exist, and the items only know the rooms by their IDs.

28.4 The index wires it together

`index.ts` holds the `Story` class: the same four hooks you’ve used all along, now calling out to the builder functions instead of doing the work inline:

```
initializeWorld(world: WorldModel): void {
  world.setMaxScore(MAX_SCORE);

  const { rooms } = createZooMap(world);
  this.roomIds = rooms;
  this.itemIds = createZooItems(world, rooms);
  this.characterIds = createCharacters(world, rooms);

  // register the petting capability, place the player...
}
```

`getCustomActions()` returns the action objects (defined near the top of `index.ts` since they coordinate across concerns), `extendParser` adds their grammar, `extendLanguage` calls `registerMessages` from `language.ts`, and `onEngineReady` installs the NPC and scheduler plugins and registers every daemon. The class reads like a table of contents for the story, which is exactly what a wiring file should be.

28.5 A feature that spans the files: after hours

Version 17 isn't only a reorganization; it adds a second act. After enough turns the zoo closes: the zookeeper leaves, and the animals, freed from human ears, start to talk. It's the perfect feature to show how a single idea now threads cleanly through the split files instead of tangling one big one:

- **events.ts** defines the after-hours daemons (created by `createAfterHoursDaemons`) that fire the closing announcements and the animals' candid lines.
- **A world-state flag** acts as the game-phase switch: a daemon sets `zoo.after_hours` to `true`, and other daemons gate their behavior on it. State values you met in Volume III turn out to be exactly the right tool for "what act are we in?"
- **characters.ts** exports two NPC behaviors for the parrot: `parrotBehavior` (daytime squawking) and `parrotAfterHoursBehavior` (articulate). A small daemon in `index.ts` watches the flag and performs a **runtime behavior swap**: when `zoo.after_hours` flips, it calls `npcService.removeBehavior('zoo-parrot')` and registers the after-hours behavior in its place. Swapping a behavior at runtime is the canonical way to change how an NPC acts mid-game.
- **scoring.ts** grows a bonus tier: 25 points for witnessing the after-hours events, on top of the 75-point base game.

Every one of those touches lands in the file that owns its concern: the daemons in `events.ts`, the phase flag in world state, the behavior swap

in `index.ts`, the bonus tier in `scoring.ts`. A whole second act, and no single file grew harder to read.

28.6 Key takeaway

As your story grows, one file becomes unwieldy. Split your story elements by **concern**: things that change together are sticky, so they belong in the same file. Each file exposes a builder and a typed set of IDs that flow forward through the build, so files stay decoupled and the `Story` class in `index.ts` is just a thin wiring layer. Version 17 proves the structure by adding a whole second act (the after-hours phase) without making any one file harder to read. The rest of this volume is about getting the zoo to players: testing, saving, building, and serving it.

29 Transcript Testing & Walkthroughs: Proving the Game Still Works

The zoo is a real game now, spread across seven files and two acts. The moment you add an eighth feature, you risk breaking one of the first seven. You need a way to *play the whole game automatically* and be told the instant something stops working. That's what transcript testing is: a recorded playthrough the engine replays and checks for you.

29.1 A test that reads like play

A transcript is a plain-text file of commands and expectations that looks like a session at the keyboard. A YAML header names it; a `---` ends the header; then each `>` line is a command and each `[...]` line is an assertion:

```
title: Feed the goats
story: familyzoo
description: Feeding the pygmy goats awards points and consumes the feed

---

> take feed
[OK: contains "Taken"]

> south
[OK: contains "Petting Zoo"]

> feed goats
[OK: contains "devour"]
[OK: not contains "don't have"]
```

Run it and the tester drives those commands through the real engine, checking each assertion against the actual output. A passing run is silent; a failing one tells you exactly which command produced the wrong result.

Comments start with `#`; section headers start with `##` and just organize the output.

29.2 Two kinds of test

Sharpee distinguishes two transcript styles, and the distinction matters:

- **Unit transcripts** (`tests/transcripts/*.transcript`) are short and isolated. Each gets a *fresh* game. Use them to pin down one feature or puzzle: “feeding the goats consumes the feed,” “the camera is required to photograph.”
- **Walkthroughs** (`walkthroughs/wt-*.transcript`) are long and *chained*: game state persists from one file to the next, so `wt-02` begins where `wt-01` left off. Together they verify the whole game progresses end to end. Walkthroughs checkpoint with `$save <name>` at the end and `$restore <name>` at the start, so each segment resumes the prior one’s state.

A unit test answers “does this feature work in isolation?” A walkthrough answers “can a player actually finish the game?” You want both.

29.3 Asserting more than text

`[OK: contains "..."]` covers most cases, but the tester checks three layers:

- **Text**: `contains`, `not contains`, `contains_any` "a" "b", `matches /regex/i`. Invert with `[FAIL: ...]` to assert a check should *not* pass; defer with `[SKIP]` or `[TODO: ...]`.
- **Events**: `[EVENT: true, type="if.event.taken"]` asserts the engine emitted a given semantic event this turn, independent of the prose. Useful when an action’s text varies but its event shouldn’t, e.g. confirming `feed goats` emits your `zoo.event.fed` event.

- **State:** `[STATE: true, player.inventory contains feed]` checks the world model directly. This is the strongest assertion: it verifies the *mutation*, not just the words. After `feed goats`, assert the `fed-...` state flag is set and the score went up: the actual effects, not the message describing them.

A message can read correctly while the world behind it is wrong (a score that never incremented, an item that was never consumed), and only a state assertion catches that.

29.4 Handling variable outcomes

Real playthroughs aren't perfectly deterministic; an NPC might wander, a daemon might fire on a different turn. Transcripts have control-flow directives for that:

- **[GOAL: ...]** / **[END GOAL]** group commands into a named objective, with optional **[REQUIRES: ...]** preconditions and **[ENSURES: ...]** postconditions, so the goal fails if the world isn't in the expected state before or after.
- **[IF: ...]** / **[END IF]** runs commands only when a condition holds (the parrot is here, the trunk wasn't stolen).
- **[WHILE: ...]** / **[END WHILE]** loops (capped at 100 iterations), and **[NAVIGATE TO: "Room"]** walks the player somewhere by name; together they cope with a randomized exit or a roaming NPC without hard-coding one exact path.

For the zoo, the after-hours act is the natural thing to bracket in a `[GOAL] : [ENSURES: ...]` the after-hours bonus was scored once the closing sequence has run.

29.5 Running them

Authors run tests through the build CLI, which compiles, bundles, and tests in one step:


```
npx sharpee build --test  
# run every transcript it finds  
npx sharpee build --test --stop-on-failure
```

It runs `walkthroughs/wt-*.transcript` as a chain and `tests/transcripts/*.transcript` individually, exactly matching the two-kinds split.

29.6 Key takeaway

A transcript test replays a recorded sequence of commands through the real engine and checks each turn against assertions you write. **Unit transcripts** run in isolation on a fresh game; **walkthroughs** (`wt-*`, run with `--chain`) keep state across files to verify the whole game finishes. Assert on text, **events**, or **state**. State is the strongest, because it checks the mutation, not the message. Control-flow directives (`[GOAL]`, `[IF]`, `[WHILE]`, `[NAVIGATE T0]`) absorb the variation real play introduces. A green suite is your license to keep adding features without fear; next we make sure a player's *own* progress survives: saving and restoring.

30 Saving & Restoring: State Lives in the World

A full play of the zoo runs to dozens of turns across two acts. Players will want to stop and come back. So how do you make the zoo saveable? The happy answer is the theme of this whole book: mostly, you don't, because the architecture already did it. This chapter explains why, and pins down the one case where you *do* write save code.

30.1 State lives in the world, so it saves itself

Every meaningful thing that changes during play lives in the **world model**: where the player is, what's in their inventory, which animals you've fed (the `fed-...` state flags), whether the zoo has closed (`zoo.after_hours`), and the scoring ledger from Volume VI. When the engine saves, it serializes the *entire* world into a single `ISaveData`, a complete snapshot carried in a compressed `worldSnapshot`.

Because your game state is *in* the world rather than in loose variables scattered through your code, restoring is just rebuilding the world from that snapshot. Score, entity positions, container contents, state flags, relationships, the ID counters: all of it comes back, because all of it was in the world to begin with. The author who kept state in the world (as every earlier chapter taught) gets save/restore for free.

30.2 The one thing you must save yourself

There is exactly one trap, and v17 already walks into it on purpose so you can see the fix. The after-hours **behavior swap** daemon keeps a local flag so it only fires once:

```

let behaviorSwapped = false;
scheduler.registerDaemon({
  id: 'zoo.daemon.parrot_behavior_swap',
  condition: (ctx) => !behaviorSwapped &&
    ctx.world.getStateValue('zoo.after_hours') === true,
  run: () => {
    behaviorSwapped = true;
    npcService.removeBehavior('zoo-parrot');
    npcService.registerBehavior(parrotAfterHoursBehavior);
    return [];
  },
  // ...
});

```

That `behaviorSwapped` variable lives in a closure, **not** in the world, so the world snapshot doesn't capture it. Save after the swap, restore, and a naïve daemon would think the swap hadn't happened and try to run again. The daemon avoids that by implementing two hooks:

```

getRunnerState(): Record<string, unknown> { return
  { behaviorSwapped }; },
restoreRunnerState(state): void { behaviorSwapped =
  (state.behaviorSwapped as boolean) ?? false; },

```

`getRunnerState` hands the engine the transient flag to fold into the save; `restoreRunnerState` reads it back. That's the rule: **any state you hold outside the world (a counter in a closure, a flag on a daemon) you must surface through these hooks, or it won't survive a save.** The cleaner your story keeps its state in the world, the less of this you write.

30.3 How the browser persists a save

The engine produces the `ISaveData`; the *client* decides where to put it. In the browser client from Volume VII, a save is wrapped in a `BrowserSaveEnvelope`, the engine snapshot plus a little browser-only

metadata (the visible score, the scrollbar transcript), and written to `localStorage`. Two paths use it:

- **Manual save/restore** through the menu, into named slots.
- **Autosave**, which piggy-backs on the per-turn channel packet: every turn boundary fires `channel:packet`, so the client captures a fresh autosave each turn with no story code at all. Re-open the page and the autosave restores you mid-game.

Restoring reverses the wrap: the client unwraps the envelope and hands the engine snapshot back, and the engine rebuilds the world from it, which is why the post-restore status line and score are correct without the client recomputing anything.

30.4 Save formats change with a version

A save is a serialized snapshot, and snapshots have a shape. The envelope carries a version field precisely so that a future change to the format can be *read* rather than silently misinterpreted: a newer client can recognize an older save and refuse or adapt instead of loading garbage. As an author you don't manage this, but it's worth knowing the format is versioned: a save written by one build won't be mistaken for a different shape by the next.

30.5 Key takeaway

Save and restore come almost free because game state lives in the **world**: the engine serializes the whole thing into one `ISaveData` that rebuilds on restore, score and positions and flags and all. The one thing you handle yourself is **transient state held outside the world**: a closure flag or daemon counter is invisible to the snapshot, so expose it through `getRunnerState / restoreRunnerState`, as v17's behavior-swap daemon does. In the browser, saves are versioned `localStorage` envelopes, autosaved every turn. With the game tested and persistable, it's time to hand it to players: building and publishing.

31 Building & Publishing: The Single-Player Browser Client

The zoo runs, it's tested, it saves. The last step is the one that turns a project into a *game*: getting it onto a screen that isn't yours. For single-player Sharpee that means one artifact, a self-contained browser client, and a handful of CLI commands to produce it.

31.1 The author toolchain

Throughout this book you've used the `sharpee` command from `@sharpee/devkit`. It's the whole build toolchain for a standalone story: it compiles your TypeScript and emits the runnable artifacts. (You install it globally as `sharpee`, below; if you ever work *inside* the Sharpee monorepo itself, the same CLI is invoked through the repo's `./sharpee` wrapper.) Crucially, *the platform is never rebuilt*: `@sharpee/sharpee` is an ordinary npm dependency your story compiles against, so your builds are fast and reproducible.

```
npm install -g @sharpee/devkit    # one-time
sharpee init my-game -y          # scaffold src/index.ts,
                                package.json, tsconfig.json
cd my-game && npm install         # pull the platform from npm
sharpee build                    # compile src/ → dist/, emit
                                the .sharpee bundle
```

`sharpee build` produces two things in `dist/`: the compiled story (`dist/index.js`) and a **.sharpee bundle**, a single zipped file of your whole story, the unit you hand to anything that runs Sharpee stories.

31.2 Adding the browser client

A `.sharpee` bundle needs a runner. For single-player, that runner is the framework-free browser client from Volume VII, and devkit builds a self-contained copy of it wrapped around your story:

```
sharpee init-browser  # adds src/browser-entry.ts (once)
sharpee build         # now also emits the web client → dist/web/
```

`dist/web/` is the deliverable: an `index.html` and its assets with your story baked in. It has no server, no build step, and no runtime dependency: open `index.html` and the game runs. The same channel architecture that let one story drive a terminal or a browser is what makes the browser build nothing but static files.

31.3 Hosting it

Because the client is static, “publishing” is “putting files on the web.” Any static host works: a personal site, GitHub Pages, itch.io, an S3 bucket. To check it locally before you upload, point any static file server at the folder:

```
sharpee build
python3 -m http.server -d dist/web
# then open the printed http://localhost:8000
```

Upload the contents of `dist/web/` and share the URL. There is nothing to install on the player’s side; their browser is the interpreter.

31.4 Versioning

Sharpee stamps a version into every build, in the form `X.Y.Z` (with a `-beta` suffix during development), and, importantly, stamps it *first*, before any compilation, so the number baked into the artifact always matches the build that produced it. The version shows in the client’s

About box and travels in saves, which is what lets the save format reason about which build wrote a file (the previous chapter’s versioned envelope). Bump it as you’d bump any package: a patch for fixes, a minor for new content, a major for a breaking change.

31.5 Publishing a story as a package

Shipping the browser client covers most authors. If instead you want to distribute your *story* for others to embed or extend, as an npm package rather than a playable build, you publish it like any scoped package, with your compiled `dist/` and the `.sharpee` bundle as its artifacts.

(Publishing the **platform** packages themselves, the `@sharpee/*` scope, is a maintainer task that runs through the monorepo’s `tsf` toolchain, and isn’t something a story author needs.)

31.6 Key takeaway

Single-player publishing produces one self-contained artifact. `sharpee build` compiles your story and emits a **.sharpee bundle**; `sharpee init-browser` then `sharpee build` wraps it in the framework-free browser client at `dist/web/`, a static `index.html` with no server and no install, which you host anywhere static files live and verify locally with any file server. Builds are fast because the platform is a pinned npm dependency, never rebuilt; every build is **version-stamped first** so the number always matches the artifact. That ships the game to one player in a browser, a complete and hostable artifact, and the finish line for the zoo you’ve built chapter by chapter.

Appendix A — Architecture Map

This appendix is a one-page map of where things live in Sharpee. Every concept the book taught belongs to one of these layers; when you're unsure where a new feature goes, the rule of thumb is the question that opens this appendix: *what layer owns this?*

31.7 The layers

Sharpee is built in layers, each with a single responsibility. Dependencies flow *inward* — a story depends on the platform, never the reverse; the engine knows nothing about any particular game.

Layer	Package(s)	Owns	In this book
Engine	@sharpee/ engine	The turn cycle, command execution, event dispatch, scheduler, save/ restore, the channel manifest + per-turn packets	Vol. I (the play loop), VI (turns & daemons), VIII (saving)
World model	@sharpee/ world-model	Entities, traits, and behaviors — all game <i>state</i> and the rules that mutate it	Vol. II (building a world), IV (custom traits & behaviors)
Standard library	@sharpee/ stdlib	The standard actions (validate/ execute/report/ blocked), scope	Vol. III (actions, scope), IV (capability dispatch)

Layer	Package(s)	Owns	In this book
		& visibility, capability dispatch, the standard channels	
Parser	@sharpee/ parser-en-us	Grammar patterns — turning typed text into a resolved command	Vol. V (extending the grammar)
Language	@sharpee/lang- en-us	All player-facing text: message IDs, templates, the formatter chain	Vol. V (the language layer, formatter chain)
Story	your project	Game-specific content and overrides — rooms, items, NPCs, custom actions, puzzles	the whole Family Zoo running example
Client	@sharpee/ platform- browser (and others)	UI: rendering channel packets to a screen, reading input	Vol. VII (the web client, decoration, media)

31.8 The two flows

Two directions of flow connect the layers, and together they *are* a turn:

Input — text becomes a **mutation**. The **client** reads a command and hands it to the **engine**, which asks the **parser** to resolve it against the

grammar (plus any the **story** added). The engine runs the matching **stdlib** action, which validates, then mutates the **world model** through behaviors, then reports what happened as events.

Output — state becomes presentation. The engine renders the turn’s prose through the **language** layer’s templates and formatter chain into text blocks, then asks every **channel** “what do you have this turn?” and assembles a **packet**. The **client** hands each channel’s value to a renderer that updates the screen.

31.9 The universal surface

The single idea that ties Volume VII back to everything before it: **every story-to-UI signal travels as a channel** — prose, score, location, prompt, images, sound. There is no special path for any of them. That data-only packet stream is what lets one unchanged story run in a terminal or a browser.

31.10 Where does this belong?

When adding a feature, ask in this order:

1. Is it game state or a rule that changes it? → **world model** (a trait/behavior).
2. Is it a common interaction pattern? → **stdlib** (an action, or capability dispatch).
3. Is it new vocabulary? → **parser** (grammar) and **language** (messages).
4. Is it specific to *this* game? → **story**.
5. Is it about how something *looks or sounds*? → **client** (a channel + renderer).

If a feature seems to need a change to the **engine** itself, stop — the architecture usually already supports what you want through a trait, a

capability, an event handler, or a channel. Reach for a platform change only when all of those genuinely don't fit.

Appendix B — Action Catalog

Every standard action shipped by `@sharpee/stdlib`, with the verb forms a player can type for it (from `@sharpee/lang-en-us`). Generated from the platform's public API (`IFActions` + the language provider's patterns); 67 actions.

Action ID	Player phrasing
<code>if.action.about</code>	about; info; credits
<code>if.action.again</code>	again; g
<code>if.action.answering</code>	answer [response]; reply [response]; respond [response]; say yes; say no; yes; no
<code>if.action.asking</code>	ask [someone] about [topic]; ask [someone] for [topic]; question [someone] about [topic]; inquire [someone] about [topic]; query [someone] about [topic]
<code>if.action.attacking</code>	attack [something]; attack [something] with [weapon]; hit [something]; hit [something] with [weapon]; strike [something]; strike [something] with [weapon]; fight [something]; kill [something]; break [something]; destroy [something]; smash [something]
<code>if.action.climbing</code>	climb; climb [something]; climb up; climb down; climb up [something]; climb down [something]; climb on [something]; climb onto [something]; climb over [something]; scale [something];

Action ID

Player phrasing

`if.action.closing`

ascend; ascend [something];
descend; descend [something];
scramble up [something]; clamber
up [something]; shin up
[something]
close [something]; shut
[something]

`if.action.consulting`

—

`if.action.drinking`

drink [something]; drink from
[something]; sip [something]; sip
from [something]; quaff
[something]; imbibe [something];
swallow [something]

`if.action.dropping`

drop [something]; put down
[something]; discard [something];
release [something]; let go of
[something]

`if.action.eating`

eat [something]; consume
[something]; devour [something];
munch [something]; munch on
[something]; nibble [something];
nibble on [something]; taste
[something]

`if.action.emptying`

—

`if.action.entering`

enter [something]; get in
[something]; get into [something];
get on [something]; go in
[something]; go into [something];
go inside [something]; board
[something]; mount [something];
sit on [something]; sit in
[something]; lie on [something]; lie

Action ID

Player phrasing

`if.action.entering_room`

in [something]; stand on
[something]; stand in [something];
climb in [something]; climb into
[something]; climb on [something];
climb onto [something]

`if.action.examining`

—
examine [something]; x
[something]; look at [something];
inspect [something]; study
[something]; read [something]

`if.action.exiting`

exit; get out; get off; go out; go
outside; leave; dismount; stand;
stand up; climb out; climb off;
disembark; alight

`if.action.giving`

give [something] to [someone];
give [someone] [something]; offer
[something] to [someone]; offer
[someone] [something]; hand
[something] to [someone]; hand
[someone] [something]; present
[something] to [someone]; present
[someone] [something]

`if.action.going`

go [direction]; [direction]; walk
[direction]; head [direction]; move
[direction]; travel [direction]

`if.action.help`

help; help [topic]; ?; commands; h

`if.action.hiding`

hide behind [thing]; hide under
[thing]; hide on [thing]; hide in
[thing]; duck behind [thing];
crouch behind [thing]

`if.action.hints`

—

`if.action.inserting`

Action ID

if.action.inventory

if.action.jumping

if.action.kissing

if.action.listening

if.action.locking

if.action.locking

if.action.locking_behind

if.action.locking_under

if.action.lowering

if.action.opening

if.action.pulling

if.action.pushing

Player phrasing

insert [something] in [something];

insert [something] into

[something]; stick [something] in

[something]; push [something] in

[something]

inventory; i; inv; take inventory;

check inventory

—

—

listen; listen to [something]; hear

[something]

lock [something]; lock [something]

with [something]; secure

[something]; secure [something]

with [something]

look; l; look around; look at

[something]; examine [something];

x [something]

—

—

lower [something]

open [something]; open up

[something]; uncloze [something]

pull [something]; pull [something]

[direction]; tug [something]; tug on

[something]; drag [something];

drag [something] [direction]; yank

[something]; draw [something]

push [something]; push

[something] [direction]; press

[something]; shove [something];

Action ID

if.action.putting

if.action.quitting

if.action.raising

if.action.reading

if.action.removing

if.action.restarting

if.action.restoring

if.action.revealing

if.action.saving

if.action.scoring

Player phrasing

shove [something] [direction];

move [something]; move

[something] [direction]

put [something] in [something];

put [something] into [something];

put [something] on [something];

put [something] onto [something];

place [something] in [something];

place [something] on [something];

insert [something] in [something];

insert [something] into

[something]

quit; exit; bye; goodbye; end; stop;

quit game; end game

raise [something]; lift [something]

read [something]; peruse

[something]

remove [something] from

[something]; take [something]

from [something]; take

[something] out of [something];

get [something] from [something];

extract [something] from

[something]

—

restore; restore [name]; load; load

[name]; load game; restore game

stand up; come out; reveal myself;

unhide; stop hiding

save; save game; save [name]; save

as [name]; store; store game

score; points

Action ID

Player phrasing

if.action.searching

search [something]; look in
[something]; look inside
[something]; look through
[something]; rummage
[something]; rummage in
[something]; rummage through
[something]; examine [something]
closely

if.action.setting

—

if.action.showing

show [something] to [someone];
show [someone] [something];
display [something] to [someone];
reveal [something] to [someone];
present [something] to [someone]

if.action.sleeping

sleep; nap; doze; rest; slumber; z

if.action.smelling

smell; smell [something]; sniff
[something]; sniff; inhale

if.action.switching_off

switch off [something]; switch
[something] off; turn off
[something]; turn [something] off;
deactivate [something]; stop
[something]; power off
[something]; power [something]
off

if.action.switching_on

switch on [something]; switch
[something] on; turn on
[something]; turn [something] on;
activate [something]; start
[something]; power on
[something]; power [something] on

if.action.taking

take [something]; get [something];
pick up [something]; grab

Action ID

`if.action.taking_off`

`if.action.talking`

`if.action.tasting`

`if.action.telling`

`if.action.throwing`

`if.action.touching`

`if.action.turning`

Player phrasing

[something]; acquire [something];
collect [something]

take off [something]; take
[something] off; remove
[something]; doff [something]
talk to [someone]; talk [someone];
speak to [someone]; speak with
[someone]; greet [someone]; say
hello to [someone]; chat with
[someone]

—
tell [someone] about [topic];
inform [someone] about [topic];
notify [someone] about [topic]; say
[topic] to [someone]

throw [something]; throw
[something] at [target]; throw
[something] to [target]; throw
[something] [direction]; hurl
[something]; hurl [something] at
[target]; toss [something]; toss
[something] to [target]; chuck
[something]; fling [something]; lob
[something]

touch [something]; feel
[something]; pat [something];
stroke [something]; poke
[something]; prod [something]
turn [something]; turn [something]
[direction]; turn [something] to
[setting]; rotate [something]; rotate
[something] [direction]; twist

Action ID

if.action.undoing

if.action.unlocking

if.action.verifying

if.action.version

if.action.waiting

if.action.waking

if.action.waving

if.action.wearing

Player phrasing

[something]; twist [something]
[direction]; spin [something]; dial
[something]; dial [something] to
[setting]; crank [something]

undo

unlock [something]; unlock
[something] with [something];
open [something] with
[something]

—

version

wait; z

—

—

wear [something]; put on
[something]; put [something] on;
don [something]

Appendix C — Trait Catalog

Every trait type defined by `@sharpee/world-model`. A trait is a unit of state and capability you add to an entity. Generated from the platform's public `TraitType` map; 40 traits.

Trait type	Description
actor	A living being — the player or an NPC.
attached	Fixed to another object; cannot be taken on its own.
breakable	Can be broken by force.
button	A pressable control.
characterModel	Rich NPC character data (ADR-141).
climbable	Can be climbed.
clothing	A wearable garment (a specialized wearable).
combatant	Takes part in combat (health, attack).
container	Holds other objects inside it.
destructible	Can be destroyed.
door	A connection between two rooms; often openable/lockable.
edible	Can be eaten or drunk.
enterable	The player can get inside or onto it.
equipped	Currently equipped or wielded.
exit	A directional exit object.
identity	

Trait type	Description
	Name, description, aliases, and article — every entity's basic identity.
<code>if.trait.acoustic</code>	Emits or carries sound (acoustic model).
<code>if.trait.acoustic_dampener</code>	Dampens sound passing through it.
<code>if.trait.concealed_state</code>	Tracks whether the object is currently concealed.
<code>if.trait.concealment</code>	Conceals other objects.
<code>if.trait.listener</code>	Reacts to sounds it can hear.
<code>lightSource</code>	Provides light; banishes darkness when lit.
<code>lockable</code>	Can be locked and unlocked with a key.
<code>moveableScenery</code>	Scenery that can be moved to reveal something.
<code>npc</code>	Marks a non-player character with behavior.
<code>openInventory</code>	Its contents are visible without opening it.
<code>openable</code>	Can be opened and closed.
<code>pullable</code>	Can be pulled.
<code>pushable</code>	Can be pushed.
<code>readable</code>	Has text the player can read.
<code>region</code>	Groups rooms into a named region.
<code>room</code>	A location the player can occupy.
<code>scene</code>	A scripted scene with activation/deactivation.
<code>scenery</code>	

Trait type	Description
	Fixed background detail; not takeable, not listed in room contents.
storyInfo	Story metadata — title, author, version.
supporter	Holds other objects on top of it.
switchable	Can be switched on and off.
vehicle	A conveyance the player can enter and travel in.
weapon	Can be used to attack.
wearable	Can be worn.

Appendix D — Message-ID Reference

Every message ID registered by @sharp/eng-en-us , with its default English text. Override any of these from a story with extendLanguage (Volume V). Generated from the language provider; 808 messages in 82 groups.

character.conversation

Message ID	Default text
character.conversation.ends	{npcName} nods, ending the conversation.
character.conversation.initiates	{npcName} approaches you. “A word, if you please.”

character.conversation.attention

Message ID	Default text
character.conversation.attention.blocks	{npcName} steps in front of you. “We’re not done here.”
character.conversation.attention.protests	{npcName} frowns as you turn away. “I wasn’t finished.”
character.conversation.attention.yields	{npcName} steps aside, yielding the conversation.

character.conversation.between.confessing

Message ID	Default text
character.conversation.between.confessing.1	{npcName} shifts uncomfortably, as though wanting to say more.
character.conversation.between.confessing.3	{npcName} opens their mouth, then closes it again.

character.conversation.between.eager

Message ID	Default text
character.conversation.between.eager.1	{npcName} watches you expectantly.
character.conversation.between.eager.3	{npcName} clears their throat, waiting for your attention.

character.conversation.between.hostile

Message ID	Default text
character.conversation.between.hostile.1	{npcName} glares at you impatiently.

character.conversation.between.neutral

Message ID	Default text
character.conversation.between.neutral.3	{npcName} seems to lose interest in the conversation.

character.conversation.between.reluctant

Message ID

Default text

character.conversation.between.reluctant.1	{npcName} seems relieved you're occupied.
--	---

character.conversation.cognitive

Message ID

Default text

character.conversation.cognitive.detached	{npcName} responds flatly, as if reciting from a great distance.
character.conversation.cognitive.drifting	{npcName} trails off, attention wandering to something only they can see.
character.conversation.cognitive.fragmented	{npcName} speaks in broken fragments, losing the thread mid-sentence.

character.conversation.response

Message ID

Default text

character.conversation.response.confabulate	{npcName} seems to be filling in the gaps from memory.
character.conversation.response.deflect	{npcName} changes the subject.
character.conversation.response.omit	

Message ID**Default text**

character.conversation.response.refuse

{npcName} says
nothing about that.

{npcName} refuses
to discuss that.

character.influence.effect**Message ID****Default text**

character.influence.effect.departed

With {influencerName} gone,
{targetName} regains
composure.

character.influence.effect.expired

The influence over
{targetName} fades.

character.influence.pc**Message ID****Default text**

character.influence.pc.action_intercepted

You find it hard to
concentrate.

character.influence.pc.focus_clouded

You were about to do
something, but you've
lost your train of
thought.

character.influence.resisted**Message ID****Default text**

character.influence.resisted.default

{targetName} seems
unaffected by
{influencerName}.

character.influence.witnessed

Message ID	Default text
<code>character.influence.witnessed.default</code>	<code>{influencerName}</code> exerts a subtle influence on <code>{targetName}</code> .

character.propagation

Message ID	Default text
<code>character.propagation.eavesdropped</code>	You overhear <code>{speakerName}</code> speaking to <code>{listenerName}</code> .

character.propagation.witnessed

Message ID	Default text
<code>character.propagation.witnessed.conspiratorial</code>	<code>{speakerName}</code> leans close to <code>{listenerName}</code> , muttering under their breath.
<code>character.propagation.witnessed.dramatic</code>	<code>{speakerName}</code> excitedly tells <code>{listenerName}</code> about something.
<code>character.propagation.witnessed.fearful</code>	<code>{speakerName}</code> nervously whispers something to <code>{listenerName}</code> .
<code>character.propagation.witnessed.neutral</code>	<code>{speakerName}</code> mentions

Message ID

Default text

character.propagation.witnessed.vague

something to
{listenerName}.
{speakerName}
vaguely alludes
to something
near
{listenerName}.

core

Message ID

Default text

core.ambiguous_reference

Which do you mean?

core.command_failed

I don't understand that.

core.command_not_understood

I don't understand that command.

core.disambiguation_prompt

Which do you mean: {options}?

core.entity_not_found

You can't see any such thing.

game.started

Message ID

Default text

game.started.banner

{title} By {author} Type HELP for
instructions.

if.action.about

Message ID

Default text

if.action.about.about_compact

{title} v{version} by {author}

if.action.about.about_footer

Thank you for playing!

if.action.about.about_header

About {title}

if.action.about.acknowledgments

Message ID	Default text
	Acknowledgments: {acknowledgments}
if.action.about.contact	Contact: {contact}
if.action.about.copyright	Copyright {copyright}
if.action.about.credits_header	Credits:
if.action.about.credits_list	{credits}
if.action.about.dedication	Dedication: {dedication}
if.action.about.description	Description: {description}
if.action.about.engine_info	Powered by {engine} version {engineVersion}
if.action.about.enjoy_game	We hope you enjoy playing {title}!
if.action.about.game_info	{title} Version {version} By {author} Released: {releaseDate}
if.action.about.game_info_simple	{title} by {author}
if.action.about.license	License: {license}
if.action.about.play_stats	Current Session: Time played: {playTime} Moves made: {sessionMoves}
if.action.about.session_info	You've been playing for {playTime} and made {sessionMoves} moves.
if.action.about.special_thanks	Special Thanks: {specialThanks}
if.action.about.technical_info	Technical Information: Engine: {engine} v{engineVersion}
	Platform: Interactive Fiction
if.action.about.website	Website: {website}

if.action.again

Message ID

Default text

if.action.again.nothing_to_repeat There is nothing to repeat.

if.action.answering

Message ID

Default text

if.action.answering.accepted	{Your} answer is accepted.
if.action.answering.answered	{You} {answer}, “{response}”
if.action.answering.answered_no	{You} {say}, “No.”
if.action.answering.answered_yes	{You} {say}, “Yes.”
if.action.answering.confused_by_answer	{Your} answer seems to confuse them.
if.action.answering.gave_answer	{You} {respond} to the question.
if.action.answering.invalid_response	That’s not a valid answer to the question.
if.action.answering.needs_yes_or_no	Please answer yes or no.
if.action.answering.no_one_asked	No one asked {you} anything.
if.action.answering.no_question	There’s nothing to answer.
if.action.answering.noted	{Your} response is noted.
if.action.answering.rejected	{Your} answer is not what they wanted to hear.
if.action.answering.too_late	It’s too late to answer that.
if.action.answering.unclear_answer	{Your} answer isn’t clear. Try again.

if.action.asking

Message ID

Default text

if.action.asking.already_told	{the:cap:target} says, “I already told you about that.”
if.action.asking.confused	{the:cap:target} looks confused.
if.action.asking.earned_trust	{the:cap:target} says, “Since you’ve proven yourself, I’ll tell you...”
if.action.asking.explains	{the:cap:target} explains about {topic}.
if.action.asking.must_do_first	{the:cap:target} says, “There’s something you need to do first.”
if.action.asking.no_idea	{the:cap:target} says, “No idea what you’re talking about.”
if.action.asking.no_target	Ask whom?
if.action.asking.no_topic	Ask about what?
if.action.asking.not_actor	{You} can only ask questions of people.
if.action.asking.not_visible	{You} {can’t} see {the:target}.
if.action.asking.not_yet	{the:cap:target} says, “I can’t tell you about that yet.”
if.action.asking.remembers	{the:cap:target} says, “Ah yes, about {topic}...”
if.action.asking.responds	{the:cap:target} tells you about {topic}.
if.action.asking.shrugs	{the:cap:target} shrugs.
if.action.asking.too_far	{the:cap:target} is too far away.
if.action.asking.unknown_topic	{the:cap:target} says, “I don’t know anything about that.”

if.action.attacking

Message ID

if.action.attacking.already_dead

if.action.attacking.attack_ineffective

if.action.attacking.attacked

if.action.attacking.attacked_with

if.action.attacking.broke

if.action.attacking.debris_created

if.action.attacking.defends

if.action.attacking.destroyed

if.action.attacking.dodges

if.action.attacking.flees

if.action.attacking.hit_blindly

if.action.attacking.hit_target

Default text

{the:cap:target} is already dead.

{Your} attack has no effect on {the:target}.

{You} {attack} {the:target}.

{You} {attack} {the:target} with {the:weapon}.

{You} {break} {the:target}!

Debris from {the:target} litters the area.

{the:cap:target} defends against {your} attack.

{You} {destroy} {the:target}!

{the:cap:target} dodges {your} attack.

{the:cap:target} flees from {you}!

{You} {swing} wildly, hitting nothing.

{You} {hit} {the:target}.

Message ID

Default text

`if.action.attacking.hit_with`

{You} {hit}
{the:target} with
{the:weapon}.

`if.action.attacking.indestructible`

{the:cap:target} is far
too solid to damage.

`if.action.attacking.items_spilled`

{the:cap:target}'s
possessions spill
onto the ground.

`if.action.attacking.kicked`

{You} {kick}
{the:target}.

`if.action.attacking.killed_blindly`

Something dies in
the darkness.

`if.action.attacking.killed_target`

{You} {have}
defeated
{the:target}!

`if.action.attacking.need_weapon_to_damage`

{the:cap:target}
requires a weapon to
damage.

`if.action.attacking.no_fighting`

Fighting won't solve
this problem.

`if.action.attacking.no_target`

Attack what?

`if.action.attacking.not_holding_weapon`

{You} aren't holding
{the:weapon}.

`if.action.attacking.not_reachable`

{You} {can't} reach
{the:target}.

`if.action.attacking.not_visible`

{You} {can't} see
{the:target}.

`if.action.attacking.passage_revealed`

A hidden passage is
revealed!

`if.action.attacking.peaceful_solution`

Violence isn't
necessary here.

Message ID	Default text
<code>if.action.attacking.punched</code>	{You} {punch} {the:target}.
<code>if.action.attacking.retaliate</code>	{the:cap:target} fights back!
<code>if.action.attacking.self</code>	Violence against {yourself} isn't the answer.
<code>if.action.attacking.shattered</code>	{the:cap:target} shatters!
<code>if.action.attacking.smashed</code>	{You} {smash} {the:target} to pieces!
<code>if.action.attacking.struck</code>	{You} {strike} {the:target}!
<code>if.action.attacking.struck_with</code>	{You} {strike} {the:target} with {the:weapon}!
<code>if.action.attacking.target_broke</code>	{the:cap:target} breaks!
<code>if.action.attacking.target_damaged</code>	{the:cap:target} shows signs of damage. ({damage} damage dealt)
<code>if.action.attacking.target_destroyed</code>	{the:cap:target} is utterly destroyed!
<code>if.action.attacking.target_shattered</code>	{the:cap:target} shatters into pieces!
<code>if.action.attacking.unarmed_attack</code>	{You} {attack} {the:target} with {your} bare hands.
<code>if.action.attacking.unnecessary_violence</code>	

Message ID

Default text

That seems unnecessarily violent.

`if.action.attacking.violence_not_the_answer`

Violence is not the answer.

`if.action.attacking.wrong_weapon_type`

{the:cap:target} can't be damaged with that type of weapon.

if.action.attacking.combat

Message ID

Default text

`if.action.attacking.combat.already_dead`

{the:cap:target} is already dead.

`if.action.attacking.combat.cannot_attack`

{You} {can't} attack {the:target}.

`if.action.attacking.combat.ended`

The battle is over.

`if.action.attacking.combat.need_weapon`

{You} {need} a weapon to attack effectively.

`if.action.attacking.combat.no_target`

Attack what?

`if.action.attacking.combat.not_hostile`

{the:cap:target} isn't hostile.

`if.action.attacking.combat.player_died`

{You} {have} been slain!

`if.action.attacking.combat.player_resurrected`

{You} {feel} life return to {your} body.

`if.action.attacking.combat.started`

Combat has begun!

Message ID	Default text
<code>if.action.attacking.combat.target_unconscious</code>	{the:cap:target} is already unconscious.

if.action.attacking.combat.attack

Message ID	Default text
<code>if.action.attacking.combat.attack.hit</code>	{You} {hit} {the:target} for {damage} damage. {You} {land} a solid blow on
<code>if.action.attacking.combat.attack.hit_heavy</code>	{the:target}, dealing {damage} damage!
<code>if.action.attacking.combat.attack.hit_light</code>	{You} {graze} {the:target}, doing {damage} damage.
<code>if.action.attacking.combat.attack.killed</code>	{You} {have} slain {the:target}!
<code>if.action.attacking.combat.attack.knocked_out</code>	{the:cap:target} collapses, unconscious!
<code>if.action.attacking.combat.attack.missed</code>	{You} {swing} at {the:target} but miss!

if.action.attacking.combat.defend

Message ID	Default text
<code>if.action.attacking.combat.defend.blocked</code>	

Message ID**Default text**

	{the:cap:target} blocks {your} attack!
if.action.attacking.combat.defend.dodged	{the:cap:target} dodges out of the way!
if.action.attacking.combat.defend.parried	{the:cap:target} parries {your} attack!

if.action.attacking.combat.health**Message ID****Default text**

	{the:cap:target}
if.action.attacking.combat.health.badly_wounded	is badly wounded.
if.action.attacking.combat.health.dead	{the:cap:target} is dead.
if.action.attacking.combat.health.healthy	{the:cap:target} appears uninjured.
if.action.attacking.combat.health.near_death	{the:cap:target} is barely clinging to life!
if.action.attacking.combat.health.unconscious	{the:cap:target} lies unconscious.
if.action.attacking.combat.health.wounded	{the:cap:target} has been wounded.

if.action.attacking.combat.special

Message ID	Default text
	{Your} blessed weapon burns the undead!
if.action.attacking.combat.special.blessed_weapon	{Your} sword glows brightly!
if.action.attacking.combat.special.sword_glow	{Your} sword's glow fades.

if.action.climbing

Message ID	Default text
if.action.climbing.already_there	{You're} already on {the:place}.
if.action.climbing.cant_go_that_way	{You} {can't} climb {direction} from here.
if.action.climbing.climbed_down	{You} {climb} down.
if.action.climbing.climbed_onto	{You} {climb} onto {the:target}.
if.action.climbing.climbed_up	{You} {climb} up.
if.action.climbing.need_equipment	{You}'d need climbing equipment for that.

Message ID

`if.action.climbing.no_target`

`if.action.climbing.not_climbable`

`if.action.climbing.nothing_to_climb`

`if.action.climbing.too_dangerous`

`if.action.climbing.too_high`

`if.action.climbing.too_slippery`

Default text

What do {you} want to climb?

{You} {can't} climb
{the:object}.

There's nothing to climb here.

That looks too dangerous to
climb.

That's too high to climb.

It's too slippery to climb.

if.action.closing

Message ID

`if.action.closing.already_closed`

`if.action.closing.cant_reach`

`if.action.closing.closed`

`if.action.closing.no_target`

`if.action.closing.not_closable`

`if.action.closing.prevents_closing`

Default text

{the:cap:item} is already closed.

{You} {can't} reach {the:item}.

{You} {close} {the:item}.

Close what?

{the:cap:item} can't be closed.

{You} {can't} close {the:item}
while {obstacle} is in the way.

if.action.drinking

Message ID

`if.action.drinking.already_consumed`

`if.action.drinking.bitter`

`if.action.drinking.container_closed`

`if.action.drinking.drunk`

Default text

There's nothing left to drink.

{You} {drink} {the:item}. It's
quite bitter.

{You} {need} to open
{the:item} first.

{You} {drink} {the:item}.

Message ID**Default text**

<code>if.action.drinking.drunk_all</code>	{You} {drink} all of {the:item}.
<code>if.action.drinking.drunk_from</code>	{You} {drink} from {the:item}.
<code>if.action.drinking.drunk_some</code>	{You} {drink} some of {the:item}.
<code>if.action.drinking.empty_now</code>	{You} {drink} the last of the {liquidType}.
<code>if.action.drinking.from_container</code>	{You} {drink} the {liquidType} from {the:item}.
<code>if.action.drinking.gulped</code>	{You} {gulp} down {the:item}.
<code>if.action.drinking.healing</code>	{You} {drink} {the:item}. {You} {feel} better!
<code>if.action.drinking.magical_effects</code>	{You} {drink} {the:item}. {You} {feel} strange...
<code>if.action.drinking.no_item</code>	Drink what?
<code>if.action.drinking.not_drinkable</code>	That's not something {you} can drink.
<code>if.action.drinking.not_reachable</code>	{You} {can't} reach {the:item}.
<code>if.action.drinking.not_visible</code>	{You} {can't} see {the:item}.
<code>if.action.drinking.quaffed</code>	{You} {quaff} {the:item} heartily.
<code>if.action.drinking.refreshing</code>	{You} {drink} {the:item}. How refreshing!
<code>if.action.drinking.satisfying</code>	{You} {drink} {the:item}. That hits the spot.
<code>if.action.drinking.sipped</code>	{You} {take} a sip of {the:item}.
<code>if.action.drinking.some_remains</code>	{You} {drink} some {liquidType}. Some remains.
<code>if.action.drinking.still_thirsty</code>	{You} {drink} {the:item}, but {you're} still thirsty.

Message ID**Default text**

`if.action.drinking.strong`

{You} {drink} {the:item}. It's strong!

`if.action.drinking.sweet`

{You} {drink} {the:item}. It's sweet.

`if.action.drinking.thirst_quenched`

{You} {drink} {the:item}.
{Your} thirst is quenched.

if.action.dropping

Message ID**Default text**

`if.action.dropping.dropped`

Dropped.

`if.action.dropping.dropped_in`

{You} {put} {the:item} in
{the:container}.

`if.action.dropping.dropped_multi`

{item}: Dropped.

`if.action.dropping.dropped_on`

{You} {put} {the:item} on
{the:surface}.

`if.action.dropping.no_target`

Drop what?

`if.action.dropping.not_held`

{You} aren't holding {the:item}.

`if.action.dropping.nothing_to_drop` {You} aren't carrying anything.

if.action.eating

Message ID**Default text**

`if.action.eating.already_consumed`

There's nothing left of
{the:item} to eat.

`if.action.eating.awful`

{You} {eat} {the:item}. It tastes awful!

`if.action.eating.bland`

{You} {eat} {the:item}. It's rather bland.

`if.action.eating.delicious`

Message ID

Default text

if.action.eating.devoured

{You} {eat} {the:item}.
Delicious!

if.action.eating.eaten

{You} {devour} {the:item}
hungrily.

if.action.eating.eaten_all

{You} {eat} {the:item}.

if.action.eating.eaten_portion

{You} {eat} all of {the:item}.

if.action.eating.eaten_some

{You} {eat} a portion of
{the:item}.

if.action.eating.filling

{You} {eat} some of {the:item}.

if.action.eating.is_drink

{You} {eat} {the:item}. That was
filling.

if.action.eating.munched

{You} should drink {the:item},
not eat it.

if.action.eating.nibbled

{You} {munch} on {the:item}.

if.action.eating.no_item

{You} {nibble} on {the:item}.

if.action.eating.not_edible

Eat what?

if.action.eating.not_reachable

That's not something {you} can
eat.

if.action.eating.not_visible

{You} {can't} reach {the:item}.

if.action.eating.poisonous

{You} {can't} see {the:item}.

if.action.eating.satisfying

{You} {eat} {the:item}. It tastes
strange...

if.action.eating.still_hungry

{You} {eat} {the:item}. Very
satisfying!

if.action.eating.tasted

{You} {eat} {the:item}, but
{you're} still hungry.

if.action.eating.tasty

{You} {taste} {the:item}.

{You} {eat} {the:item}. It's quite
tasty.

if.action.entering

Message ID

if.action.entering.already_inside

if.action.entering.cant_enter

if.action.entering.container_closed

if.action.entering.entered

if.action.entering.entered_on

if.action.entering.no_target

if.action.entering.not_enterable

if.action.entering.not_here

if.action.entering.occupied

if.action.entering.too_full

if.action.entering.too_small

Default text

{You're} already in
{the:place}.

{You} {can't} enter
{the:place}: {reason}.

{the:cap:container} is closed.

{You} {get} into {the:place}.

{You} {get} onto {the:place}.

Enter what?

{You} {can't} enter
{the:place}.

{You} {don't} see {the:place}
here.

{the:cap:place} is already
occupied.

{the:cap:place} is full
(maximum {max} occupants).

{the:cap:place} is too small for
{you} to enter.

if.action.examining

Message ID

if.action.examining.brief_description

if.action.examining.cant_see

if.action.examining.container_closed

if.action.examining.container_contents

Default text

{description}

{You} {can't} see
{the:item} here.

{the:cap:item} is closed.

In {the:container} {you}
{see} {items}.

Message ID	Default text
if.action.examining.container_empty	{the:cap:item} is empty.
if.action.examining.container_open	{the:cap:item} is open.
if.action.examining.description	{description}
if.action.examining.examined	{description}
if.action.examining.examined_container	{description}
if.action.examining.examined_door	{description}
if.action.examining.examined_readable	{description}
if.action.examining.examined_self	{description}
if.action.examining.examined_supporter	{description}
if.action.examining.examined_switchable	{description}
if.action.examining.examined_wall	{description}
if.action.examining.examined_wearable	{description}
if.action.examining.no_description	{You} {see} nothing special about {the:item}.
if.action.examining.no_target	Examine what?
if.action.examining.not_visible	{You} {can't} see {the:item} here.
if.action.examining.nothing_special	{You} {see} nothing special about {the:item}.
if.action.examining.surface_contents	On {the:surface} {you} {see} {items}.
if.action.examining.worn_by_other	{actor} is wearing {the:item}.
if.action.examining.worn_by_you	{You} {are} wearing {the:item}.

if.action.exiting

Message ID	Default text
if.action.exiting.already_outside	{You're} not inside anything.
if.action.exiting.cant_exit	{You} {can't} exit {the:place}.
if.action.exiting.container_closed	{the:cap:container} is closed.
if.action.exiting.exit_blocked	The way out is blocked.
if.action.exiting.exited	{You} {get} out of {the:place}.
if.action.exiting.exited_from	{You} {get} {preposition} {the:place}.
if.action.exiting.must_stand_first	{You}'ll need to stand up first.
if.action.exiting.nowhere_to_go	There's nowhere to go from here.

if.action.giving

Message ID	Default text
if.action.giving.accepts	{the:cap:recipient} accepts {the:item}.
if.action.giving.given	{You} {give} {the:item} to {the:recipient}.
if.action.giving.gratefully_accepts	{the:cap:recipient} gratefully accepts {the:item}.
if.action.giving.inventory_full	{the:cap:recipient} says, "I can't carry any more."
if.action.giving.no_item	Give what?
if.action.giving.no_recipient	Give it to whom?
if.action.giving.not_actor	{You} can only give things to people.
if.action.giving.not_holding	

Message ID

Default text

	{You} aren't holding {the:item}.
	{the:cap:recipient}
<code>if.action.giving.not_interested</code>	doesn't seem interested in {the:item}.
	{the:cap:recipient} is too far away.
<code>if.action.giving.recipient_not_reachable</code>	
<code>if.action.giving.recipient_not_visible</code>	{You} {can't} see {the:recipient}.
	{the:cap:recipient}
<code>if.action.giving.refuses</code>	politely declines.
	{the:cap:recipient}
<code>if.action.giving.reluctantly_accepts</code>	reluctantly takes {the:item}.
	{You} already {have} {the:item}!
<code>if.action.giving.self</code>	
	{the:cap:recipient} says,
<code>if.action.giving.too_heavy</code>	"That's too heavy for me."

if.action.going

Message ID

Default text

<code>if.action.going.already_there</code>	{You're} already there.
<code>if.action.going.arrived</code>	{You} {arrive}.
<code>if.action.going.cant_go</code>	{You} {can't} go that way.
	{You} {can't} go through {obstacle}.
<code>if.action.going.cant_go_through</code>	
	{You} can {see} {items} here.
<code>if.action.going.contents_list</code>	
<code>if.action.going.destination_not_found</code>	{You} {can't} go that way.

Message ID

if.action.going.door_closed
if.action.going.door_locked
if.action.going.moved
if.action.going.movement_blocked
if.action.going.need_light
if.action.going.no_direction
if.action.going.no_exit
if.action.going.no_exit_that_way
if.action.going.no_exits
if.action.going.not_in_room
if.action.going.nowhere_to_go
if.action.going.room_description
if.action.going.too_dark
if.action.going.went

Default text

{the:cap:door} is closed.
{the:cap:door} is locked.
{You} {go} {direction}.
{message}
It's too dark to go that way safely.
{You}'ll have to say which direction to go.
{You} {can't} go that way.
{You} {can't} go that way.
There are no obvious exits.
{You're} not in a place where {you} can go anywhere.
{You}'ll have to say which compass direction to go in.
{name} {description}
It is pitch dark. You are likely to be eaten by a grue.
{You} {go} {direction}.

if.action.help

Message ID

if.action.help.first_time_help

Default text

New to Interactive Fiction? Try these commands to get started: -
LOOK to see where you are -
INVENTORY to see what you're carrying -
EXAMINE interesting

Message ID

Default text

`if.action.help.general_help`

objects - Go in compass directions (NORTH, SOUTH, etc.)

Welcome to Interactive Fiction!

Basic commands: - LOOK (L):

Examine your surroundings -

INVENTORY (I): List what you're carrying - EXAMINE (X) [object]:

Look at something closely - TAKE/DROP [object]: Pick up or put down items - GO [direction] or just [direction]: Move around For more help on a specific topic, type HELP [topic].

`if.action.help.help_footer`

For a complete list of commands, consult the game documentation.

`if.action.help.help_movement`

Movement commands: - GO

NORTH/SOUTH/EAST/WEST (or just N/S/E/W) - UP/DOWN (U/D) - IN/OUT - ENTER [place] - EXIT

`if.action.help.help_objects`

Object commands: - TAKE/GET [object] - DROP [object] - EXAMINE/LOOK AT [object] - OPEN/CLOSE [object] - PUT [object] IN/ON [container] - WEAR/REMOVE [clothing]

`if.action.help.help_special`

Special commands: - SAVE/RESTORE: Save and load your game - SCORE: Check your progress - WAIT (Z): Let time pass - AGAIN (G): Repeat last command - QUIT: Exit the game

`if.action.help.help_topic`

Help on {topic}:

Message ID	Default text
<code>if.action.help.hints_available</code>	Hints are available. Type HINTS to see them.
<code>if.action.help.hints_disabled</code>	Hints are not available in this game. If you're stuck, try: - LOOK around carefully - EXAMINE everything - Check your INVENTORY - Try different verbs with objects
<code>if.action.help.stuck_help</code>	
<code>if.action.help.unknown_topic</code>	No help available on '{topic}'. Type HELP for general help.

if.action.hiding

Message ID	Default text
<code>if.action.hiding.already_hidden</code>	{You're} already hidden.
<code>if.action.hiding.behind</code>	{You} {slip} behind {the target}.
<code>if.action.hiding.cant_hide_there</code>	{You} {can't} hide {position} {the target}.
<code>if.action.hiding.inside</code>	{You} {climb} into {the target}, concealing {yourself}.
<code>if.action.hiding.nothing_to_hide</code>	{You} {can't} hide there.
<code>if.action.hiding.on</code>	{You} {crouch} on {the target}, out of sight.
<code>if.action.hiding.under</code>	{You} {crawl} under {the target}.

if.action.inserting

Message ID	Default text
<code>if.action.inserting.already_there</code>	{the:cap:item} is already in {the:destination}.
<code>if.action.inserting.container_closed</code>	{the:cap:container} is closed.

Message ID

Default text

`if.action.inserting.inserted`

{You} {insert} {the:item} into {the:container}.

`if.action.inserting.no_destination`

Insert {the:item} into what?

`if.action.inserting.no_target`

Insert what?

`if.action.inserting.not_container`

{You} {can't} insert things into {the:destination}.

`if.action.inserting.not_held`

{You} {need} to be holding {the:item} first.

`if.action.inserting.not_insertable`

{the:cap:item} can't be inserted into things.

`if.action.inserting.wont_fit`

{the:cap:item} won't fit in {the:container}.

if.action.inventory

Message ID

Default text

`if.action.inventory.burden_heavy`

{You're} carrying quite a load.

`if.action.inventory.burden_light`

{You're} traveling light.

`if.action.inventory.burden_overloaded`

{You're} weighed down with everything {you're} carrying.

`if.action.inventory.carrying`

{You} {be} carrying:

`if.action.inventory.carrying_and_wearing`

{You} {be} carrying and wearing:

`if.action.inventory.carrying_count`

{You} {be} carrying {holdingCount} item(s).

`if.action.inventory.empty`

{You} aren't carrying anything.

`if.action.inventory.hands_empty`

{Your} hands are empty.

Message ID

`if.action.inventory.holding_list`
`if.action.inventory.inventory_empty`
`if.action.inventory.inventory_header`
`if.action.inventory.item_list`
`if.action.inventory.nothing_at_all`
`if.action.inventory.pockets_empty`
`if.action.inventory.wearing`
`if.action.inventory.wearing_count`
`if.action.inventory.worn_list`

Default text

{items}
{You} aren't carrying anything.
{You} {be} carrying:
{item}
{You} aren't carrying anything at all.
{Your} pockets are empty.
{You} {be} wearing:
{You} {be} wearing
{wearingCount}
item(s).
{items} (worn)

if.action.listening

Message ID

`if.action.listening.active_devices`
`if.action.listening.ambient_sounds`
`if.action.listening.container_sounds`
`if.action.listening.device_off`
`if.action.listening.device_running`

Default text

{You} can {hear}
{devices} operating nearby.
{You} {hear} the usual ambient sounds.
{You} {hear} faint sounds from inside {the:target}.
{the:cap:target} is silent.
{the:cap:target} is making a soft humming sound.

Message ID	Default text
<code>if.action.listening.liquid_sounds</code>	{You} {hear} liquid sloshing in {the:target}.
<code>if.action.listening.listened_environment</code>	{You} {listen} carefully.
<code>if.action.listening.listened_to</code>	{You} {listen} carefully to {the:target}.
<code>if.action.listening.no_sound</code>	{the:cap:target} isn't making any sound.
<code>if.action.listening.not_visible</code>	{You} {can't} see {the:target} well enough to focus on its sounds.
<code>if.action.listening.silence</code>	{You} {hear} nothing out of the ordinary.

if.action.locking

Message ID	Default text
<code>if.action.locking.already_locked</code>	{the:cap:item} is already locked.
<code>if.action.locking.cant_reach</code>	{You} {can't} reach {the:item}.
<code>if.action.locking.key_not_held</code>	{You} {need} to be holding {the:key}.
<code>if.action.locking.locked</code>	{You} {lock} {the:item}.
<code>if.action.locking.locked_with</code>	{You} {lock} {the:item} with {the:key}.
<code>if.action.locking.no_key</code>	What do {you} want to lock it with?
<code>if.action.locking.no_target</code>	Lock what?
<code>if.action.locking.not_closed</code>	{You} {need} to close {the:item} first.
<code>if.action.locking.not_lockable</code>	{the:cap:item} can't be locked.
<code>if.action.locking.wrong_key</code>	

Message ID

Default text

{the:cap:key} doesn't fit
{the:item}.

if.action.looking

Message ID

Default text

if.action.looking.container_contents	In {the:container} {you} {see} {items}.
if.action.looking.contents_list	{You} can {see} {items} here.
if.action.looking.exits	Exits: {exits}
if.action.looking.nothing_special	{You} {see} nothing special.
if.action.looking.room_dark	It's pitch dark, and {you} {can't} see a thing.
if.action.looking.room_description	{name} {description}
if.action.looking.surface_contents	On {the:surface} {you} {see} {items}.
if.action.looking.you_see	{You} can {see} {items} here.

if.action.lowering

Message ID

Default text

if.action.lowering.already_down	That's already lowered.
if.action.lowering.cant_lower_that	{You} {can't} lower {the:target}.
if.action.lowering.lowered	{You} {lower} {the:target}.
if.action.lowering.no_target	Lower what?

if.action.opening

Message ID	Default text
if.action.opening.already_open	{the:cap:item} is already open.
if.action.opening.cant_reach	{You} {can't} reach {the:item}.
if.action.opening.its_empty	{You} {open} {the:container}, which is empty.
if.action.opening.locked	{the:cap:item} is locked.
if.action.opening.no_target	Open what?
if.action.opening.not_openable	{the:cap:item} can't be opened.
if.action.opening.opened	{You} {open} {the:item}.
if.action.opening.revealing	Opening {the:container} reveals {items}.

if.action.pulling

Message ID	Default text
if.action.pulling.bell_rings	{You} {pull} {the:target}. A bell rings somewhere!
if.action.pulling.comes_loose	{You} {pull} {the:target} and it comes loose!
if.action.pulling.cord_activates	{You} {give} {the:target} a firm tug.
if.action.pulling.cord_pulled	{You} {pull} {the:target}.
if.action.pulling.firmly_attached	{You} {pull} {the:target}, but it's firmly attached.
if.action.pulling.fixed_in_place	{the:cap:target} is fixed in place.
if.action.pulling.lever_clicks	{You} {pull} {the:target} with a satisfying click.
if.action.pulling.lever_pulled	{You} {pull} {the:target}.

Message ID

`if.action.pulling.lever_toggled`

`if.action.pulling.no_target`

`if.action.pulling.not_reachable`

`if.action.pulling.not_visible`

`if.action.pulling.pulled_direction`

`if.action.pulling.pulled_nudged`

`if.action.pulling.pulled_with_effort`

`if.action.pulling.pulling_does_nothing`

`if.action.pulling.too_heavy`

`if.action.pulling.tugging_useless`

`if.action.pulling.wearing_it`

`if.action.pulling.wont_budge`

Default text

{You} {pull} {the:target},
switching it {newState}.

Pull what?

{You} {can't} reach
{the:target}.

{You} {can't} see
{the:target}.

{You} {pull} {the:target}
{direction}.

{You} {tug} at {the:target},
moving it slightly.

With effort, {you} {drag}
{the:target} {direction}.

Pulling {the:target} has no
effect.

{the:cap:target} is too
heavy to pull (weighs
{weight}kg).

Tugging on {the:target}
accomplishes nothing.

{You} {can't} pull
{the:target} while wearing
it.

{the:cap:target} won't
budge.

if.action.pushing

Message ID

`if.action.pushing.button_clicks`

Default text

{You} {press} {the:target}.
Click!

Message ID

`if.action.pushing.button_pushed`
`if.action.pushing.fixed_in_place`
`if.action.pushing.no_target`
`if.action.pushing.not_reachable`
`if.action.pushing.not_visible`
`if.action.pushing.pushed_direction`
`if.action.pushing.pushed_nudged`
`if.action.pushing.pushed_with_effort`
`if.action.pushing.pushing_does_nothing`
`if.action.pushing.reveals_passage`
`if.action.pushing.switch_toggled`
`if.action.pushing.too_heavy`
`if.action.pushing.wearing_it`
`if.action.pushing.wont_budge`

Default text

`{You} {push} {the:target}.`
`{the:cap:target}` is fixed in place.
Push what?
`{You} {can't} reach {the:target}.`
`{You} {can't} see {the:target}.`
`{You} {push} {the:target} {direction}.`
`{You} {give} {the:target}` a push, but it doesn't move far.
With considerable effort,
`{you} {push} {the:target} {direction}.`
Pushing `{the:target}` has no effect.
As `{you} {push} {the:target} {direction}`, it slides aside, revealing a hidden passage!
`{You} {push} {the:target}`, toggling it `{newState}`.
`{the:cap:target}` is far too heavy to push (weighs `{weight}kg`).
`{You} {can't} push {the:target}` while wearing it.

Message ID

Default text

{the:cap:target} won't
budge.

if.action.putting

Message ID

Default text

if.action.putting.already_there

{the:cap:item} is already
{relation} {the:destination}.

if.action.putting.cant_put_in_itself

{You} {can't} put {the:item}
inside itself.

if.action.putting.cant_put_on_itself

{You} {can't} put {the:item}
on itself.

if.action.putting.container_closed

{the:cap:container} is closed.

if.action.putting.no_destination

Where do {you} want to put
{the:item}?

if.action.putting.no_room

There's no room in
{the:container}.

if.action.putting.no_space

There's no space on
{the:surface}.

if.action.putting.no_target

Put what?

if.action.putting.not_container

{You} {can't} put things in
{the:destination}.

if.action.putting.not_held

{You} {need} to be holding
{the:item} first.

if.action.putting.not_surface

{You} {can't} put things on
{the:destination}.

if.action.putting.put_in

{You} {put} {the:item} in
{the:container}.

if.action.putting.put_on

{You} {put} {the:item} on
{the:surface}.

if.action.quitting

Message ID	Default text
if.action.quitting.achievements_earned	{You} earned {count} achievements during {your} play!
if.action.quitting.final_score	{Your} final score was {finalScore} out of {maxScore}.
if.action.quitting.final_stats	Final Statistics: Score: {finalScore}/{maxScore} Moves: {moves} Time played: {playTime}
if.action.quitting.quit_and_saved	Game saved. Thanks for playing! Final score: {finalScore} out of {maxScore} Moves: {moves}
if.action.quitting.quit_cancelled	Quit cancelled.
if.action.quitting.quit_confirm_query	Are {you} sure {you} want to quit?
if.action.quitting.quit_confirmed	Thanks for playing! Final score: {finalScore} out of {maxScore} Moves: {moves}
if.action.quitting.quit_save_query	Would {you} like to save before quitting?
if.action.quitting.quit_unsaved_query	{You} {have} unsaved progress. What would {you} like to do?

if.action.raising

Message ID	Default text
<code>if.action.raising.already_up</code>	That's already raised.
<code>if.action.raising.cant_raise_that</code>	{You} {can't} raise {the:target}.
<code>if.action.raising.no_target</code>	Raise what?
<code>if.action.raising.raised</code>	{You} {raise} {the:target}.

if.action.reading

Message ID	Default text
<code>if.action.reading.cannot_read_now</code>	{reason}
<code>if.action.reading.not_readable</code>	There's nothing written on {the:item}.
<code>if.action.reading.read_book</code>	{the:cap:item} reads: {text}
<code>if.action.reading.read_book_page</code>	{the:cap:item} (page {currentPage} of {totalPages}): {text}
<code>if.action.reading.read_inscription</code>	{the:cap:item} reads: {text}
<code>if.action.reading.read_sign</code>	{the:cap:item} says: {text}
<code>if.action.reading.read_text</code>	{the:cap:item} reads: {text}
<code>if.action.reading.what_to_read</code>	What do you want to read?

if.action.removing

Message ID	Default text
<code>if.action.removing.already_have</code>	{You} already {have} {the:item}.
<code>if.action.removing.cant_reach</code>	{You} {can't} reach {the:item}.
<code>if.action.removing.container_closed</code>	

Message ID

Default text

`if.action.removing.no_source`

{the:cap:container} is closed.

`if.action.removing.no_target`

Remove {the:item} from what?

`if.action.removing.not_in_container`

Remove what?

{the:cap:item} isn't in {the:container}.

`if.action.removing.not_on_surface`

{the:cap:item} isn't on {the:surface}.

`if.action.removing.removed_from`

{You} {take} {the:item} from {the:container}.

`if.action.removing.removed_from_surface`

{You} {take} {the:item} from {the:surface}.

if.action.restoring

Message ID

Default text

`if.action.restoring.available_saves`

Available saves: {saves}

`if.action.restoring.choose_save`

Which save would {you} like to restore?

`if.action.restoring.confirm_restore`

Restore game from '{saveName}'? Current progress will be lost.

`if.action.restoring.corrupt_save`

The save file '{saveName}' appears to be corrupted.

`if.action.restoring.game_loaded`

Game loaded from '{saveName}'.

`if.action.restoring.game_restored`

Game restored.

`if.action.restoring.import_save`

Import a save file to restore.

Message ID	Default text
<code>if.action.restoring.incompatible_save</code>	This save file is from a different version and cannot be loaded.
<code>if.action.restoring.no_saves</code>	No saved games found.
<code>if.action.restoring.no_saves_available</code>	No saved games available.
<code>if.action.restoring.quick_restore</code>	Quick restore completed.
<code>if.action.restoring.restore_details</code>	Restored: {saveName} Score: {score} Moves: {moves}
<code>if.action.restoring.restore_failed</code>	Failed to restore game.
<code>if.action.restoring.restore_not_allowed</code>	{You} cannot restore a game at this time.
<code>if.action.restoring.restore_successful</code>	{Your} saved game has been restored successfully.
<code>if.action.restoring.resuming_game</code>	Resuming {your} adventure...
<code>if.action.restoring.save_imported</code>	Save file imported successfully.
<code>if.action.restoring.save_not_found</code>	No save named '{saveName}' was found.
<code>if.action.restoring.unsaved_progress</code>	{You} {have} unsaved progress. Restore anyway?
<code>if.action.restoring.welcome_back</code>	Welcome back! Game restored from {saveName}.

if.action.revealing

Message ID

Default text

if.action.revealing.not_hidden {You're} not hiding.

if.action.revealing.revealed {You} {come} out of hiding.

if.action.saving

Message ID

Default text

if.action.saving.auto_save Auto-saving game...

if.action.saving.confirm_overwrite A save named '{saveName}' already exists. Overwrite it?

if.action.saving.game_saved Game saved.

if.action.saving.game_saved_as Game saved as '{saveName}'.

if.action.saving.invalid_save_name '{saveName}' is not a valid save name.

if.action.saving.no_save_slots No save slots available.

if.action.saving.overwrite_save Previous save '{saveName}' has been overwritten.

if.action.saving.quick_save Quick save completed.

if.action.saving.save_details Saved: {saveName} Score: {score} Moves: {moves}

if.action.saving.save_exported Save file exported successfully.

if.action.saving.save_failed Failed to save game.

if.action.saving.save_in_progress Another save is already in progress.

if.action.saving.save_not_allowed {You} cannot save the game at this time.

if.action.saving.save_reminder Don't forget to save {your} game regularly!

if.action.saving.save_slot

Message ID

Default text

`if.action.saving.save_successful`

Game saved to slot
{saveName}.

`if.action.saving.saved_locally`

{Your} game has been saved
successfully.

`if.action.saving.saved_to_cloud`

Game saved to local storage.

Game saved to cloud storage.

if.action.scoring

Message ID

Default text

`if.action.scoring.early_game`

{You're} just getting
started!

`if.action.scoring.game_complete`

{You} {have} completed the
game!

`if.action.scoring.late_game`

{You're} nearing the end of
{your} adventure.

`if.action.scoring.mid_game`

{You're} making good
progress.

`if.action.scoring.no_achievements`

{You} {have}n't earned any
special achievements yet.

`if.action.scoring.no_scoring`

This isn't that kind of game.

`if.action.scoring.perfect_score`

{You} {have} achieved a
perfect score of {maxScore}
points!

`if.action.scoring.rank_amateur`

{You} {are} ranked as an
Amateur adventurer.

`if.action.scoring.rank_expert`

{You} {are} ranked as an
Expert adventurer.

`if.action.scoring.rank_master`

{You} {are} ranked as a
Master adventurer!

Message ID	Default text
<code>if.action.scoring.rank_novice</code>	{You} {are} ranked as a Novice adventurer.
<code>if.action.scoring.rank_proficient</code>	{You} {are} ranked as a Proficient adventurer.
<code>if.action.scoring.score_display</code>	{You} {have} scored {score} out of a possible {maxScore}, in {moves} turns.
<code>if.action.scoring.score_simple</code>	{Your} score is {score} points.
<code>if.action.scoring.score_with_rank</code>	{You} {have} scored {score} out of {maxScore}, earning {you} the rank of {rank}.
<code>if.action.scoring.scoring_not_enabled</code>	There is no score in this game.
<code>if.action.scoring.with_achievements</code>	{You} {have} earned the following achievements: {achievements}.

if.action.searching

Message ID	Default text
<code>if.action.searching.container_closed</code>	{the:cap:target} is closed.
<code>if.action.searching.container_contents</code>	In {the:target} {you} {see}: {items}.
<code>if.action.searching.empty_container</code>	{the:cap:target} is empty.
<code>if.action.searching.found_concealed</code>	Hidden {where}, {you} {discover}: {items}.
<code>if.action.searching.found_items</code>	{You} {discover}: {items}.
<code>if.action.searching.not_reachable</code>	{You} {can't} reach {the:target} to search it.

Message ID	Default text
<code>if.action.searching.not_visible</code>	{You} {can't} see {the:target} to search it.
<code>if.action.searching.nothing_special</code>	{You} {find} nothing of interest.
<code>if.action.searching.searched_location</code>	{You} {search} around carefully.
<code>if.action.searching.searched_object</code>	{You} {search} {the:target} thoroughly.
<code>if.action.searching.supporter_contents</code>	On {the:target} {you} {see}: {items}.

if.action.showing

Message ID	Default text
<code>if.action.showing.no_item</code>	Show what?
<code>if.action.showing.no_viewer</code>	Show it to whom?
<code>if.action.showing.not_actor</code>	{You} can only show things to people.
<code>if.action.showing.not_carrying</code>	{You} aren't carrying {the:item}.
<code>if.action.showing.self</code>	{You} {examine} {the:item} closely.
<code>if.action.showing.shown</code>	{You} {show} {the:item} to {the:viewer}.
<code>if.action.showing.viewer_examines</code>	{the:cap:viewer} examines {the:item} carefully.
<code>if.action.showing.viewer_impressed</code>	{the:cap:viewer} looks impressed.
<code>if.action.showing.viewer_nods</code>	{the:cap:viewer} nods.
<code>if.action.showing.viewer_not_visible</code>	

Message ID**Default text**`if.action.showing.viewer_recognizes`

{You} {can't} see
{the:viewer}.
{the:cap:viewer} recognizes
{the:item}!

`if.action.showing.viewer_too_far`

{the:cap:viewer} is too far
away to see clearly.

`if.action.showing.viewer_unimpressed`

{the:cap:viewer} seems
unimpressed.

`if.action.showing.wearing_shown`

{You} {show} {the:viewer}
that {you're} wearing
{the:item}.

if.action.sleeping**Message ID****Default text**`if.action.sleeping.already_well_rested`

{You're} already well-
rested and don't feel
tired.

`if.action.sleeping.brief_nap`

{You} {take} a brief
nap.

`if.action.sleeping.cant_sleep_here`

{You} {can't} sleep in
{location}.

`if.action.sleeping.deep_sleep`

{You} {fall} into a
deep, restful sleep.

`if.action.sleeping.disturbed_sleep`

{Your} sleep is
disturbed.

`if.action.sleeping.dozed_off`

{You} {doze} off for a
bit.

`if.action.sleeping.fell_asleep`

{You} {fall} into a deep
sleep.

`if.action.sleeping.nightmares`

Message ID

Default text

	{You} {have} unsettling dreams.
<code>if.action.sleeping.peaceful_sleep</code>	{You} {enjoy} a peaceful sleep.
<code>if.action.sleeping.slept</code>	{You} {sleep} for a while.
<code>if.action.sleeping.slept_fitfully</code>	{You} {sleep} fitfully.
<code>if.action.sleeping.too_dangerous_to_sleep</code>	It's too dangerous to sleep in {location}.
<code>if.action.sleeping.woke_refreshed</code>	{You} {wake} feeling refreshed.

if.action.smelling

Message ID

Default text

<code>if.action.smelling.burning_scent</code>	{the:cap:target} gives off a smoky smell.
<code>if.action.smelling.container_food_scent</code>	{You} {smell} food inside {the:target}.
<code>if.action.smelling.drink_scent</code>	{the:cap:target} has a pleasant aroma.
<code>if.action.smelling.food_nearby</code>	{You} {smell} food nearby.
<code>if.action.smelling.food_scent</code>	{the:cap:target} smells delicious.
<code>if.action.smelling.fresh_scent</code>	{the:cap:target} smells fresh and clean.
<code>if.action.smelling.musty_scent</code>	{the:cap:target} smells a bit musty.
<code>if.action.smelling.no_particular_scent</code>	{the:cap:target} has no particular smell.

Message ID

`if.action.smelling.no_scent`

`if.action.smelling.not_visible`

`if.action.smelling.room_scents`

`if.action.smelling.smelled`

`if.action.smelling.smelled_environment`

`if.action.smelling.smoke_detected`

`if.action.smelling.too_far`

Default text

{You} {don't} smell anything unusual.

{You} {can't} see {the:target} to smell it.

The air carries various scents.

{You} {smell} {the:target}.

{You} {sniff} the air.

{You} {detect} a faint smell of smoke.

{the:cap:target} is too far away to smell.

if.action.switching_off

Message ID

`if.action.switching_off.already_off`

`if.action.switching_off.device_stops`

`if.action.switching_off.door_closes`

`if.action.switching_off.light_off`

`if.action.switching_off.light_off_still_lit`

Default text

{the:cap:target} is already off.

{the:cap:target} powers down with a soft whir.

{the:cap:target} switches off and closes.

{You} {switch} off {the:target}, plunging the area into darkness.

{You} {switch} off {the:target}.

Message ID

`if.action.switching_off.no_target`

`if.action.switching_off.not_reachable`

`if.action.switching_off.not_switchable`

`if.action.switching_off.not_visible`

`if.action.switching_off.silence_falls`

`if.action.switching_off.switched_off`

`if.action.switching_off.was_temporary`

`if.action.switching_off.with_sound`

Default text

Switch off what?

{You} {can't} reach {the:target}.

{the:cap:target} isn't something {you} can switch off.

{You} {can't} see {the:target}.

{You} {switch} off {the:target}. Silence falls.

{You} {switch} off {the:target}.

{the:cap:target} switches off (it had {remainingTime} seconds left).

{You} {switch} off {the:target}. {sound}

if.action.switching_on

Message ID

`if.action.switching_on.already_on`

`if.action.switching_on.device_humming`

`if.action.switching_on.door_opens`

`if.action.switching_on.illuminates_darkness`

Default text

{the:cap:target} is already on.

{the:cap:target} hums to life.

{the:cap:target} switches on and opens.

Message ID

Default text

`if.action.switching_on.light_on`

{the:cap:target}
switches on,
banishing the
darkness.

`if.action.switching_on.no_power`

{You} {switch} on
{the:target},
illuminating the
area.

`if.action.switching_on.no_target`

{the:cap:target} has
no power source.

`if.action.switching_on.not_reachable`

Switch on what?

`if.action.switching_on.not_switchable`

{You} {can't} reach
{the:target}.

`if.action.switching_on.not_visible`

{the:cap:target} isn't
something {you} can
switch on.

`if.action.switching_on.switched_on`

{You} {can't} see
{the:target}.

`if.action.switching_on.temporary_activation`

{You} {switch} on
{the:target}.
{the:cap:target}

`if.action.switching_on.with_sound`

switches on
temporarily.
{You} {switch} on
{the:target}. {sound}

if.action.taking

Message ID

Default text

`if.action.taking.already_have`

{You} already {have} {the:item}.

`if.action.taking.cannot_take`

{You} {can't} take {the:item}.

Message ID**Default text**

<code>if.action.taking.cant_take_room</code>	<code>{You} {can't} take {the:item}.</code>
<code>if.action.taking.cant_take_self</code>	<code>{You} {can't} take {yourself}.</code>
<code>if.action.taking.container_full</code>	<code>{You're} carrying too much already.</code>
<code>if.action.taking.fixed_in_place</code>	<code>{the:cap:item} is fixed in place.</code>
<code>if.action.taking.no_target</code>	<code>Take what?</code>
<code>if.action.taking.taken</code>	<code>Taken.</code>
<code>if.action.taking.taken_from</code>	<code>{You} {take} {the:item} from {the:container}.</code>
<code>if.action.taking.taken_multi</code>	<code>{item}: Taken.</code>
<code>if.action.taking.too_heavy</code>	<code>Your load is too heavy. You will have to leave something behind.</code>

if.action.taking_off**Message ID****Default text**

<code>if.action.taking_off.cant_remove</code>	<code>{You} {can't} take off {the:item}.</code>
<code>if.action.taking_off.no_target</code>	<code>Take off what?</code>
<code>if.action.taking_off.not_wearing</code>	<code>{You} aren't wearing {the:item}.</code>
<code>if.action.taking_off.prevents_removal</code>	<code>{You}'ll need to take off {the:blocking} first.</code>
<code>if.action.taking_off.removed</code>	<code>{You} {take} off {the:item}.</code>

if.action.talking**Message ID****Default text**

<code>if.action.talking.acknowledges</code>	<code>{the:cap:target} acknowledges {you}.</code>
---	---

Message ID	Default text
<code>if.action.talking.casual_greeting</code>	<code>{the:cap:target}</code> says, “Hey!”
<code>if.action.talking.first_meeting</code>	<code>{You}</code> <code>{introduce}</code> <code>{yourself}</code> to <code>{the:target}</code> .
<code>if.action.talking.formal_greeting</code>	<code>{the:cap:target}</code> says, “Good day to you.”
<code>if.action.talking.friendly_greeting</code>	<code>{the:cap:target}</code> smiles in recognition.
<code>if.action.talking.greets_again</code>	<code>{the:cap:target}</code> says, “Hello again.”
<code>if.action.talking.greets_back</code>	<code>{the:cap:target}</code> says, “Hello there!”
<code>if.action.talking.has_topics</code>	<code>{the:cap:target}</code> seems willing to discuss various topics.
<code>if.action.talking.no_response</code>	<code>{the:cap:target}</code> doesn’t respond.
<code>if.action.talking.no_target</code>	Talk to whom?
<code>if.action.talking.not_actor</code>	<code>{You}</code> can only talk to people.
<code>if.action.talking.not_available</code>	<code>{the:cap:target}</code> doesn’t want to talk right now.
<code>if.action.talking.not_visible</code>	<code>{You}</code> <code>{can’t}</code> see <code>{the:target}</code> .
<code>if.action.talking.nothing_to_say</code>	<code>{the:cap:target}</code> has nothing particular to say.
<code>if.action.talking.remembers_you</code>	<code>{the:cap:target}</code> says, “Ah, it’s you again.”
<code>if.action.talking.self</code>	Talking to <code>{yourself}</code> is a sign of madness.
<code>if.action.talking.talked</code>	<code>{You}</code> <code>{greet}</code> <code>{the:target}</code> .
<code>if.action.talking.too_far</code>	<code>{the:cap:target}</code> is too far away for conversation.

if.action.telling

Message ID

if.action.telling.already_knew

if.action.telling.bored

if.action.telling.dismissive

if.action.telling.grateful

if.action.telling.ignores

if.action.telling.informed

if.action.telling.interested

if.action.telling.no_target

if.action.telling.no_topic

if.action.telling.not_actor

if.action.telling.not_interested

if.action.telling.not_visible

if.action.telling.told

if.action.telling.too_far

if.action.telling.very_interested

Default text

{the:cap:target} says, “Yes, I’m aware of that.”

{the:cap:target} looks bored.

{the:cap:target} says, “So what?”

{the:cap:target} says, “Thank you for telling me!”

{the:cap:target} ignores what {you’re} saying.

{You} {inform} {the:target} about {topic}.

{the:cap:target} listens with interest.

Tell whom?

Tell them about what?

{You} can only tell things to people.

{the:cap:target} doesn’t seem interested.

{You} {can’t} see {the:target}.

{You} {tell} {the:target} about {topic}.

{the:cap:target} is too far away.

{the:cap:target} says, “Really? Tell me more!”

if.action.throwing

Message ID

Default text

if.action.throwing.bounces_off

{the:cap:item} bounces off {the:target}.

if.action.throwing.breaks_against

{the:cap:item} smashes against {the:target}!

if.action.throwing.breaks_on_impact

{the:cap:item} shatters on impact!

if.action.throwing.fragile_breaks

The fragile {item} breaks into pieces.

if.action.throwing.hits_target

{You} {throw} {the:item} at {the:target}. It hits!

if.action.throwing.lands_in

{the:cap:item} lands in {the:target}.

if.action.throwing.lands_on

{the:cap:item} lands on {the:target}.

if.action.throwing.misses_target

{You} {throw} {the:item} at {the:target}, but miss.

if.action.throwing.no_exit

There's no exit {direction}.

if.action.throwing.no_item

Throw what?

if.action.throwing.not_holding

{You} aren't holding {the:item}.

if.action.throwing.sails_through

{the:cap:item} sails through the exit to the {direction}.

if.action.throwing.self

{You} {can't} throw things at {yourself}.

if.action.throwing.target_angry

{the:cap:target} doesn't appreciate being hit with {the:item}.

if.action.throwing.target_catches

{the:cap:target} catches {the:item}!

Message ID**Default text**`if.action.throwing.target_ducks`

{the:cap:target} ducks out of the way.

`if.action.throwing.target_not_here`

{the:cap:target} isn't here.

`if.action.throwing.target_not_visible`

{You} {can't} see {the:target}.

`if.action.throwing.thrown`

{You} {throw} {the:item}.

`if.action.throwing.thrown_at`

{You} {throw} {the:item} at {the:target}.

`if.action.throwing.thrown_direction`

{You} {throw} {the:item} {direction}.

`if.action.throwing.thrown_down`

{You} {toss} {the:item} to the ground.

`if.action.throwing.thrown_gently`

{You} gently {toss} {the:item}.

`if.action.throwing.too_heavy`

{the:cap:item} is too heavy to throw far (weighs {weight}kg).

if.action.touching**Message ID****Default text**`if.action.touching.device_vibrating`

{the:cap:target} is vibrating slightly.

`if.action.touching.feels_cold`

{the:cap:target} feels cold.

`if.action.touching.feels_hard`

{the:cap:target} feels hard and solid.

`if.action.touching.feels_hot`

{the:cap:target} is hot! {You} {pull} {your} hand back quickly.

Message ID

`if.action.touching.feels_normal`

`if.action.touching.feels_rough`

`if.action.touching.feels_smooth`

`if.action.touching.feels_soft`

`if.action.touching.feels_warm`

`if.action.touching.feels_wet`

`if.action.touching.immovable_object`

`if.action.touching.liquid_container`

`if.action.touching.no_target`

`if.action.touching.not_reachable`

`if.action.touching.not_visible`

`if.action.touching.patted`

`if.action.touching.poked`

`if.action.touching.prodded`

`if.action.touching.stroked`

`if.action.touching.touched`

`if.action.touching.touched_gently`

Default text

{the:cap:target} feels as
{you}'d expect.

{the:cap:target} feels rough.

{the:cap:target} feels smooth.

{the:cap:target} feels soft.

{the:cap:target} feels warm to
the touch.

{the:cap:target} feels damp.

{the:cap:target} is solid and
immovable.

{You} {feel} liquid sloshing
inside {the:target}.

Touch what?

{You} {can't} reach
{the:target}.

{You} {can't} see {the:target}
to touch it.

{You} {pat} {the:target}.

{You} {poke} {the:target}.

{You} {prod} {the:target}.

{You} {stroke} {the:target}.

{You} {touch} {the:target}.

{You} gently {touch}
{the:target}.

if.action.turning

Message ID

`if.action.turning.cant_turn_that`

Default text

{the:cap:target} isn't
something {you} can turn.

Message ID

Default text

<code>if.action.turning.crank_turned</code>	<code>{You} {crank} {the:target}.</code>
<code>if.action.turning.dial_adjusted</code>	<code>{You} {adjust} {the:target}</code> <code>{direction}.</code>
<code>if.action.turning.dial_set</code>	<code>{You} {turn} {the:target} to</code> <code>{setting}.</code>
<code>if.action.turning.dial_turned</code>	<code>{You} {turn} {the:target}.</code>
<code>if.action.turning.flow_changes</code>	<code>{You} {turn} {the:target},</code> adjusting the flow.
<code>if.action.turning.key_needs_lock</code>	<code>{You} {need} to put</code> <code>{the:target} in a lock first.</code>
<code>if.action.turning.key_turned</code>	<code>{You} {turn} {the:target} in</code> the lock.
<code>if.action.turning.knob_clicks</code>	<code>{You} {turn} {the:target}</code> with a click.
<code>if.action.turning.knob_toggled</code>	<code>{You} {turn} {the:target},</code> switching it <code>{newState}</code> .
<code>if.action.turning.knob_turned</code>	<code>{You} {turn} {the:target}.</code> As <code>{you} {turn}</code>
<code>if.action.turning.mechanism_activated</code>	<code>{the:target}, {you} {hear}</code> machinery activate! <code>{You} {turn} {the:target}.</code>
<code>if.action.turning.mechanism_grinds</code>	Gears grind and machinery moves.
<code>if.action.turning.no_target</code>	Turn what?
<code>if.action.turning.not_reachable</code>	<code>{You} {can't} reach</code> <code>{the:target}.</code>
<code>if.action.turning.not_visible</code>	<code>{You} {can't} see</code> <code>{the:target}.</code>
<code>if.action.turning.nothing_happens</code>	<code>{You} {turn} {the:target},</code> but nothing happens.
<code>if.action.turning.requires_more_turns</code>	

Message ID

Default text

`if.action.turning.rotated`
`if.action.turning.spun`
`if.action.turning.turned`
`if.action.turning.valve_closed`
`if.action.turning.valve_opened`
`if.action.turning.wearing_it`
`if.action.turning.wheel_turned`

{You} {turn} {the:target}. It seems to need more turning.
{You} {rotate} {the:target}.
{You} {spin} {the:target}.
{You} {turn} {the:target}.
{You} {turn} {the:target}, closing the valve.
{You} {turn} {the:target}, opening the valve.
{You} {can't} turn {the:target} while wearing it.
{You} {turn} {the:target}.

if.action.undoing

Message ID

Default text

`if.action.undoing.nothing_to_undo` Nothing to undo.
`if.action.undoing.undo_failed` Undo failed.
`if.action.undoing.undo_success` Previous turn undone.
`if.action.undoing.undo_to_turn` Undone. (Now at turn {turn})

if.action.unlocking

Message ID

Default text

`if.action.unlocking.already_unlocked` {the:cap:item} is already unlocked.
`if.action.unlocking.cant_reach` {You} {can't} reach {the:item}.

Message ID**Default text**`if.action.unlocking.key_not_held`

{You} {need} to be holding
{the:key}.

`if.action.unlocking.no_key`

What do {you} want to
unlock it with?

`if.action.unlocking.no_target`

Unlock what?

`if.action.unlocking.not_lockable`

{the:cap:item} can't be
unlocked.

`if.action.unlocking.still_locked`

{the:cap:item} is locked.

`if.action.unlocking.unlocked`

{You} {unlock} {the:item}.

`if.action.unlocking.unlocked_with`

{You} {unlock} {the:item}
with {the:key}.

`if.action.unlocking.wrong_key`

{the:cap:key} doesn't fit
{the:item}.

if.action.version**Message ID****Default text**`if.action.version.version_compact`

{storyTitle} v{storyVersion}
(Sharpee v{engineVersion})

`if.action.version.version_full`

{storyTitle} v{storyVersion}

Sharpee Engine

v{engineVersion} Built:

{buildDate}

{storyTitle} v{storyVersion}

`if.action.version.version_no_date`

Sharpee Engine

v{engineVersion}

if.action.waiting

Message ID	Default text
if.action.waiting.grows_restless	{You} {grow} restless from waiting.
if.action.waiting.nothing_happens	{You} {wait}. Nothing happens.
if.action.waiting.patience_rewarded	{Your} patience is rewarded.
if.action.waiting.something_approaches	As {you} {wait}, {you} {hear} something approaching.
if.action.waiting.time_passes	Time passes...
if.action.waiting.time_runs_out	{You}'ve waited too long!
if.action.waiting.waited	Time passes.
if.action.waiting.waited_anxiously	{You} {wait} anxiously.
if.action.waiting.waited_briefly	{You} {wait} for a moment.
if.action.waiting.waited_for_event	{You} {wait} for something to happen.
if.action.waiting.waited_in_vehicle	{You} {wait} in {the:vehicle}.
if.action.waiting.waited_patiently	{You} {wait} patiently.

if.action.wearing

Message ID	Default text
if.action.wearing.already_wearing	{You're} already wearing {the:item}.
if.action.wearing.cant_wear_that	{You} {can't} wear {the:item}.
if.action.wearing.hands_full	

Message ID

Default text

	{You} {need} to have {your} hands free to put that on.
if.action.wearing.no_target	Wear what?
if.action.wearing.not_held	{You} {need} to be holding {the:item} first.
if.action.wearing.not_wearable	{You} {can't} wear {the:item}.
if.action.wearing.worn	{You} {put} on {the:item}.

if.platform

Message ID

Default text

if.platform.prompt >

npc

Message ID

Default text

npc.arrives	{npcName} arrives.
npc.attacks	{npcName} attacks you!
npc.confused	{npcName} looks confused.
npc.cries	{npcName} cries.
npc.departs	{npcName} departs.
npc.drops	{npcName} drops {itemName}.
npc.enters	{npcName} enters from the {direction}.
npc.farewell	{npcName} bids you farewell.
npc.follows	{npcName} follows you.
npc.greets	{npcName} greets you.
npc.growls	{npcName} growls menacingly.
npc.hits	{npcName} hits you for {damage} damage!
npc.ignores_player	{npcName} ignores you.

Message ID	Default text
<code>npc.killed</code>	<code>{npcName}</code> has been slain.
<code>npc.laughs</code>	<code>{npcName}</code> laughs.
<code>npc.leaves</code>	<code>{npcName}</code> leaves to the <code>{direction}</code> .
<code>npc.misses</code>	<code>{npcName}</code> swings at you but misses!
<code>npc.mutters</code>	<code>{npcName}</code> mutters, “ <code>{text}</code> ”
<code>npc.no_response</code>	<code>{npcName}</code> doesn’t respond.
<code>npc.notices_player</code>	<code>{npcName}</code> notices you.
<code>npc.shouts</code>	<code>{npcName}</code> shouts, “ <code>{text}</code> ”
<code>npc.sighs</code>	<code>{npcName}</code> sighs.
<code>npc.speaks</code>	<code>{npcName}</code> says, “ <code>{text}</code> ”
<code>npc.takes</code>	<code>{npcName}</code> picks up <code>{itemName}</code> .
<code>npc.unconscious</code>	<code>{npcName}</code> collapses, unconscious.
<code>npc.whispers</code>	<code>{npcName}</code> whispers, “ <code>{text}</code> ”

npc.combat.attack

Message ID	Default text
<code>npc.combat.attack.hit</code>	The axe gets you right in the side. Ouch!
<code>npc.combat.attack.hit_heavy</code>	The troll hits you with a glancing blow, and you are momentarily stunned.
<code>npc.combat.attack.hit_light</code>	The flat of the troll’s axe skins across your forearm.
<code>npc.combat.attack.killed</code>	The troll lands a killing blow. You are dead.
<code>npc.combat.attack.knocked_out</code>	The flat of the troll’s axe hits you delicately on the head, knocking you out.
<code>npc.combat.attack.missed</code>	

Message ID

Default text

The troll swings his axe, but it misses.

npc.guard

Message ID

Default text

npc.guard.attacks {npcName} attacks you!

npc.guard.blocks {npcName} blocks your way!

npc.guard.defeated {npcName} is no longer a threat.

sound.heard.ambient

Message ID

Default text

sound.heard.ambient.fragments {You} {catch} the faint sound of {kind}.

sound.heard.ambient.full {You} {hear} {kind}.

sound.heard.ambient.muffled {You} {hear} a muffled {kind}.

sound.heard.ambient.presence-only {You} {hear} something at the edge of hearing.

sound.heard.default

Message ID

Default text

sound.heard.default.fragments {You} {catch} broken {kind}.

sound.heard.default.full {You} {hear} {kind}.

sound.heard.default.muffled {You} {hear} a muffled {kind}.

sound.heard.default.presence-only {You} {hear} something distant.

sound.heard.speech

Message ID

sound.heard.speech.fragments

sound.heard.speech.full

sound.heard.speech.muffled

sound.heard.speech.presence-
only

Default text

{You} {catch} fragments of speech.

{You} {hear}: "{content}"

{You} {catch} a muffled voice:
"{content}"

{You} {hear} voices nearby.

Appendix E — Grammar Reference

The core grammar patterns the English parser (@sharpee/parser-en-us) ships, generated from the platform’s grammar definitions; 118 core rules. Stories add their own with `extendParser` (Volume V); the parser also auto-generates a `verb [object]` pattern for every registered verb (see Appendix B for verb phrasings).

31.11 Pattern syntax

Token	Meaning
<code>word</code>	a literal word the player types
<code>a\ b</code>	alternatives — any one of the listed words
<code>:slot</code>	an object slot — matches an entity the player names (<code>:target</code> , <code>:item</code> , ...)
<code>[word]</code>	an optional word (skipping it costs a little match confidence)

Priority orders competing rules (higher wins): 100+ for semantic rules with trait constraints, 100 standard, 95 synonyms, 90 abbreviations.

Constraint shows trait requirements on a slot (e.g. `container` → `container`) that make a rule match only suitable objects.

31.12 Core grammar rules

Pattern	Action	Priority	Constraint
<code>look\ l</code>	<code>if.action.looking</code>	—	—
<code>examine\ x\ inspect :target</code>	<code>if.action.examing</code>	—	—

Pattern	Action	Priority	Constraint
look at :target	if.action.examining	95	—
look [carefully] at :target	if.action.examining_carefully	96	—
look [around]	if.action.looking	101	—
search [carefully]	if.action.searching	100	—
search :target	if.action.searching	100	—
look in\ inside :target	if.action.searching	100	—
look through :target	if.action.searching	100	—
rummage in\ through :target	if.action.searching	95	—
take\ get\ grab :item	if.action.taking	—	—
pick up :item	if.action.taking	100	—
drop\ discard :item	if.action.dropping	—	—
put down :item	if.action.dropping	100	—
eat\ consume\ devour :item	if.action.eating	—	—
drink\ sip\ quaff :item	if.action.drinking	—	—
put :item in\ into\ inside :container	if.action.inserting	100	container → container
insert :item in\ into :container	if.action.inserting	100	container → container
put :item on\ onto :supporter	if.action.putting	100	supporter → supporter
hang :item on :hook	if.action.putting if.action.reading	110 —	— —

Pattern	Action	Priority	Constraint
read\ peruse\ study :target			
inventory\ inv\ i	if.action.inventory	—	—
go :direction	if.action.going	100	direction → direction
a bare direction (north, n, up, ...)	if.action.going	—	directions
open :door	if.action.opening	100	door → openable
close :door	if.action.closing	100	door → openable
turn\ switch\ flip on :device	if.action.switching_on	—	device → switchable
turn\ switch\ flip off :device	if.action.switching_off	—	device → switchable
turn :device on	if.action.switching_on	—	device → switchable
turn :device off	if.action.switching_off	—	device → switchable
push\ press\ shove\ move :target	if.action.pushing	—	—
pull\ drag\ yank :target	if.action.pulling	—	—
lower :target	if.action.lowering	—	—
raise\ lift :target	if.action.raising	—	—
wait\ z	if.action.waiting	—	—
save	if.action.saving	100	—
restore	if.action.restoring	100	—
restart	if.action.restarting	100	—

Pattern	Action	Priority	Constraint
quit\ q	if.action.quitting	—	—
undo	if.action.undoing	100	—
score	if.action.scoring	100	—
version	if.action.version	100	—
help	if.action.help	100	—
about	if.action.about	100	—
info	if.action.about	100	—
credits	if.action.about	100	—
trace	author.trace	100	—
trace on	author.trace	100	—
trace off	author.trace	100	—
trace parser on	author.trace	100	—
trace parser off	author.trace	100	—
trace validation on	author.trace	100	—
trace validation off	author.trace	100	—
trace system on	author.trace	100	—
trace system off	author.trace	100	—
trace all on	author.trace	100	—
trace all off	author.trace	100	—
give :item to :recipient	if.action.giving	100	recipient → actor
give :recipient :item	if.action.giving	95	recipient → actor
offer :item to :recipient	if.action.giving	100	recipient → actor
show :item to :recipient	if.action.showing	100	recipient → actor
show :recipient :item	if.action.showing	95	recipient → actor

Pattern	Action	Priority	Constraint
throw :item at :target	if.action.throwing	100	—
throw :item to :recipient	if.action.throwing	100	—
take :item from :container with\ using :tool	if.action.taking_with	110	instrument
unlock :door with\ using :key	if.action.unlocking	110	instrument
open :container with\ using :tool	if.action.opening_with	110	instrument; container → openable
cut :object with\ using :tool	if.action.cutting	110	instrument
attack\ kill\ fight\ slay\ murder\ hit\ strike :target	if.action.attacking	—	—
attack :target with\ using :weapon	if.action.attacking	110	instrument
kill :target with\ using :weapon	if.action.attacking	110	instrument
hit :target with\ using :weapon	if.action.attacking	110	instrument
strike :target with\ using :weapon	if.action.attacking	110	instrument
dig :location with\ using :tool	if.action.digging	110	instrument
say :message	if.action.saying	100	—
say :message to :recipient	if.action.saying_to	105	recipient → actor

Pattern	Action	Priority	Constraint
write :message	if.action.writing	100	—
write :message on :surface	if.action.writing_on	105	—
shout :message	if.action.shouting	100	—
whisper :message to :recipient	if.action.whispering	100	recipient → actor
tell :recipient about :topic	if.action.telling	100	recipient → actor
ask :recipient about :topic	if.action.asking	100	recipient → actor
touch\ rub\ feel\ pat\ stroke\ poke\ prod :target	if.action.touching	—	—
enter :portal	if.action.entering	100	portal → enterable
get in :portal	if.action.entering	100	portal → enterable
get into :portal	if.action.entering	100	portal → enterable
climb in :portal	if.action.entering	100	portal → enterable
climb into :portal	if.action.entering	100	portal → enterable
go in :portal	if.action.entering	100	portal → enterable
go into :portal	if.action.entering	100	portal → enterable
exit	if.action.exiting	100	—
get out	if.action.exiting	100	—
leave	if.action.exiting	95	—

Pattern	Action	Priority	Constraint
climb out	if.action.exiting	100	—
board :vehicle	if.action.entering	100	vehicle → enterable
get on :vehicle	if.action.entering	100	vehicle → enterable
exit :container	if.action.exiting	100	container → enterable
disembark	if.action.exiting	100	—
disembark :vehicle	if.action.exiting	100	vehicle → enterable
get off :vehicle	if.action.exiting	100	vehicle → enterable
alight	if.action.exiting	95	—
again	if.action.again	100	—
g	if.action.again	90	—
hide behind :target	if.action.hiding	100	—
duck behind :target	if.action.hiding	100	—
crouch behind :target	if.action.hiding	100	—
hide under :target	if.action.hiding	100	—
duck under :target	if.action.hiding	100	—
crouch under :target	if.action.hiding	100	—
hide on :target	if.action.hiding	100	—
hide in :target	if.action.hiding	100	—
hide inside :target	if.action.hiding	100	—
duck inside :target	if.action.hiding	100	—
stand up	if.action.revealing	100	—
come out	if.action.revealing	100	—
reveal myself	if.action.revealing	100	—

Pattern	Action	Priority	Constraint
unhide	if.action.revealing	100	—
stop hiding	if.action.revealing	100	—